嵌入式 Linux 应用开发实验班(快速入门)
（V1.1）

深圳百问网科技有限公司

2025.4.29

第1章 课程目标

1.1 快！做出产品！

1.1.1 为什么要快

嵌入式开发中，基础知识太多了，学完 ABCDEFG，你都没办法做出一个最简单的产品：
① 在屏幕上点击图标控制 LED
② 在屏幕上显示温湿度的值
③ 更进一步，使用微信远程控制这个产品。

我们就以这个功能为例，抛弃暂时没必要深究的知识，把产品做出来。然后你在它的基础上扩展功能：该补什么知识就去补。

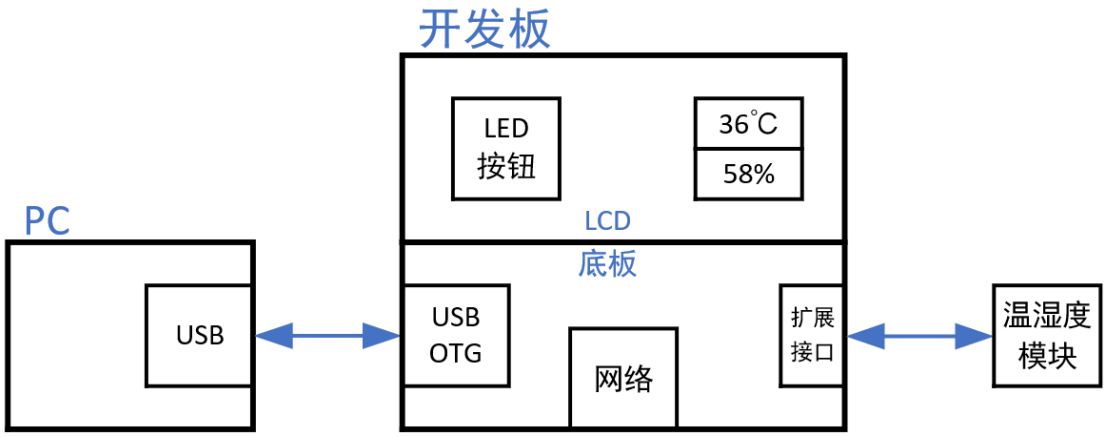
1.1.2 怎么快

别自己装 Ubuntu 了，用我们的！
别在开发板上用网线了，用 USB！
别学什么 Makefile 了，用我们的模板！
别学习太多、别深究太多，跟着我们的步骤学习！
入门之后，你再去深究、探索。

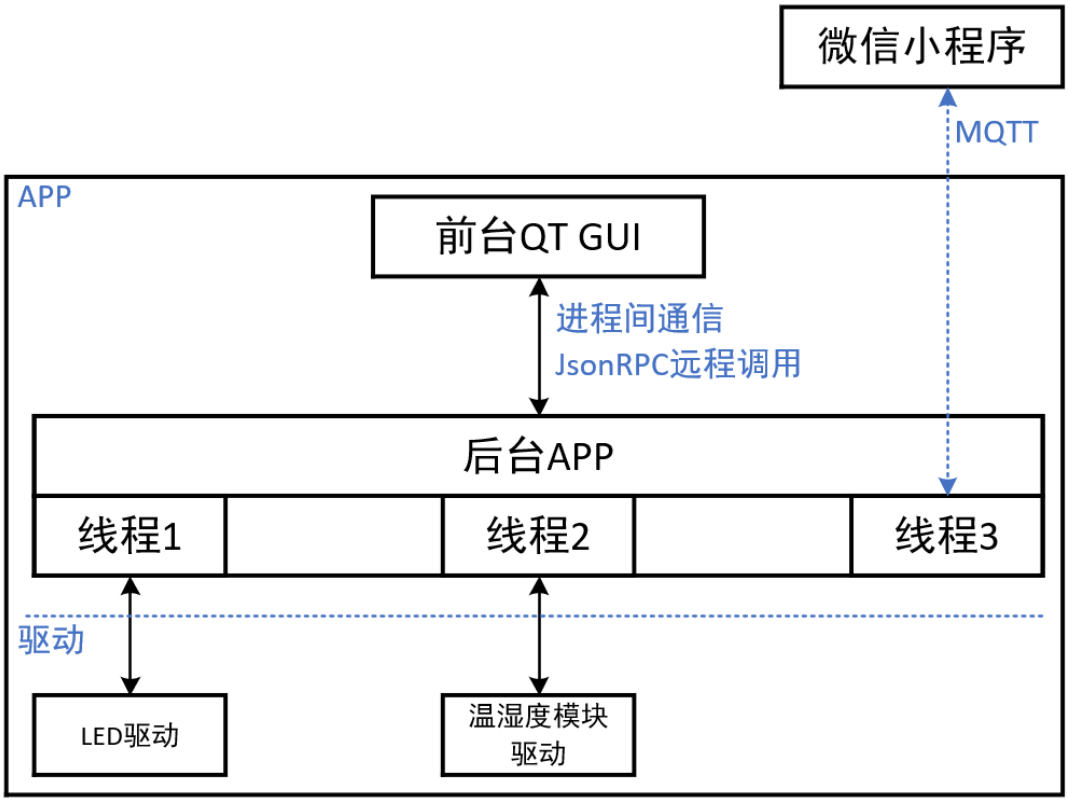
1.2 做什么产品

1.2.1 硬件框图

功能为：
① 在屏幕上点击图标控制 LED
② 在屏幕上显示温湿度的值
③ 更进一步，使用微信远程控制这个产品。



1.2.2 软件结构



1.3 上机演示

第2章　搭建开发环境

2.1 下载网盘资料

打开百问网下载中心，在左侧找到你的开发板，下载网盘资料：



2.2 安装软件

安装如下软件：



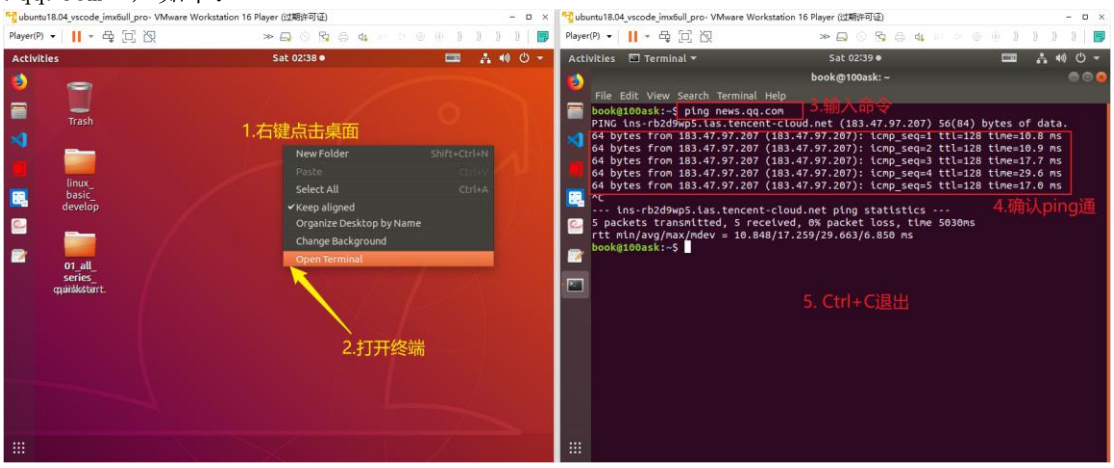
2.3 体验 Ubuntu

2.3.1 确保使用 NAT 网卡



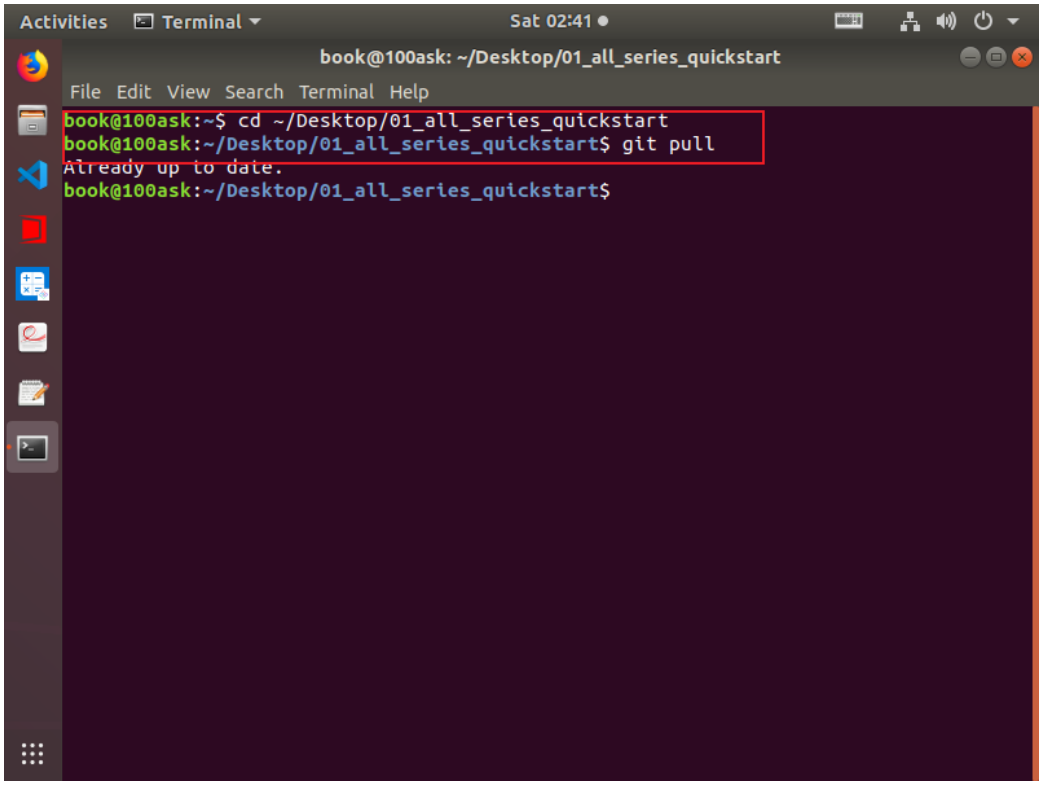
2.3.2 验证网络

启动虚拟机后（用户名 book，密码 123456），在终端里确认可以上网“ping news.qq.com”，如下：

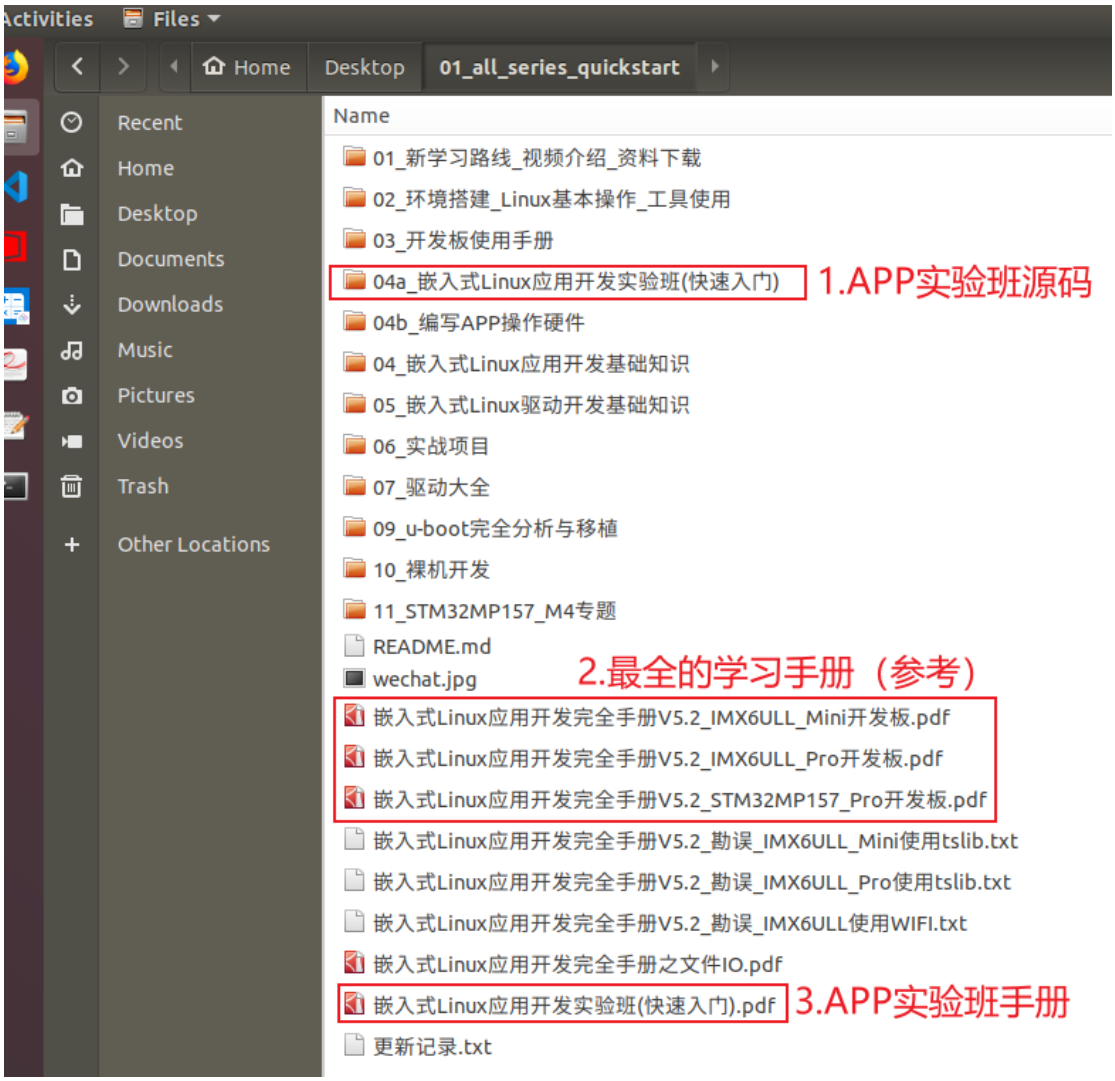


2.3.3 更新 GIT 仓库

在终端里进入“~/Desktop/01_all_series_quickstart”目录，执行“git pull”命令：

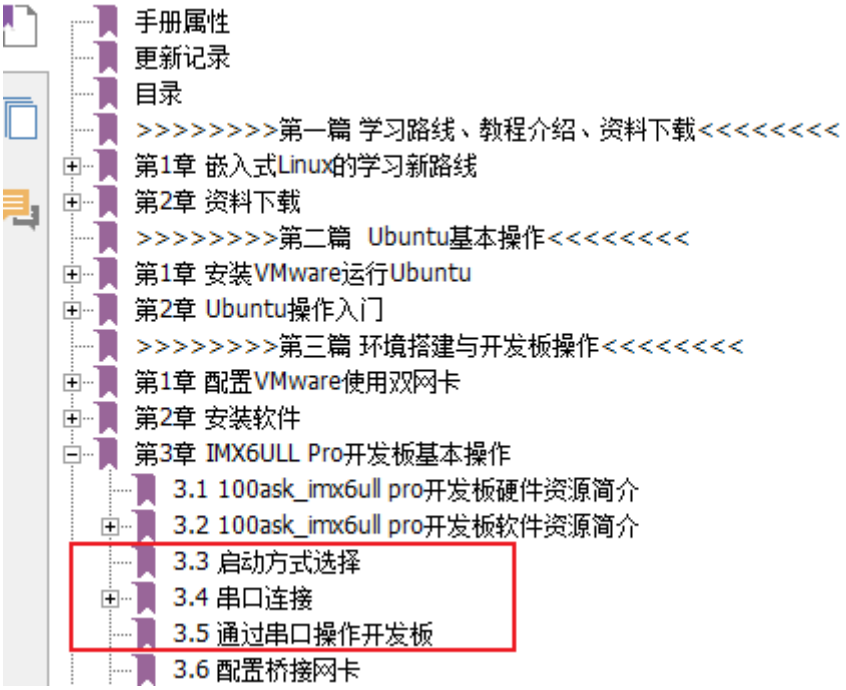


2.4 配套资料


Activities
Files
Home
Desktop
01_all_series_quickstart
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Other Locations
Name
01_新学习路线_视频介绍_资料下载
02_环境搭建_Linux基本操作_工具使用
03_开发板使用手册
04a_嵌入式Linux应用开发实验班(快速入门)
1.APP实验班源码
04b_编写APP操作硬件
04_嵌入式Linux应用开发基础知识
05_嵌入式Linux驱动开发基础知识
06_实战项目
07_驱动大全
09_u-boot完全分析与移植
10_裸机开发
11_STM32MP157_M4专题
README.md
wechat.jpg
2.最全的学习手册（参考）
嵌入式Linux应用开发完全手册V5.2_IMX6ULL_Mini开发板.pdf
嵌入式Linux应用开发完全手册V5.2_IMX6ULL_Pro开发板.pdf
嵌入式Linux应用开发完全手册V5.2_STM32MP157_Pro开发板.pdf
嵌入式Linux应用开发完全手册V5.2_勘误_IMX6ULL_Mini使用tslib.txt
嵌入式Linux应用开发完全手册V5.2_勘误_IMX6ULL_Pro使用tslib.txt
嵌入式Linux应用开发完全手册V5.2_勘误_IMX6ULL使用WIFI.txt
嵌入式Linux应用开发完全手册之文件IO.pdf
嵌入式Linux应用开发实验班(快速入门).pdf
3.APP实验班手册
更新记录.txt

2.5 连接开发板

参考《嵌入式 Linux 应用开发完全手册 V5.2_xxx 开发板》如下章节：



2.6 使用 ADB

2.6.1 给 Ubuntu 添加 adb 的支持

你使用的 Ubuntu 可能已经支持 adb，如果执行下面命令发现有文件“/usr/bin/adb”的话，就不用再安装：

```
ls /usr/bin/adb
```

如果没有“/usr/bin/adb”，则在 Ubuntu 中执行以下命令安装 adb：

```
sudo apt install adb
sudo chmod a+x /usr/bin/adb
sudo chmod a+s /usr/bin/adb
sudo adb devices
sudo adb kill-server
```

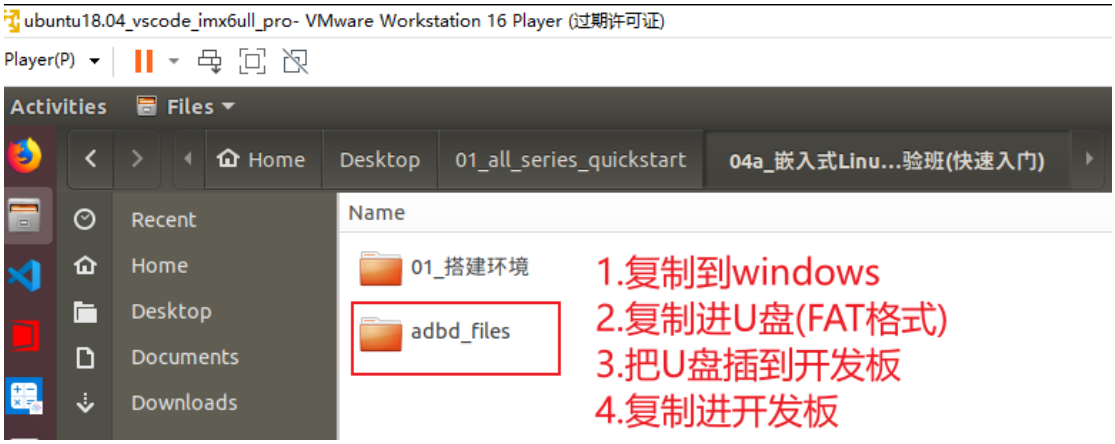
2.6.2 给开发板添加 adb 的支持

IMX6ULL 里“可能已经支持 adb”，那么就不用按照本节操作。

怎么确定 IMX6ULL 使用已经支持 adb？在开发板执行如下命令，如果有这个文件就表示支持 adb 了：

```
ls /usr/bin/adb
```

在 Ubuntu 中，有如下文件：



把 adbd_files 下的文件，全部复制到开发板根目录。

① 先把 U 盘格式化为 FAT 格式



② 把 Ubuntu 里面的“adbd_files”整个目录复制到 U 盘

③ 把 U 盘插到开发板上，会在串口打印类似如下的提示信息：

```
[root@100ask:/]# [91631.354919] usb 1-1.3: new high-speed USB device number 5 using ci_hdrc
[91631.532679] usb-storage 1-1.3:1.0: USB Mass Storage device detected
[91631.579097] scsi host0: usb-storage 1-1.3:1.0
[91632.628942] scsi 0:0:0:0: Direct-Access     Kingston DataTraveler 3.0 PMAP PQ: 0 ANSI: 6
[91632.680020] sd 0:0:0:0: [sda] 242417664 512-byte logical blocks: (124 GB/116 GiB)
[91632.706895] sd 0:0:0:0: [sda] Write Protect is off
[91632.716931] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[91634.335530]  sda: sda1
[91634.360998] sd 0:0:0:0: [sda] Attached SCSI removable disk
```

然后在开发板上执行 mount 命令，如下：

```
mount  /dev/sda1  /mnt
```

④ 复制文件进开发板：

执行如下命令：

```
cd  /mnt/adbd_files
cp * / -rf

dos2unix /lib/udev/rules.d/61-usb-adbd.rules
dos2unix /usr/bin/usb_config
dos2unix /etc/init.d/S99adbd

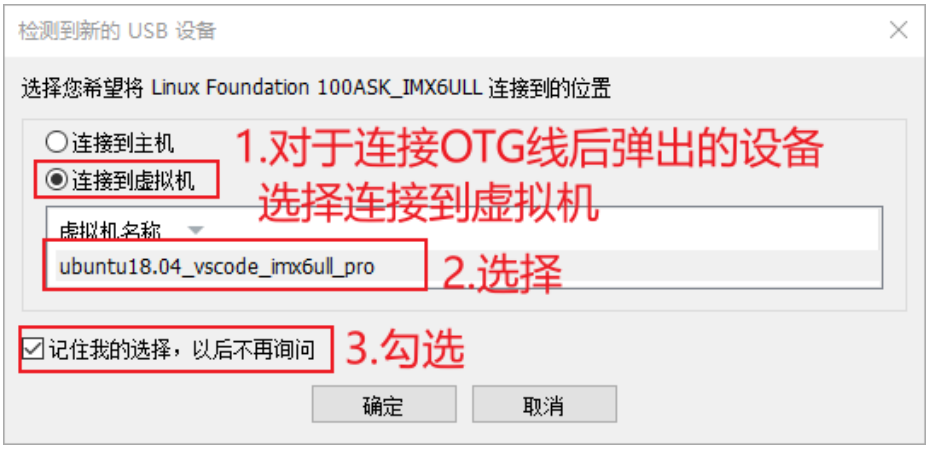
chmod +x /usr/bin/adb
chmod +x /usr/bin/adbd
chmod +x /usr/bin/usb_config
chmod +x  /etc/init.d/S99adbd

reboot
```

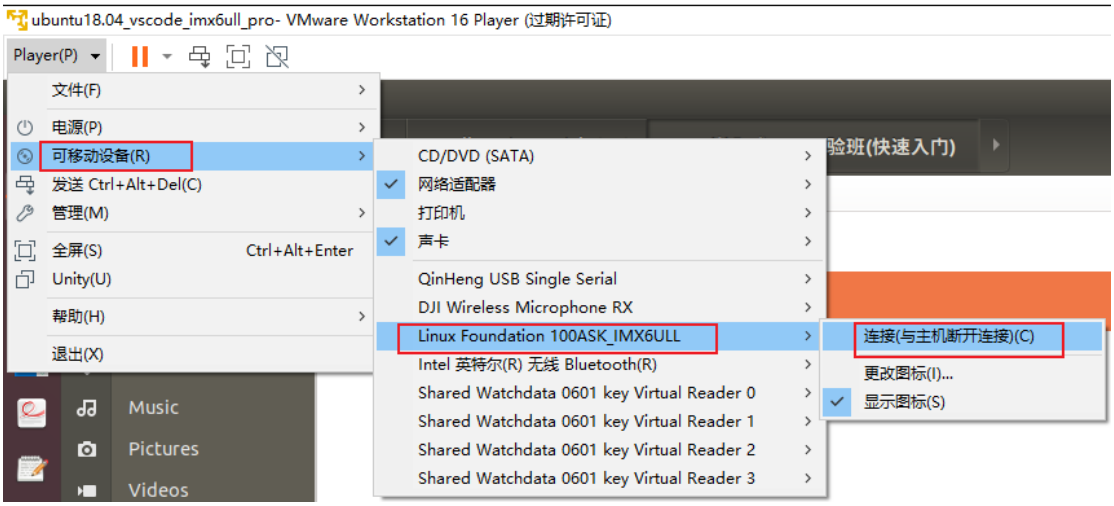
最后重启。

2.6.3 使用 adb

使用 USB 线连接开发板的 OTG 口和 PC，在 VMWare 里选择把新设备连接到 Ubuntu，如下：



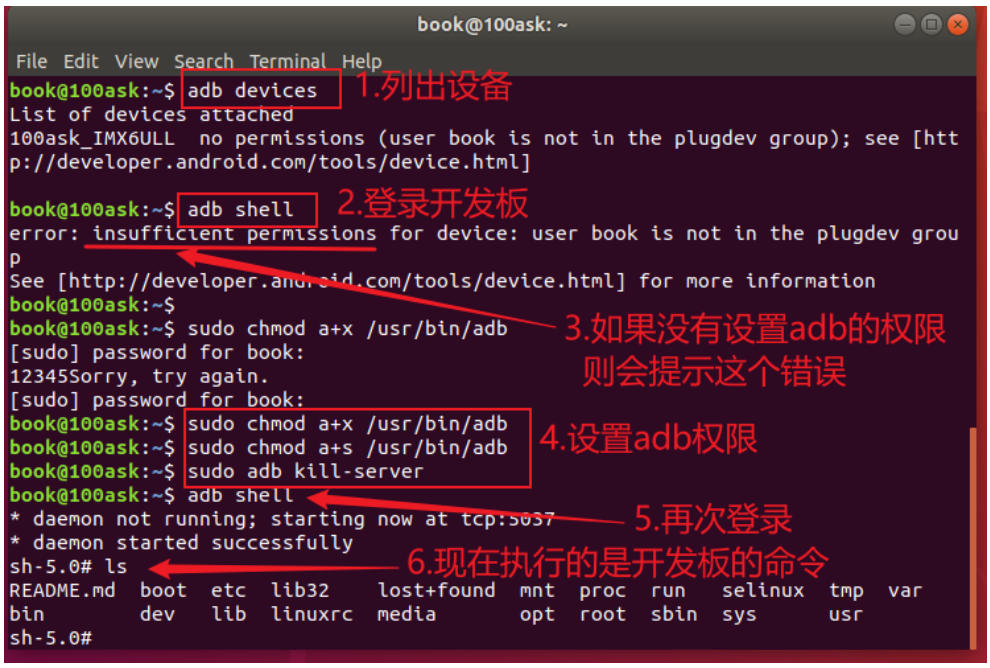
如果没有弹出上述界面，也可以如下操作：



然后就可以在 Ubuntu 上使用 adb 操作开发板了：

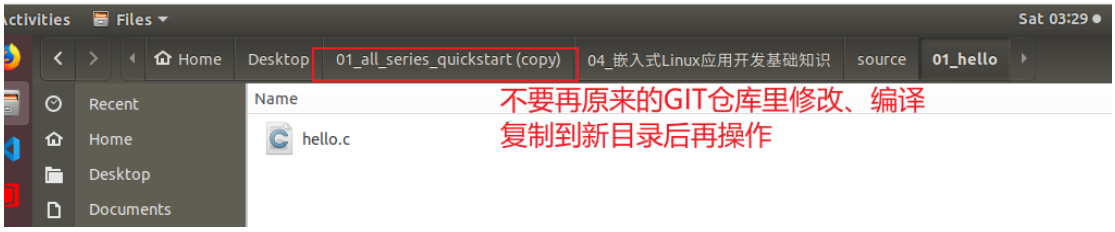
```
adb devices
adb push 1.txt /root       # 把Ubuntu的文件放到开发板的/root目录
adb pull /root/1.txt 2.tx # 把开发板的/root/1.txt下载并改名为2.txt
adb shell                  # 登录开发板
```

如下图所示：


book@100ask: ~
File Edit View Search Terminal Help
book@100ask:~$ adb devices
1.列出设备
List of devices attached
100ask_IMX6ULL no permissions (user book is not in the plugdev group); see [htt
p://developer.android.com/tools/device.html]
book@100ask:~$ adb shell
2.登录开发板
error: insufficient permissions for device: user book is not in the plugdev grou
p
See [http://developer.android.com/tools/device.html] for more information
book@100ask:~$
book@100ask:~$ sudo chmod a+x /usr/bin/adb
3.如果没有设置adb的权限
则会提示这个错误
[sudo] password for book:
12345Sorry, try again.
[sudo] password for book:
book@100ask:~$ sudo chmod a+x /usr/bin/adb
book@100ask:~$ sudo chmod a+s /usr/bin/adb
4.设置adb权限
book@100ask:~$ sudo adb kill-server
book@100ask:~$ adb shell
5.再次登录
* daemon not running; starting now at tcp:5037
* daemon started successfully
6.现在执行的是开发板的命令
sh-5.0# ls
README.md boot etc lib32 lost+found mnt proc run selinux tmp var
bin dev lib linuxrc media opt root sbin sys usr
sh-5.0#

2.7 体验第 1 个程序

本节源码为：



在 Ubuntu 上编译：

```
$ arm-buildroot-linux-gnueabihf-gcc -o hello hello.c
$ adb push hello  /root
```

然后就可以执行开发板下的“/root/hello”程序了。
如下图所示：



第3章 QT GUI 程序开发

3.1 Ubuntu 操作

3.1.1 设置交叉编译工具链

以 100ASK_IMX6ULL_Pro 为例，在本课程提供的 Ubuntu 里已经设置好了工具链，无需再次设置。如果你使用其他开发板，则仿照下面的命令进行设置：

```
export ARCH=arm
export CROSS_COMPILE=arm-buildroot-linux-gnueabihf-
export PATH=$PATH:/home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin
```

3.1.2 编译支持 QT 的系统

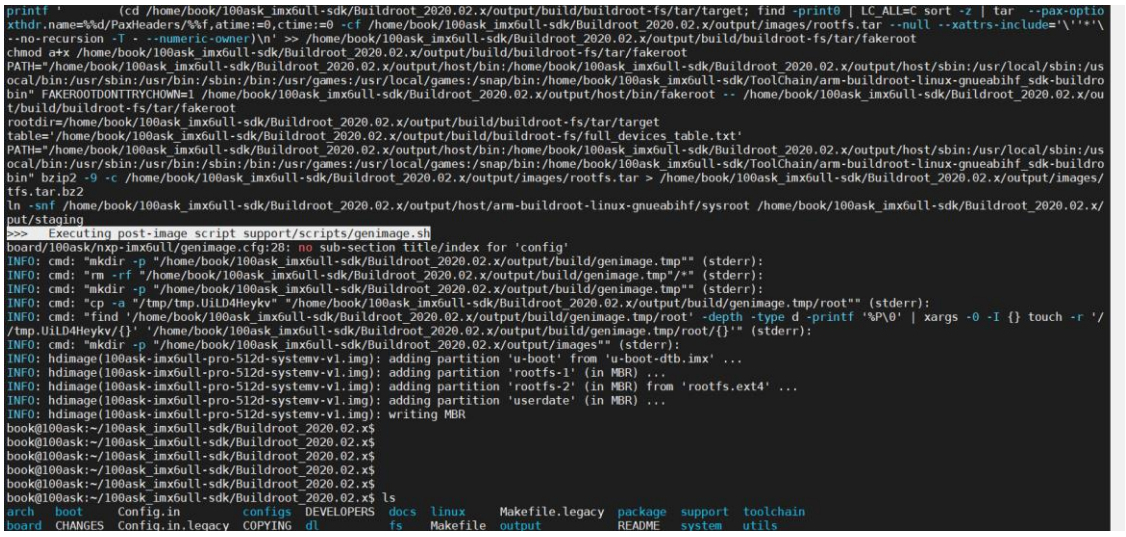
以 100ASK_IMX6ULL_Pro 为例，执行如下命令：

① 先更新 repo：编译 QT 系统时需要从网上下载很多软件包，有些软件包在国外网站不容易下载，我们把这些软件包放到了国内的网站，需要跟新一下 repo 仓库。执行如下命令：

```
cd /home/book/100ask_imx6ull-sdk
../repo/repo sync -j4
```

② 然后执行如下命令开始编译：

```
cd /home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x
make clean
make 100ask_imx6ull_pro_ddr512m_systemV_qt5_defconfig
make all -j4
```



编译成功后文件输出路径为 output/images

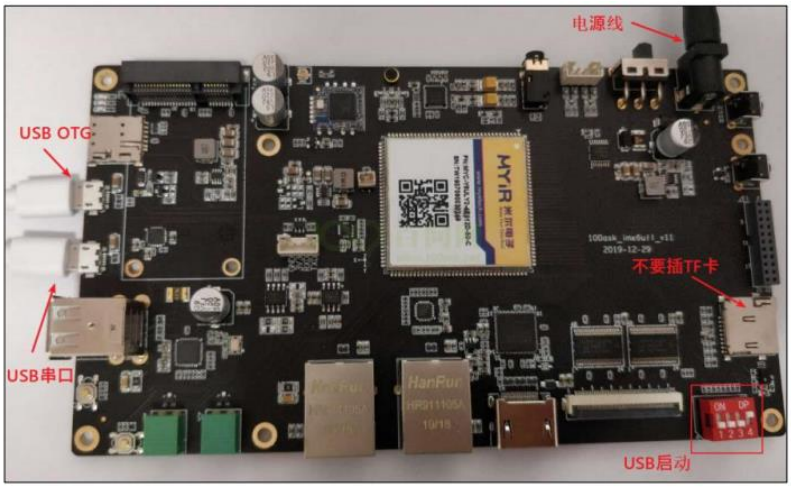
```
buildroot2020.02.x
   ├── output
      ├── images
         ├── 100ask_imx6ull-14x14.dtb <--设备树文件
         ├── rootfs.ext2              <--ext2 格式根文件系统
         ├── rootfs.ext4 -> rootfs.ext2      <--ext2 格式根文件系统
         ├── rootfs.tar
         ├── rootfs.tar.bz2           <--打包并压缩的根文件系统，用于
NFSROOT 启动
         ├── 100ask-imx6ull-pro-512d-systemv-v1.img          <--完整的系统
镜像(可以用来烧写 emmc 和 sd 卡)
         ├── u-boot-dtb.imx           <--u-boot 镜像
         └── zImage                <--内核镜像
```

3.1.3 烧写

把上面编译出来的 100ask-imx6ull-pro512d-systemv-v1.img 重命名为 emmc.img(或 sdcard.img)，把 emmc.img(或 sdcard.img) 放到“100ask_imx6ull 烧写工具”的 files 目录里，就可以使用工具烧写到 EMMC(或 SD/TF 卡)。

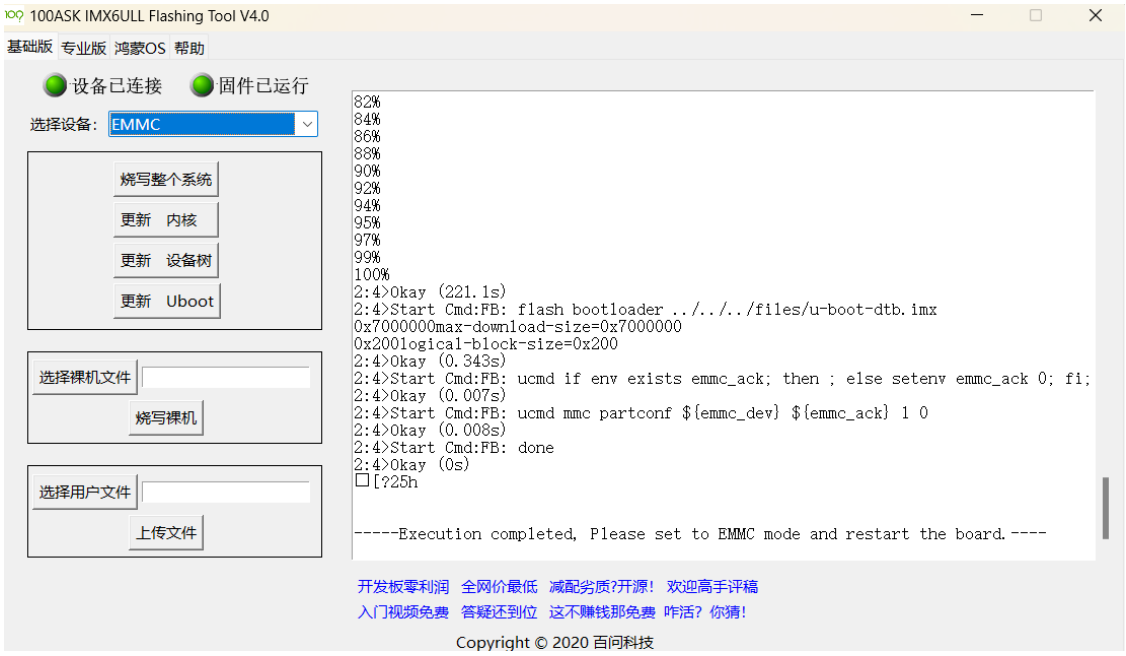
具体参考《嵌入式 Linux 应用开发完全手册 V5.2_IMX6ULL_Pro 开发板》第三篇第 7 章：“烧写整个系统或更新部分系统”

连接 USB OTG 线，设置 USB 启动：



打开烧写工具烧写整个系统：


100ask_imx6ull_flashing_tool
bin
doc
files // files目录下保存要烧写的文件，文件名不能变，要更新请覆盖
100ask_imx6ull-14x14.dtb // 设备树
emmc.img // EMMC映像文件
led.imx // 裸板文件，演示用
sdcard.img // SD/TF卡映像文件
u-boot-dtb.imx // u-boot
zImage // linux内核
firmware
scripts
basic
pro
100ask_imx6ull_flashing_tool.exe // GUI程序，双击运行，使用它来烧写


100ASK IMX6ULL Flashing Tool V4.0
基础版 专业版 鸿蒙OS 帮助
设备已连接
固件已运行
选择设备: EMMC
烧写整个系统
更新 内核
更新 设备树
更新 Uboot
选择裸机文件
烧写裸机
选择用户文件
上传文件
82%
84%
86%
88%
90%
92%
94%
95%
97%
99%
100%
2:4>Okay (221.1s)
2:4>Start Cmd:FB: flash bootloader ../../../files/u-boot-dtb.imx
0x7000000max-download-size=0x7000000
0x200logical-block-size=0x200
2:4>Okay (0.343s)
2:4>Start Cmd:FB: ucmd if env exists emmc_ack; then ; else setenv emmc_ack 0; fi;
2:4>Okay (0.007s)
2:4>Start Cmd:FB: ucmd mmc partconf ${emmc_dev} ${emmc_ack} 1 0
2:4>Okay (0.008s)
2:4>Start Cmd:FB: done
2:4>Okay (0s)
□[?25h
-----Execution completed, Please set to EMMC mode and restart the board.----
开发板零利润 全网价最低 减配劣质?开源! 欢迎高手评稿
入门视频免费 答疑还到位 这不赚钱那免费 咋活? 你猜!
Copyright © 2020 百问科技

3.2 安装 Qtcreator

QtCreator 下载网址：https://download.qt.io/archive/qt/5.12/5.12.9/

这里我们使用的 qtcreator 版本为 5.12.9，无法打开 QT 网站的话，请使用网盘中的文件。

此安装过程类似 Windows 下应用的安装方法，双击安装即可，如下图所示：



在安装过程中，比较特殊的地方如下图所示：



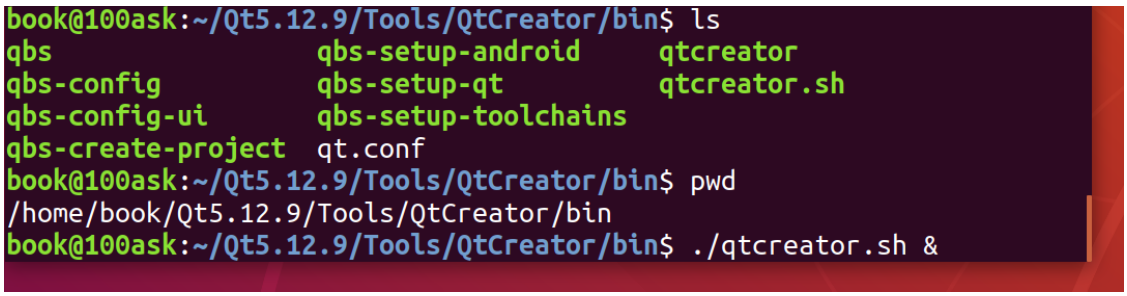
安装成功后打开 QT，可以直接双击图标打开：

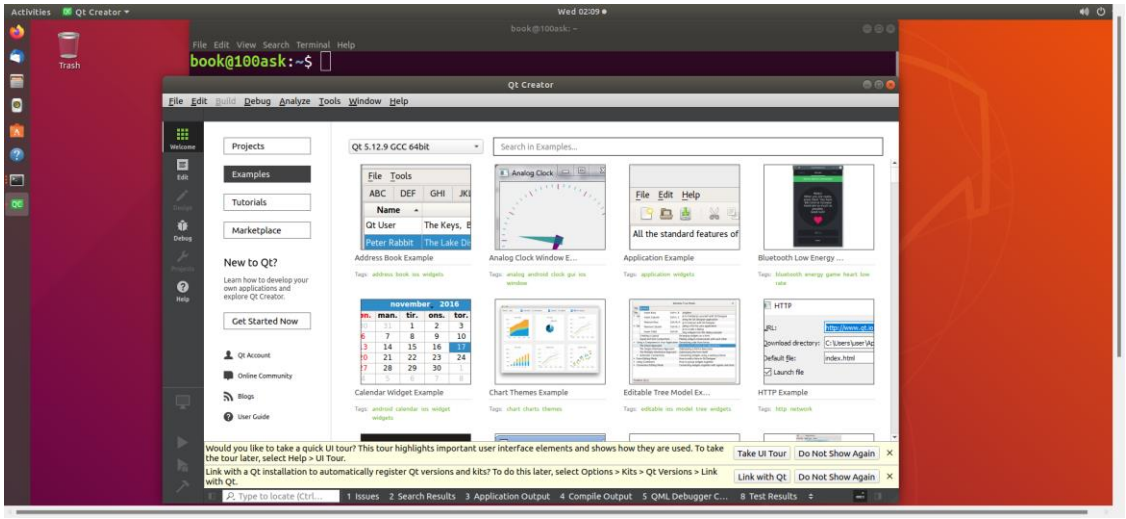


也可以通过命令打开 QT，命令如下：

```
cd   ~/Qt5.12.9/Tools/QtCreator/bin/
./qtcreator.sh &
```

这里的“&”符号代表后台运行， 不占用控制命令终端：





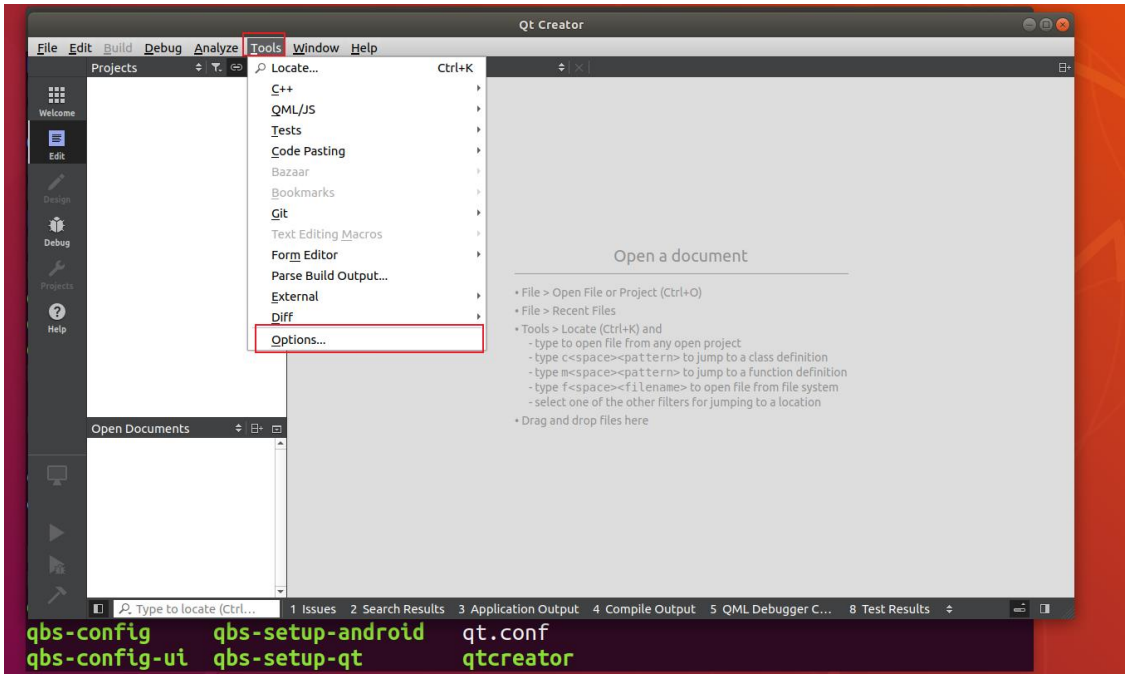
3.3 配置 QtCreator 开发环境

在这里我们配置 QtCreator 开发环境主要是为了方便将测试程序移植到我们的开发板上。安装完成后直接双击打开，或者进入目录，运行如下命令启动 QtCreator，如图：

```
book@100ask:~/Qt5.12.9/Tools/QtCreator/bin$ ls
qbs                  qbs-setup-android     qtcreator
qbs-config           qbs-setup-qt          qtcreator.sh
qbs-config-ui        qbs-setup-toolchains
qbs-create-project   qt.conf
book@100ask:~/Qt5.12.9/Tools/QtCreator/bin$ pwd
/home/book/Qt5.12.9/Tools/QtCreator/bin
book@100ask:~/Qt5.12.9/Tools/QtCreator/bin$ ./qtcreator.sh &
```

3.3.1 打开选项界面

在 QtCreator 界面中，依次点击 tools -> options：



3.3.2 设置编译器

我们需要设置 c++编译器、c 编译器：

① 对于 100ASK_IMX6ULL_Pro，编译器目录如下：

```
# C编译器
/home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-gcc

# C++编译器
/home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-g++
```

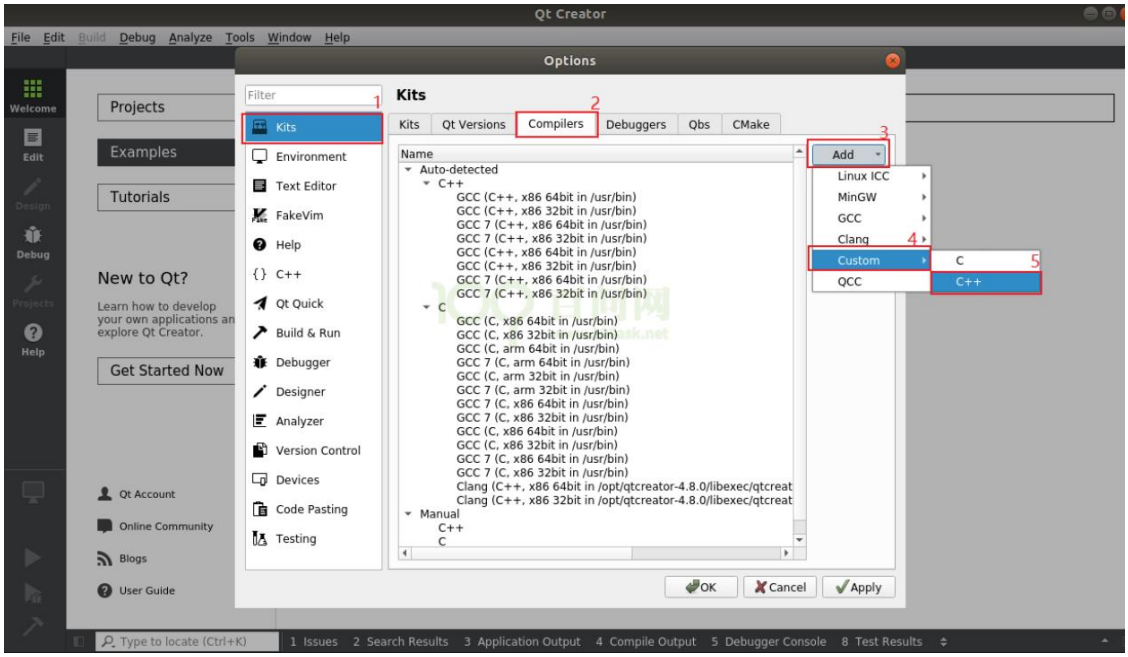
② 对于 100ASK_IMX6ULL_Mini，编译器目录如下：

```
# C编译器
/home/book/100ask_imx6ull_mini-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-gcc

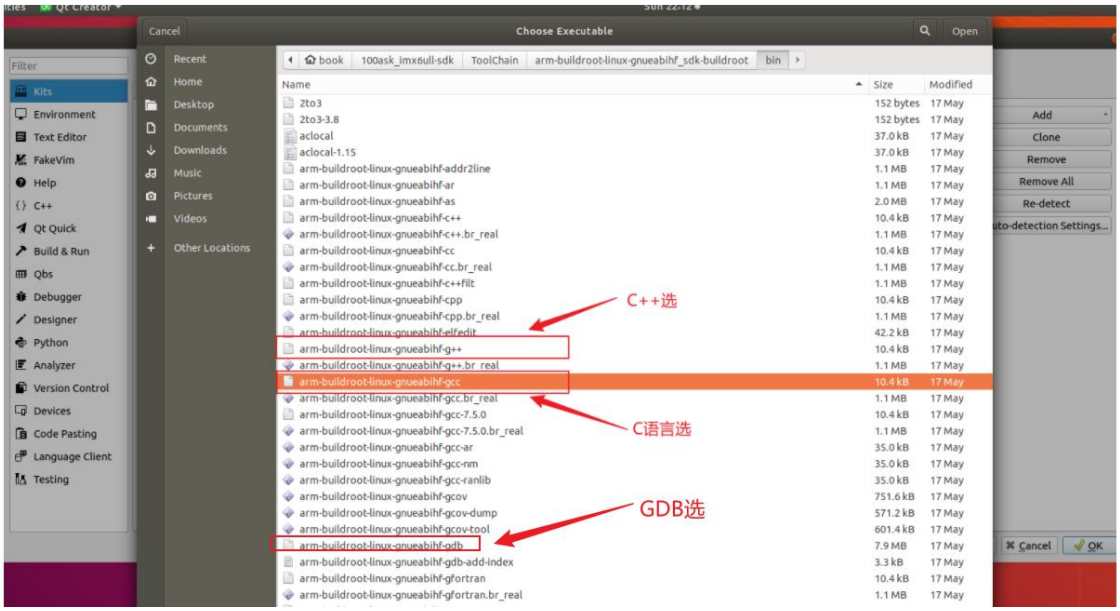
# C++编译器
/home/book/100ask_imx6ull_mini-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-g++
```

以设置 100ASK_IMX6ULL_Pro 的 c++编译器为例。

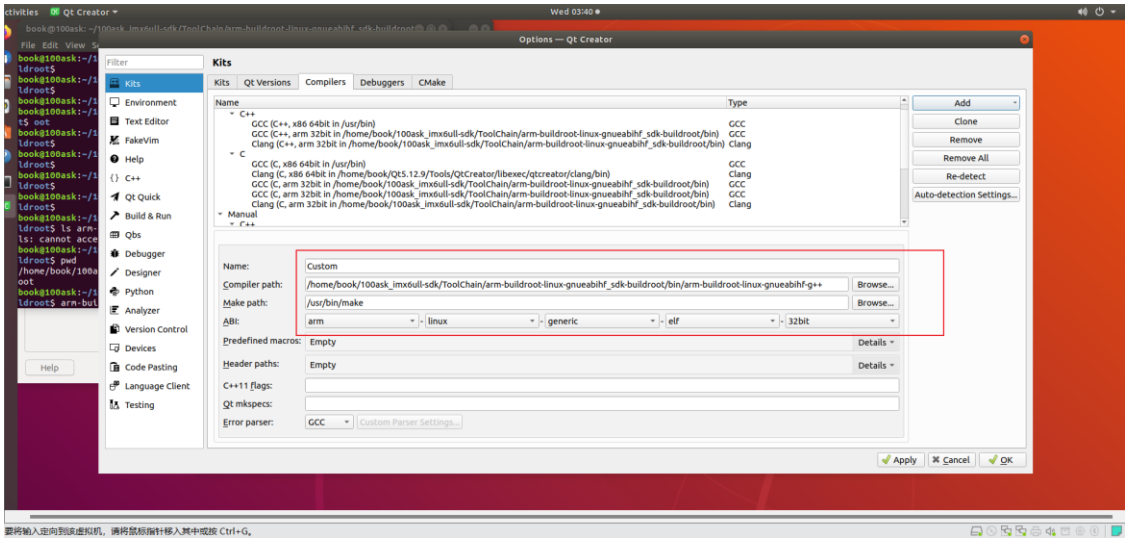
在出现的选项对话框中，在左边点击 Kits，右边选择 Compilers 标签，并点击 Add 选择 Custom->C++：



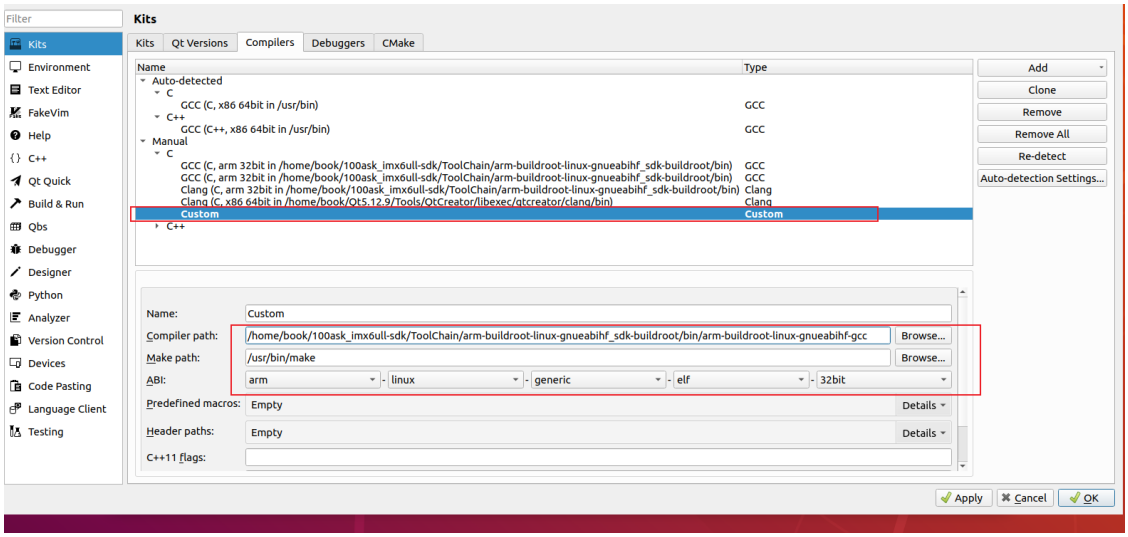
在弹出的对话框中选择编译器，如下图所示：


Cancel
Choose Executable
Open
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Other Locations
book
100ask_imx6ull-sdk
ToolChain
arm-buildroot-linux-gnueabihf_sdk-buildroot
bin
Name
Size
Modified
C++选
C语言选
GDB选
Filter
Kits
Environment
Text Editor
FakeVim
Help
C++
Qt Quick
Build & Run
Qbs
Debugger
Designer
Python
Analyzer
Version Control
Devices
Code Pasting
Language Client
Testing
Add
Clone
Remove
Remove All
Re-detect
Cancel
OK

然后如下图设置“Make Path”为“/usr/bin/make”，设置 ABI 为“arm、linux、generic、elf、32bit”：



使用同样方法设置C编译器，结果如下图所示：



3.3.3 设置调试器

还需要设置使用哪个调试器：

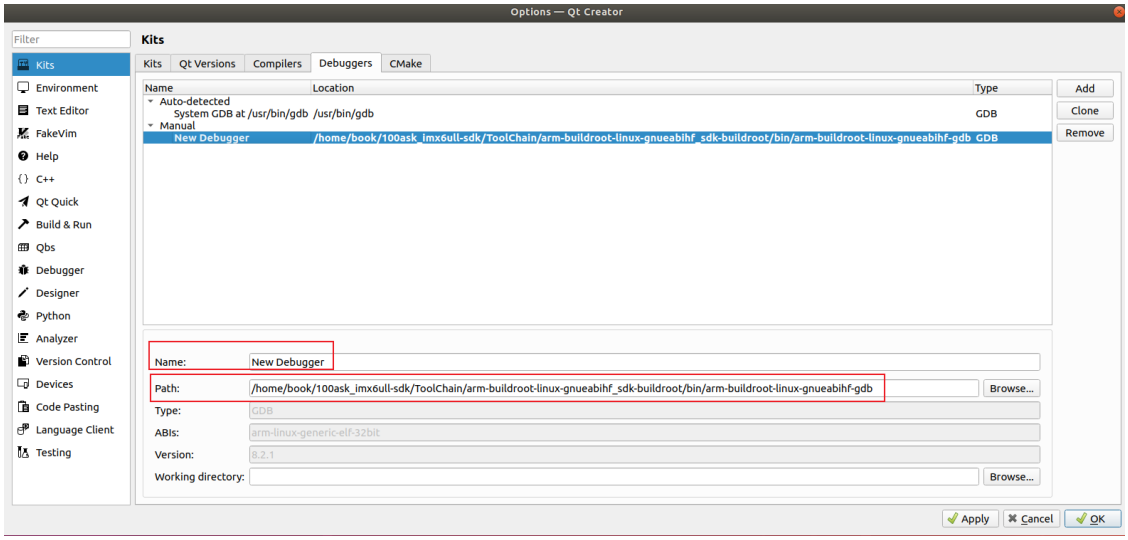
① 对于 100ASK_IMX6ULL_Pro，调试器目录如下：

```
# 调试器
/home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-gdb
```

② 对于 100ASK_IMX6ULL_Mini，调试器目录如下：

```
# 调试器
/home/book/100ask_imx6ull_mini-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-gdb
```

在 Kits->Debuggers 里设置一下 GDB，以 100ASK_IMX6ULL_Pro 为例，如下图：

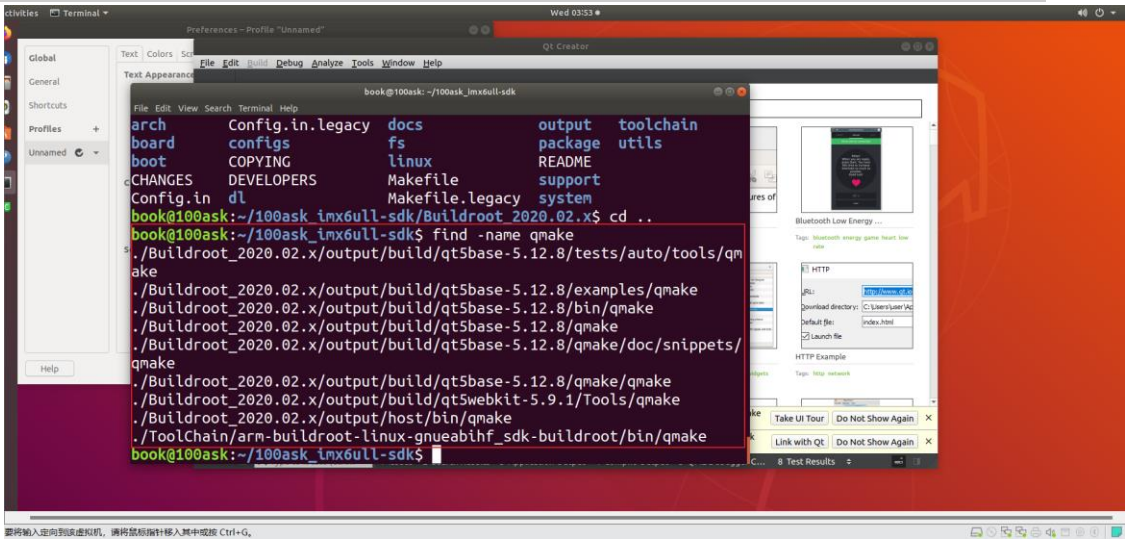


最后记得点 Apply 保存。

3.3.4 添加 QT 版本

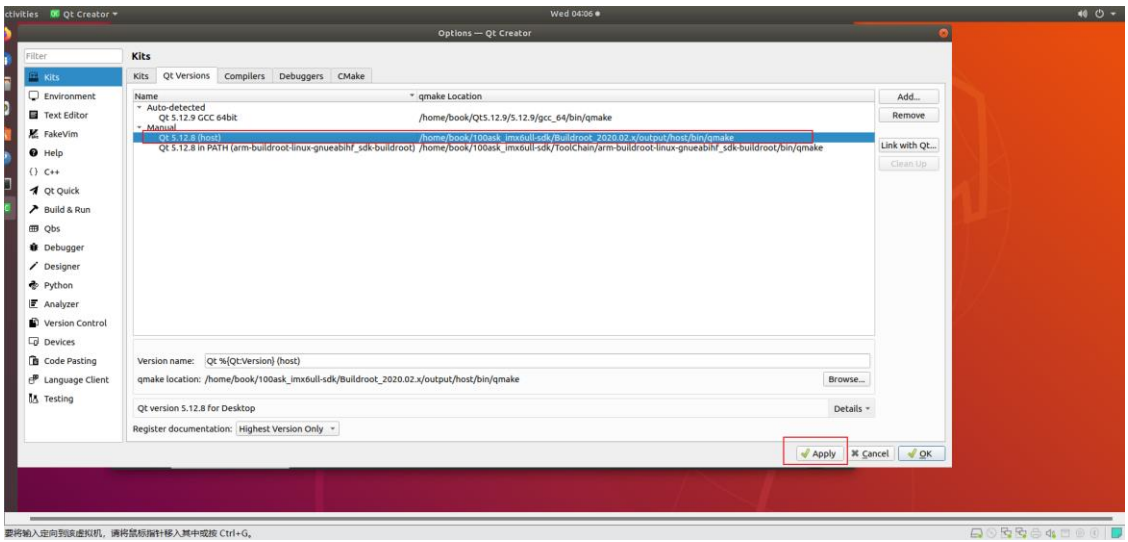
假设你已经按照上面编译出了 QT 的 SDK。 以 IMX6ULL_Pro 为例，你已经做了这些事情： 在执行“make 100askimx6ull-qtdefconfig”配置后，再执行“make all”可以编译出 QT 的 SDK 包。可以在/home/book/100ask_imx6ull-sdk 下执行这个命令找到 qmake，记住它的目录：

```
find -name qmake
```



在 QT 设置界面中，选择 Qt Version 标签，在右侧点击“Add...”，会弹出对话框,切换目录到选择 qmake 文件后,点击 open 按钮,设置完成之后，点击 Apply 按钮保存。

注意：qmake 文件是 buildroot 编译根文件系统后生成的，文件在 buildroot 目录下。



3.3.5 配置 kit 标签

继续选择上边的 Kits 标签，点击右侧 Add，填写相应内容：

① Name：输入 100ask_imx6ull

② Sysroot：输入交叉编译工具链的 sysroot 目录

对于 100ASK_IMX6ULL_Pro，sysroot 目录如下：

```
/home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/arm-buildroot-linux-gnueabihf/sysroot/
```

对于 100ASK_IMX6ULL_Mini，sysroot 目录如下：

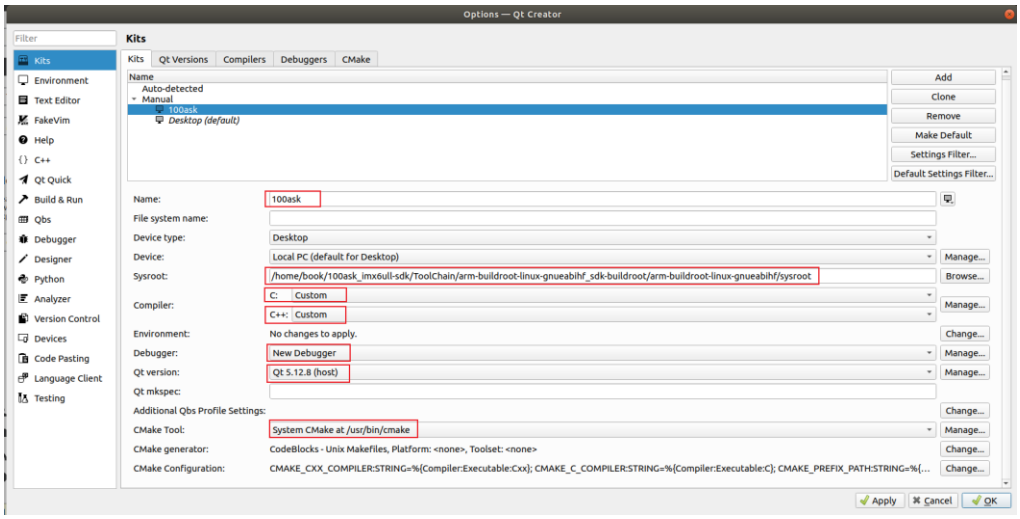
```
/home/book/100ask_imx6ull_mini-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/arm-buildroot-linux-gnueabihf/sysroot/
```

③ compiler：c 和 c++这两个选择框里，都选择 Custom

④ Debugger：选择刚刚配置的 Debuggers，名称为 New Debugger

⑤ Qtversio：选择上图中配置的“Qt5.12.8 (host)”

配置完成后点击 apply,点击 OK。



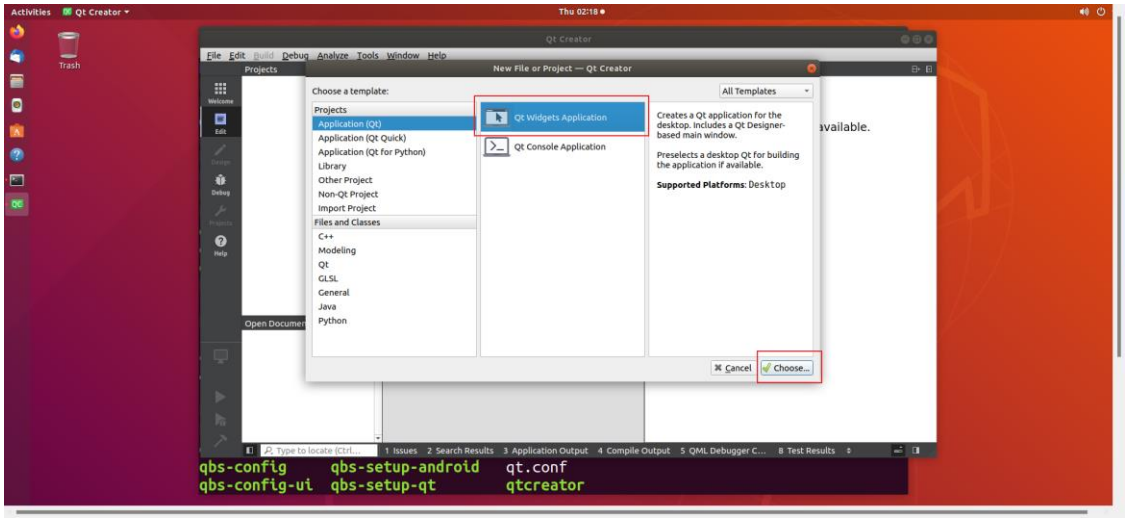
配置完成后点击 apply，点击 OK。

3.4 编写 QT 程序

第 1 步创建新项目。

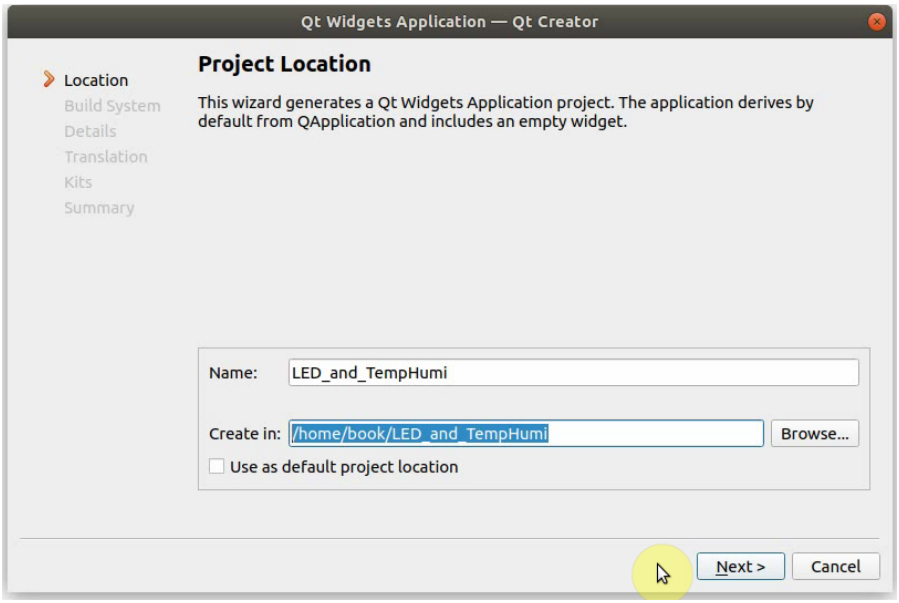
运行 QtCreator 后，在菜单栏选择 File -> New File or Project，选择项目类型。

在打开的对话框中,依次选择 Application -> Qt Widgets Application ,点击 Choose... ， 如下图所示:

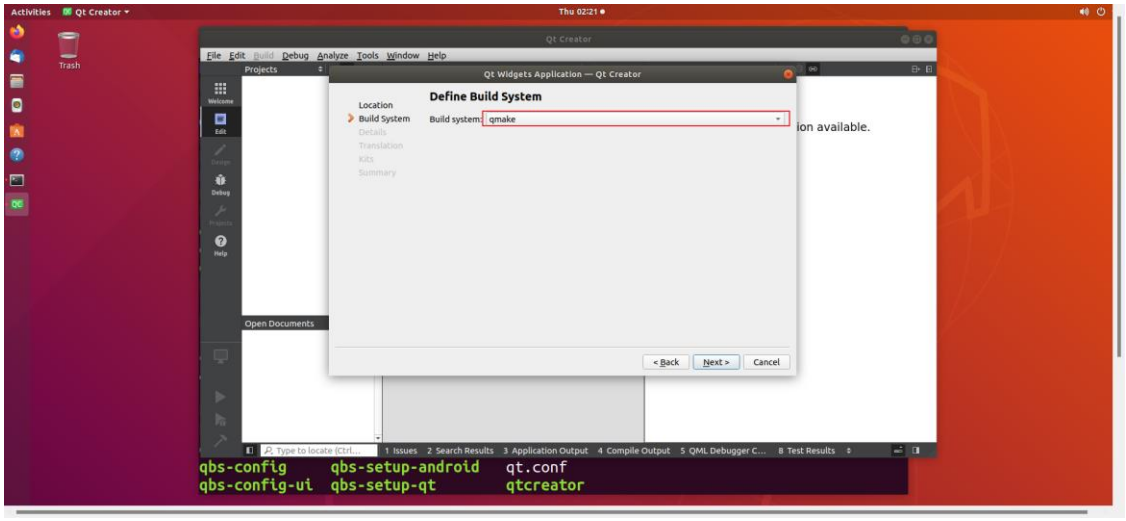


第 2 步：输入项目名字、设置保存位置。

在弹出的 Qt Widgets Application 对话框中，设置项目名称为“LED_and_TempHumi”,Create in 一栏填写项目的存储路径（点击右边的 Brouse 按钮创建这个目录），点击 Next。如下：

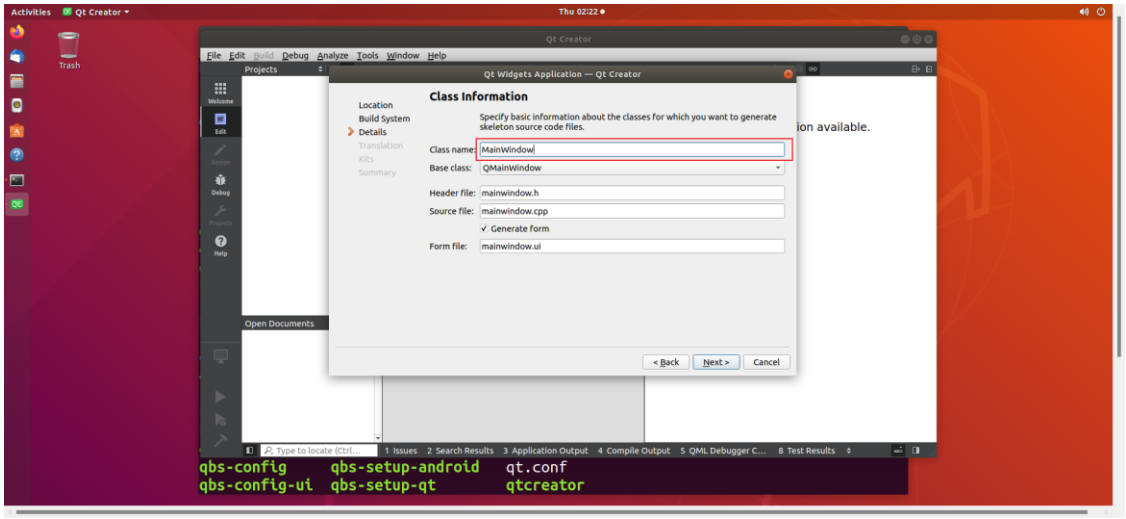


第 3 步 设置构建系统，然后点击 Next 进入下一步，如下:

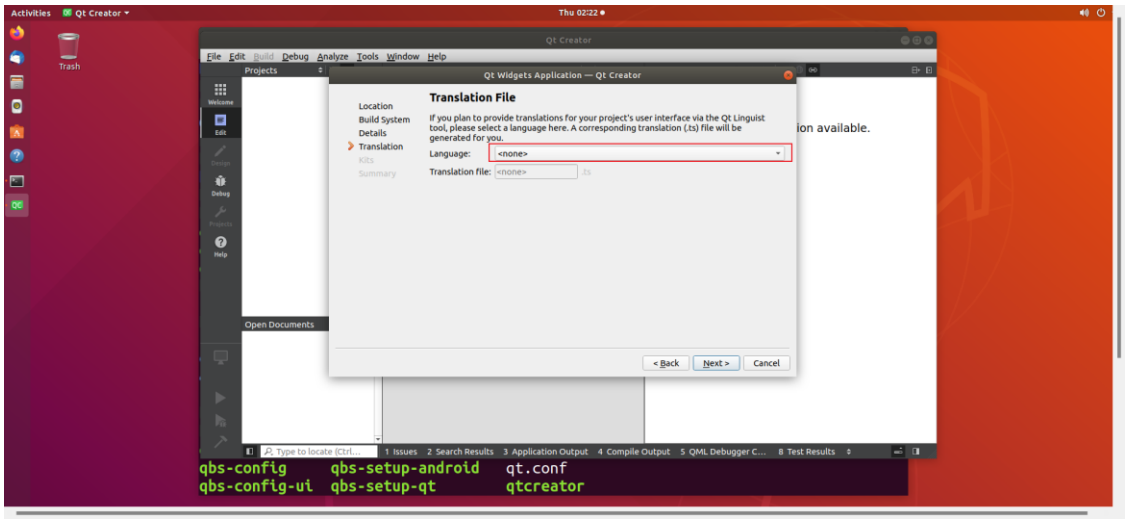


第 4 步 选择基类：

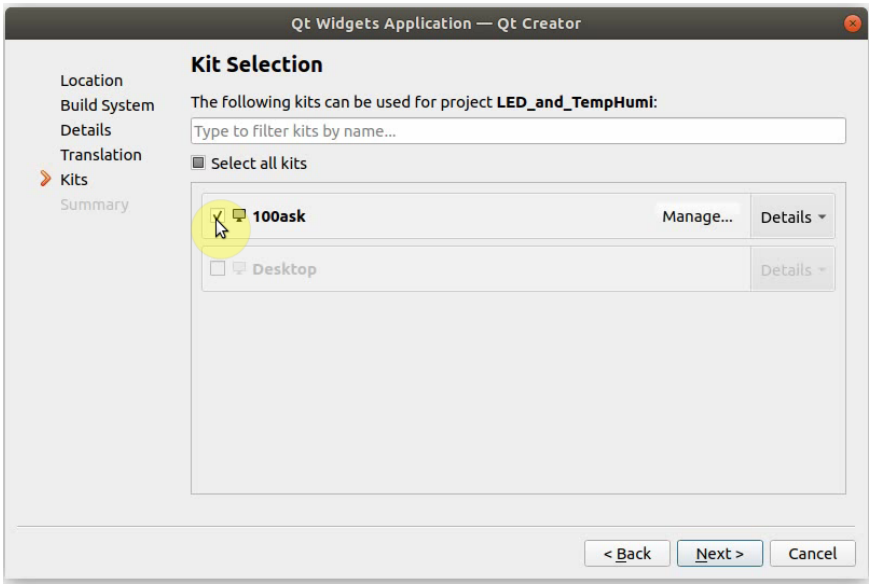
当前的的应用继承自哪种 Widget，默认选择 QMainWindow，然后点击 Next 进入下一步。



第 5 步 默认 none，继续点击 next：

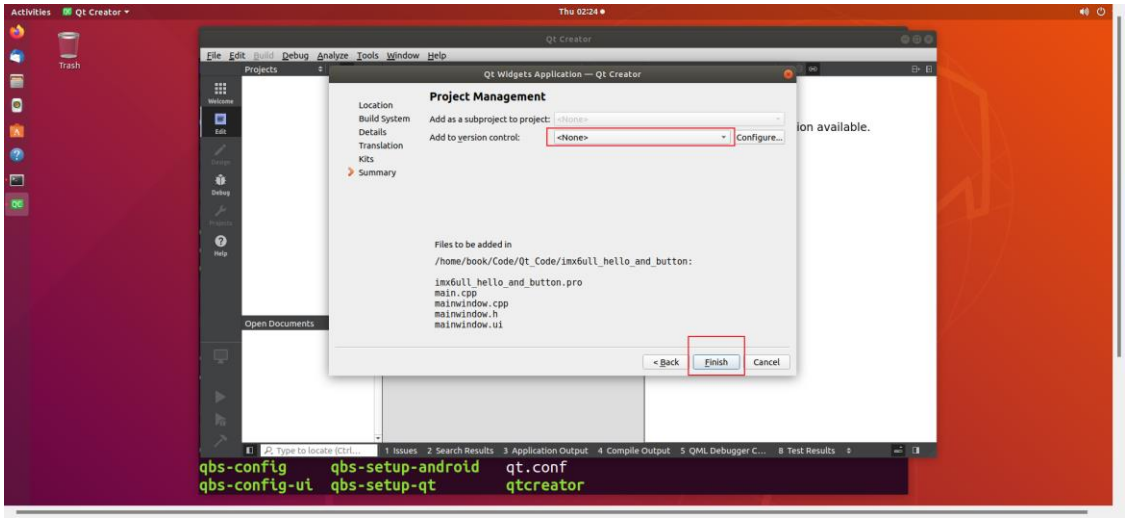


第 6 步 选择之前添加好的 Kits(100ask)，继续点击 next：



第 7 步 完成项目创建：

以上信息填写完后，点击 Finish，完成项目的创建。



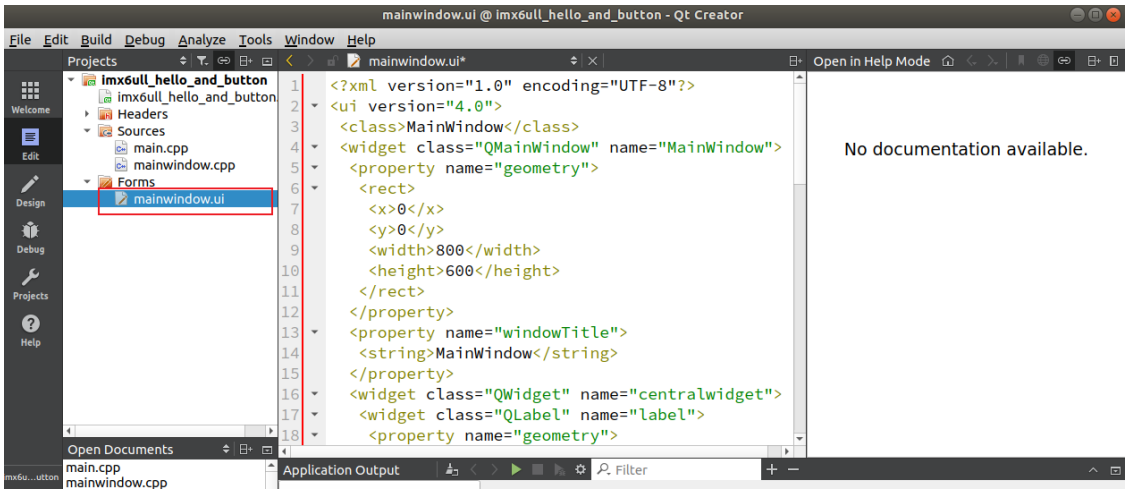
3.5 编译 Qt 程序

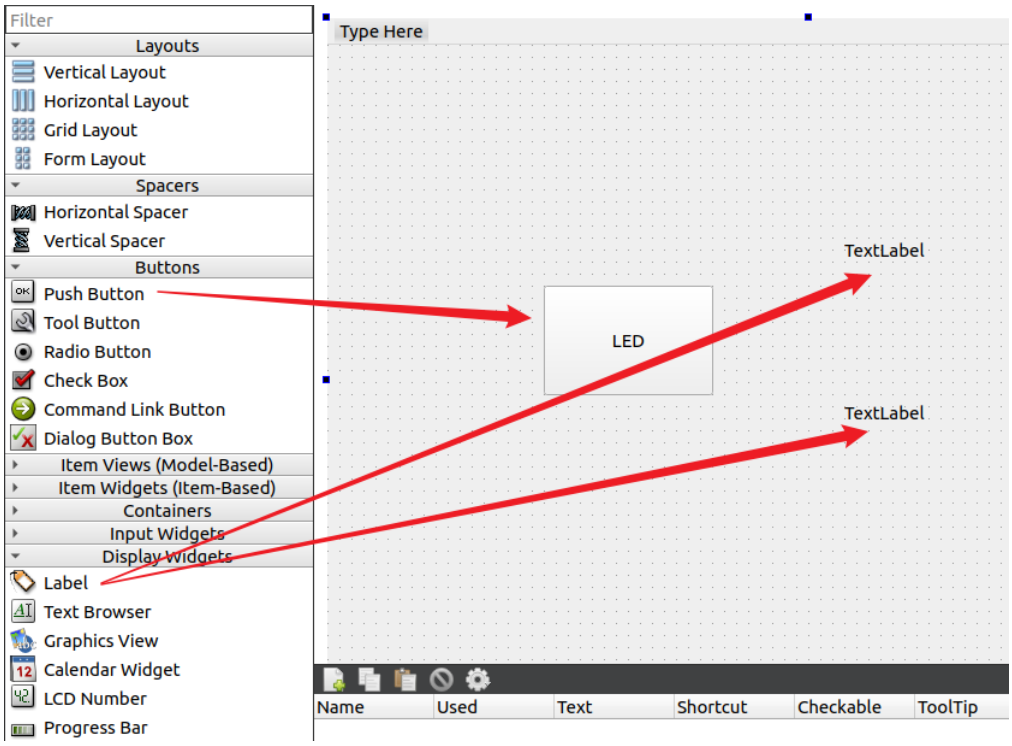
第 1 步 修改界面：

双击左侧的 Forms 里的 mainwindow.ui 文件，打开 Design 视图。

然后如下图所示，从左侧 Display Widgets 栏目下，拖动 Label 和 PushPutton 到中间的区域。

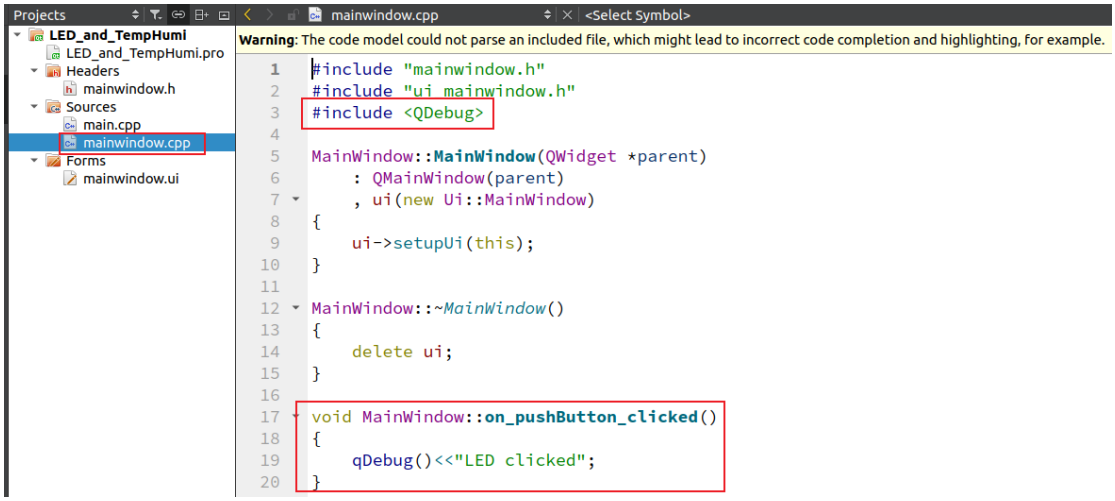
最后双击中间区域的 Label 还有 Push Button 修改内容如下图：





第 2 步 添加测试代码：

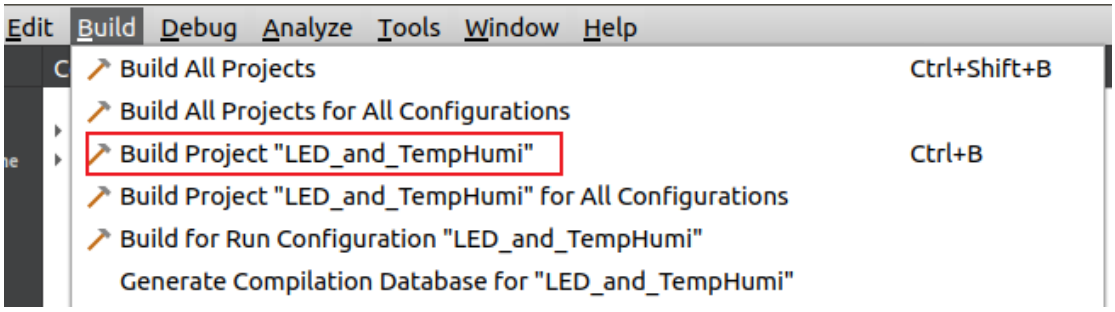




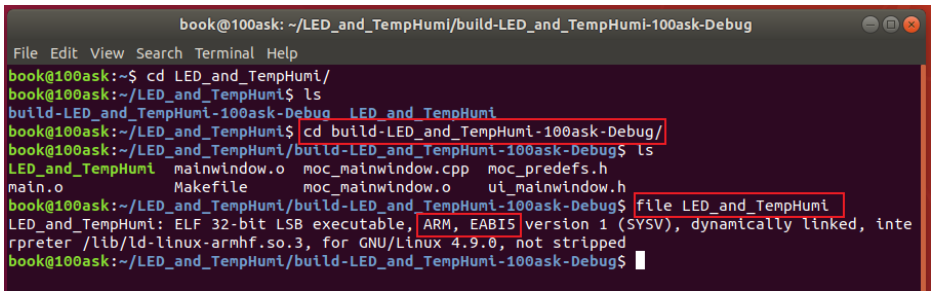
第 3 步 构建：

点击菜单栏 Build -> Build Project LED_and_TempHumi，开始编译、构建项目。

在构建过程中，会在左下侧是“Compile Output”窗口打印构建信息。如果有错误，请根据提示出错信息修改，然后重新构建。



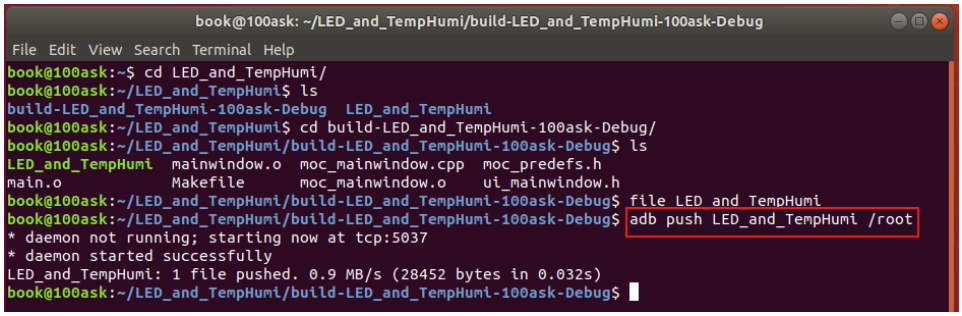
第 4 步 查看构建结果：项目构建成功后，编译好的二进制文件存放在刚刚设置的目录中，可以使用 file 命令查看该 APP 是否被编译为 ARM 架构：


book@100ask: ~/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug
File Edit View Search Terminal Help
book@100ask:~$ cd LED_and_TempHumi/
book@100ask:~/LED_and_TempHumi$ ls
build-LED_and_TempHumi-100ask-Debug LED_and_TempHumi
book@100ask:~/LED_and_TempHumi$ cd build-LED_and_TempHumi-100ask-Debug/
book@100ask:~/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug$ ls
LED_and_TempHumi mainwindow.o moc_mainwindow.cpp moc_predefs.h
main.o   Makefile   moc_mainwindow.o   ui_mainwindow.h
book@100ask:~/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug$ file LED_and_TempHumi
LED_and_TempHumi: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, inte
rpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 4.9.0, not stripped
book@100ask:~/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug$

3.6 在开发板上运行 Qt 程序

3.6.1 手工运行

使用 ADB 把可执行程序 LED_and_TempHumi 放到到开发板上：



在开发板上执行如下命令：

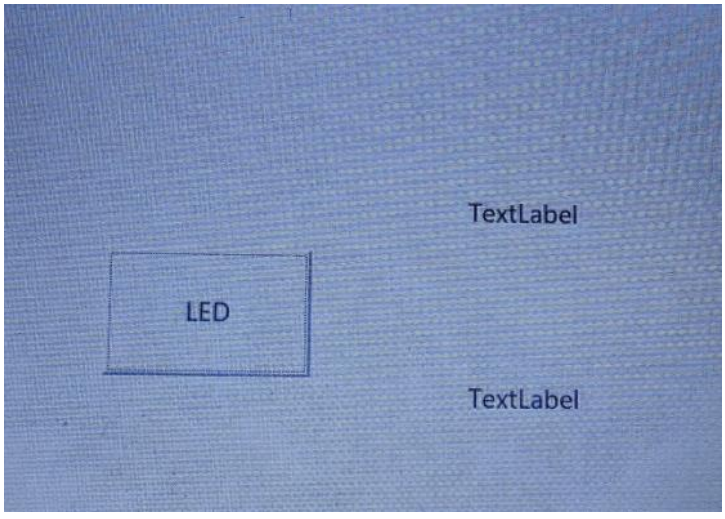
```
// 1. 关闭原来的GUI
mv /etc/init.d/*lvgl* /root
mv /etc/init.d/*hmi* /root
reboot

// 2. 禁止LCD黑屏
echo -e "\033[9;0]" > /dev/tty0

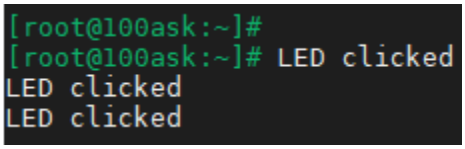
// 3. 设置QT运行环境
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/

// 4. 运行QT程序
/root/LED_and_TempHumi
```

在屏幕上可以看到如下界面：



点击LED按钮，可以在控制台看到如下输出信息：



3.6.2 自动运行

创建一个文件：/etc/init.d/S99myqt，内容如下：

```
#!/bin/sh

start() {
echo -e "\033[9;0]" > /dev/tty0
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/

psplash-write "PROGRESS 95"
psplash-write "QUIT"
/root/LED_and_TempHumi  &
}

stop() {
    killall LED_and_TempHumi
}

case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    *)
        echo "Usage: $0 {start| stop|restart}"
        exit 1
esac

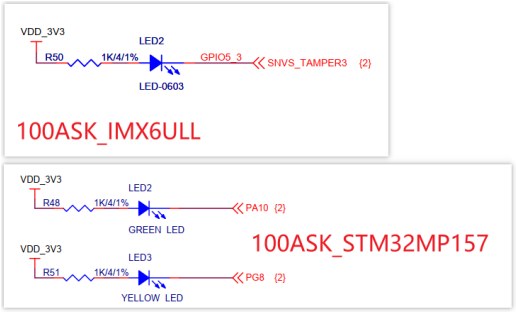
exit $?
```

以后开发板启动后即可自动运行QT程序。

第4章 使用文件 IO 操作硬件

4.1 通过 GPIO SYSFS 操作 LED

在 QT 界面，我们要实现点击“LED”按钮就可以控制板子上的 LED。LED 接线图如下：



在 Linux 系统里，我们可以使用 2 种方法去操作上面的 LED：

① 使用 GPIO SYSFS 系统：这需要一定的硬件知识，需要设置引脚的方向、数值

② 使用驱动程序：无需硬件知识，使用 open/read/write 接口调用驱动即可。

4.1.1 GPIO SYSFS 系统体验

一个芯片内部，有多个 GPIO 控制器，每个 GPIO 控制器有多个引脚。在 /sys/bus/gpio/devices 目录下，列出了所有的 GPIO 控制器，如下表示有 6 个 GPIO 控制器：

```
# ls /sys/bus/gpio/devices
gpiochip0  gpiochip1  gpiochip2  gpiochip3  gpiochip4  gpiochip5
```

怎么查看每个 GPIO 控制器的详细信息？/sys/class/gpio/gpiochipXXX 下，有这些信息：

```
[root@100ask:~]# ls /sys/class/gpio/gpiochip* -dl
/sys/class/gpio/gpiochip0
/sys/class/gpio/gpiochip128
/sys/class/gpio/gpiochip32
/sys/class/gpio/gpiochip504
/sys/class/gpio/gpiochip64
/sys/class/gpio/gpiochip96
[root@100ask:~]#
[root@100ask:~]# ls /sys/class/gpio/gpiochip0 -l
base      // 这个GPIO控制器的GPIO编号基值
device
label     // 名字
ngpio     // 引脚个数
power
subsystem
uevent

[root@100ask:~]# cat /sys/class/gpio/gpiochip0/label
209c000.gpio
```

怎么查看 GPIO 使用情况？执行如下命令即可：

```
# cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 0-31, parent: platform/209c000.gpio, 209c000.gpio:
 gpio-5   (                    |goodix_ts_int       ) in  hi IRQ
 gpio-19  (                    |cd                  ) in  hi IRQ
 gpio-20  (                    |spi_imx             ) out hi

gpiochip1: GPIOs 32-63, parent: platform/20a0000.gpio, 20a0000.gpio:

gpiochip2: GPIOs 64-95, parent: platform/20a4000.gpio, 20a4000.gpio:
 gpio-68  (                    |lcdif_rst           ) out hi

gpiochip3: GPIOs 96-127, parent: platform/20a8000.gpio, 20a8000.gpio:
 gpio-110 (                    |User2 Button        ) in  hi IRQ
 gpio-120 (                    |spi_imx             ) in  lo
 gpio-122 (                    |spi_imx             ) in  lo

gpiochip4: GPIOs 128-159, parent: platform/20ac000.gpio, 20ac000.gpio:
 gpio-129 (                    |User1 Button        ) in  hi IRQ
 gpio-130 (                    |goodix_ts_rst       ) out hi
 gpio-133 (                    |sysfs               ) out hi
 gpio-134 (                    |phy-reset           ) out hi
 gpio-135 (                    |spi32766.0          ) out hi
 gpio-136 (                    |?                   ) out lo
 gpio-137 (                    |phy-reset           ) out hi
 gpio-138 (                    |spi4                ) out hi
 gpio-139 (                    |spi4                ) out lo
```

4.1.2 通过 SYSFS 使用 GPIO

如果只是简单的引脚控制(比如输出、查询输入值)，可以不编写驱动程序。
但是涉及中断的话，就需要编写驱动程序了。

1．确定 GPIO 编号

查看每个/sys/class/gpio/gpiochipXXX 目录下的 label，确定是你要用的 GPIO 控制器，也称为 GPIO Bank。
根据它名字 gpiochipXXX，就可以知道基值是 XXX。
基值加上引脚 offset，就是这个引脚的编号。

对于 GPIOx_y，一般有一个公式：编号=(x-1)*32+y。x 从 1 开始。
在 IMX6ULL 的原理图里，LED 使用的引脚是 GPIO5_3，它的编号为：(5-1)*32+3=131。

2．导出/设置方向/读写值

举例：IMX6ULL 使用的 GPIO5_3 引脚编号是 131，可以如下操作：

```
echo 131 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio131/direction
echo 1 > /sys/class/gpio/gpio131/value
```

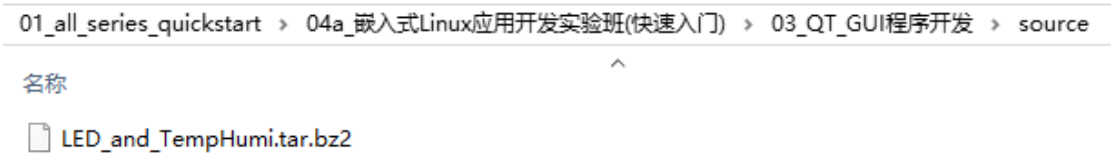
```
echo 131 > /sys/class/gpio/unexport
```

如果 GPIO5_3 是按键，可以如下读取它的值：

```
echo 131 > /sys/class/gpio/export
echo in > /sys/class/gpio/gpio131/direction
cat /sys/class/gpio/gpio131/value
echo 131 > /sys/class/gpio/unexport
```

4.1.3 编写应用程序

在这个程序的基础上修改代码：

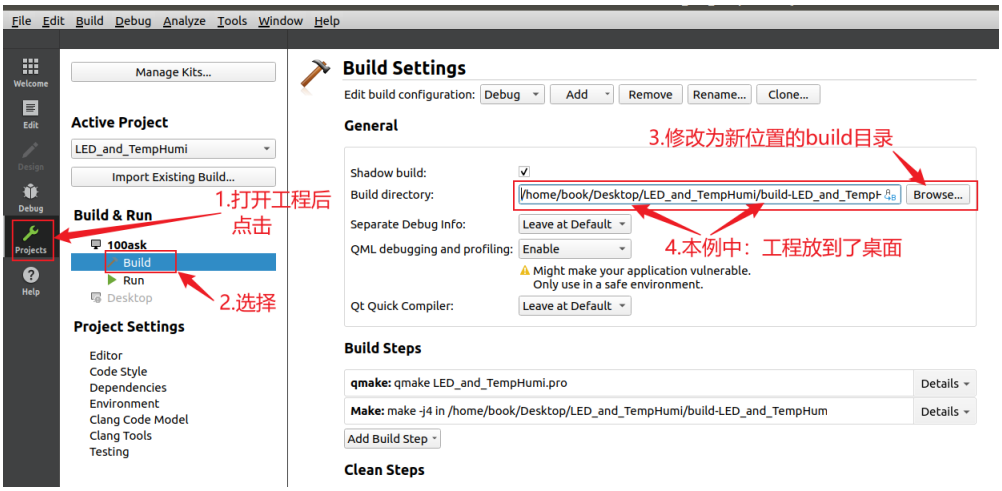


改好的程序放在这个目录下：



1．复制工程到其他路径

复制工程到其他目录的话，一定要修改工程的 Build 目录，如下修改：



2．编写代码

增加了 led.h、led.cpp，led.cpp 源码如下：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#include <string.h>
```

```
#include <unistd.h>
#include <QDebug>

void led_init(void)
{
    /*
     * echo 131 > /sys/class/gpio/export
     * echo out > /sys/class/gpio/gpio131/direction
     */

    int fd;

    fd = open("/sys/class/gpio/export", O_WRONLY);
    if (fd < 0)
    {
        qDebug()<<"open /sys/class/gpio/export err";
        return ;
    }

    write(fd, "131\n", 4);

    close(fd);

    fd = open("/sys/class/gpio/gpio131/direction", O_WRONLY);
    if (fd < 0)
    {
        qDebug()<<"open /sys/class/gpio/gpio131/direction err";
        return ;
    }

    write(fd, "out\n", 4);

    close(fd);
}

void led_control(int on)
{
    /*
     * echo 1 > /sys/class/gpio/gpio131/value
     * echo 0 > /sys/class/gpio/gpio131/value
     */
    static int fd = -1;

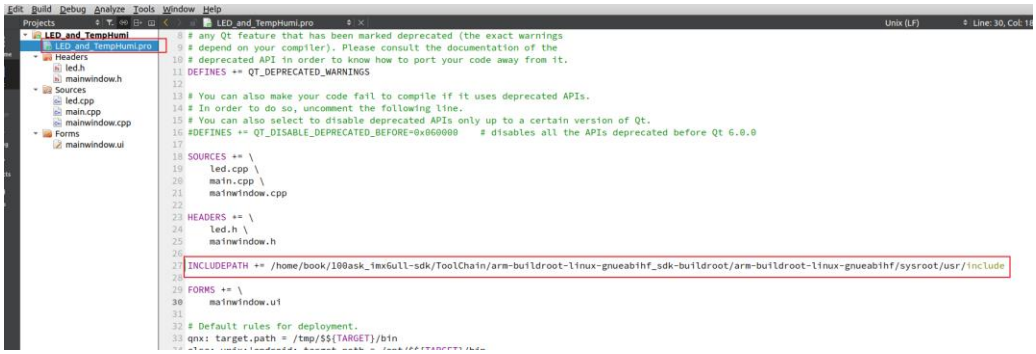
    if (fd == -1)
```

```
    {
        fd = open("/sys/class/gpio/gpio131/value", O_RDWR);
    }
    if (fd < 0)
    {
        qDebug()<<"open /sys/class/gpio/gpio131/value err";
        return ;
    }

    if (on)
    {
        write(fd, "0\n", 2);
    }
    else
    {
        write(fd, "1\n", 2);
    }
}
```

3．解决头文件找不到的问题

编译时会提示找不到文件，如下设置：



上述文字为（注意从pdf里复制时容易丢失字符，要人眼检查）：

```
INCLUDEPATH += /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/arm-buildroot-linux-gnueabihf/sysroot/usr/include
```

4.1.4 上机实验

先使用ps命令查看原来的“LED_and_TempHumi”，使用它的PID杀掉它，比如：

```
[root@100ask:~]# ps
PID   USER     COMMAND
    1 root     init
    2 root     [kthreadd]
    3 root     [ksoftirqd/0]
  316 root     /usr/bin/swupdate -f /etc/
  330 root     /usr/bin/adbd
  341 root     /root/LED and TempHumi
  343 root     -bash
  349 root     [kworker/u2:3]
  352 root     ps
[root@100ask:~]# kill -9 341
```

然后在Ubuntu使用ADB上传编译好的“LED_and_TempHumi”：

```
book@100ask:~$ cd Desktop/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug/
book@100ask:~/Desktop/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug$ ls
LED_and_TempHumi  mainwindow.o        moc_mainwindow.o
led.o             Makefile            moc_predefs.h
main.o            moc_mainwindow.cpp  ui_mainwindow.h
book@100ask:~/Desktop/LED_and_TempHumi/build-LED_and_TempHumi-100ask-Debug$ adb push LED_and_TempHumi /root
LED_and_TempHumi: 1 file pushed. 1.0 MB/s (29112 bytes in 0.028s)
```

最后，在开发板设置环境变量、运行程序：

```
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/
/root/LED_and_TempHumi
```

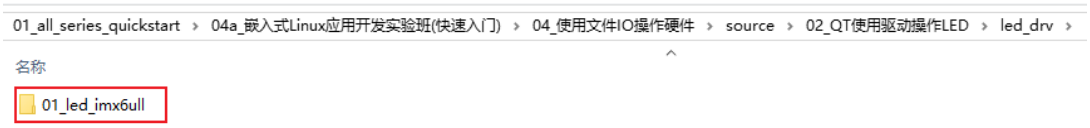
就可以在开发板的QT界面点击按钮控制LED了。

4.2 通过驱动程序操作 LED

4.2.1 编译/安装驱动程序

在驱动程序实验班里讲解驱动，本教程是“应用开发实验班（快速入门）”，所以这节课程只是使用驱动，不讲解驱动。

驱动程序：



在 Ubuntu 上，进入源码目录，执行 make 命令即可编译：

```
book@100ask:~/Desktop/01_led_imx6ull$ make
make -C /home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x/output/build/linux-origin_master M=`pwd` modules
make[1]: Entering directory '/home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x/output/build/linux-origin_master'
  CC [M]  /home/book/Desktop/01_led_imx6ull/led_drv.o
/home/book/Desktop/01_led_imx6ull/led_drv.c: In function ‘gpio_drv_read’:
/home/book/Desktop/01_led_imx6ull/led_drv.c:64:35: warning: array subscript has type ‘char’ [-Wchar-subscripts]
  tmp_buf[1] = gpio_get_value(gpios[tmp_buf[0]].gpio);
                                   ^
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/book/Desktop/01_led_imx6ull/led_drv.mod.o
  LD [M]  /home/book/Desktop/01_led_imx6ull/led_drv.ko
make[1]: Leaving directory '/home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x/output/build/linux-origin_master'
arm-buildroot-linux-gnueabihf-gcc -o led_test led_test.c
book@100ask:~/Desktop/01_led_imx6ull$
```

注意，如果使用 100ASK_IMX6ULL_Mini 开发板，需要修改 Makefile，指定内核目录，然后再执行 make 命令：

```
#KERN_DIR =  /home/book/100ask_imx6ull-sdk/Linux-4.9.88 # 板子所用内核源码的目录
#KERN_DIR =  /home/book/100ask_imx6ull_mini-sdk/Buildroot_2020.02.x/output/build/linux-origin_master
KERN_DIR =  /home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x/output/build/linux-origin_master

all:
        make -C $(KERN_DIR) M=`pwd` modules
        $(CROSS_COMPILE)gcc -o led_test led_test.c
clean:
        make -C $(KERN_DIR) M=`pwd` modules clean
        rm -rf modules.order  led_test

# 参考内核源码drivers/char/ipmi/Makefile
# 要想把a.c, b.c编译成ab.ko, 可以这样指定:
# ab-y := a.o b.o
# obj-m += ab.o


obj-m += led_drv.o
```

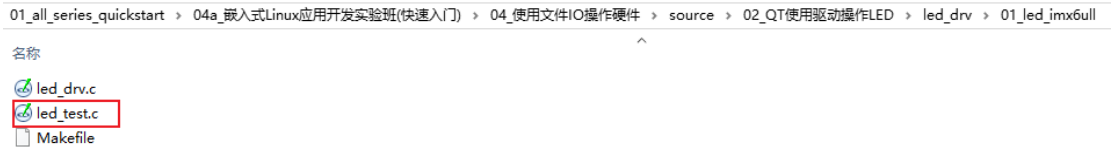
如果使用100ASK_IMX6ULL_Mini
红框里第1行的"#"去掉，
红框里第2行前面加上"#"

把驱动程序 led_drv.ko、led_test 通过 ADB 上传到开发板，然后执行如下命令可以看到灯亮、灭：

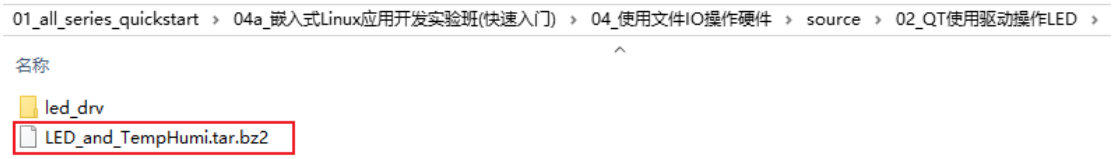
```
# 移除QT程序，否则它占用LED引脚导致无法安装驱动
mv /etc/init.d/S99myqt /root
reboot
# 重启后执行如下命令
insmod  /root/led_drv.ko
ls /dev/100ask_led
/root/led_test 0 on
/root/led_test 0 off
```

4.2.2 编写应用程序

仿照如下文件修改 QT 代码：



改好的应用程序为：



修改的代码为“led.cpp”，代码如下：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#include <string.h>
#include <unistd.h>
#include <QDebug>

static int fd;

void led_init(void)
{
    fd = open("/dev/100ask_led", O_RDWR);
    if (fd < 0)
    {
        qDebug()<<"open /dev/100ask_led failed";
    }
}

void led_control(int on)
{
    char buf[2];
    buf[0] = 0;

    if (on)
    {
        buf[1] = 0;
    }
    else
    {
```

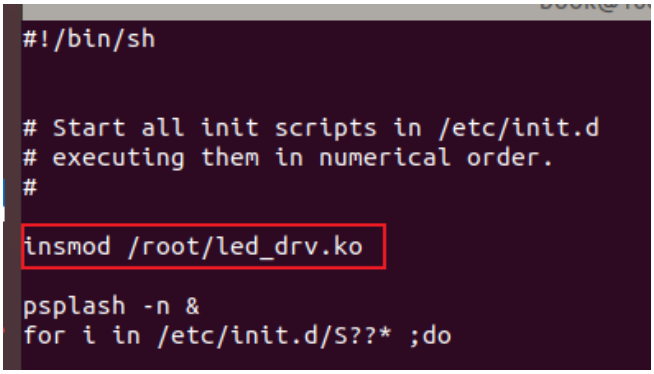
```
        buf[1] = 1;
    }
    write(fd, buf, 2);
}
```

4.2.3 上机实验

修改开发板文件“/etc/init.d/rcS”，在文件开头加入下列代码加载驱动：

```
insmod  /root/led_drv.ko
```

效果如下：



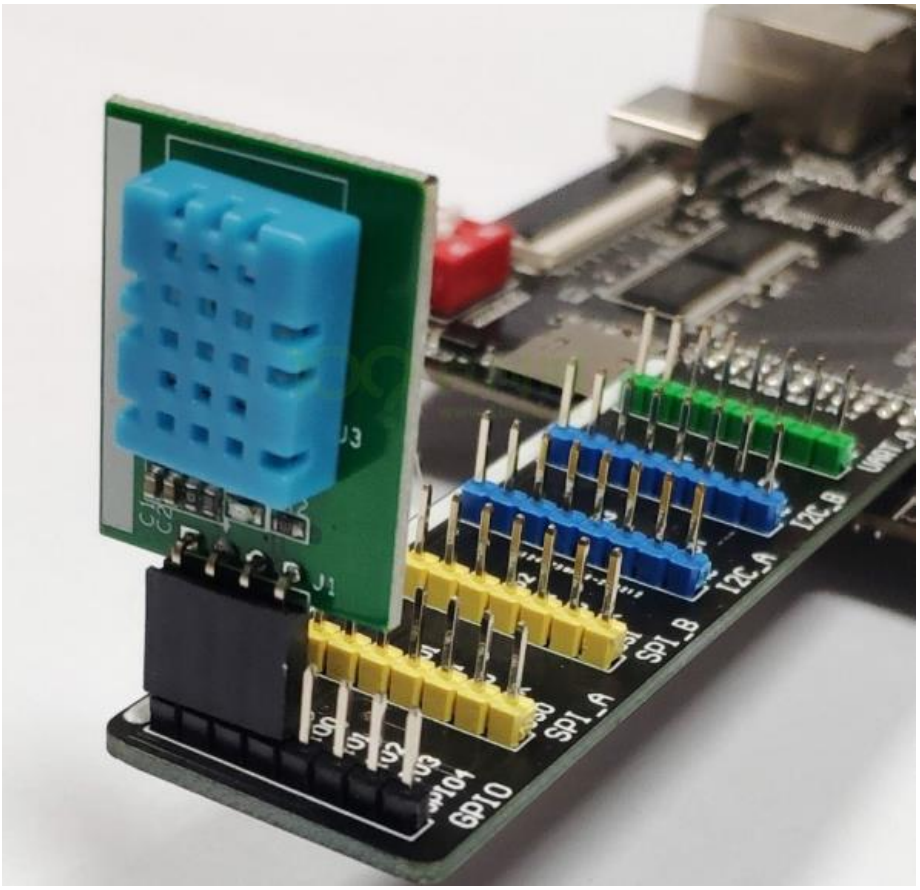
替换开发板“/root/LED_and_TempHumi”文件后重启。

等 QT 界面出现后，点击按键即可操作 LED。

4.3 使用温湿度传感器

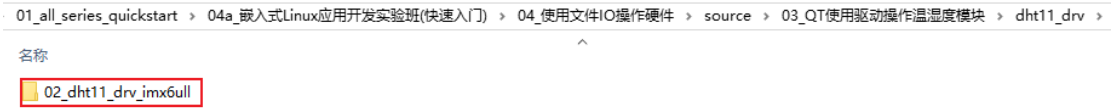
4.3.1 使用驱动程序

对于100ASK_IMX6ULL_Pro或于100ASK_IMX6ULL_Mini开发板，如下接DHT11温湿度模块：

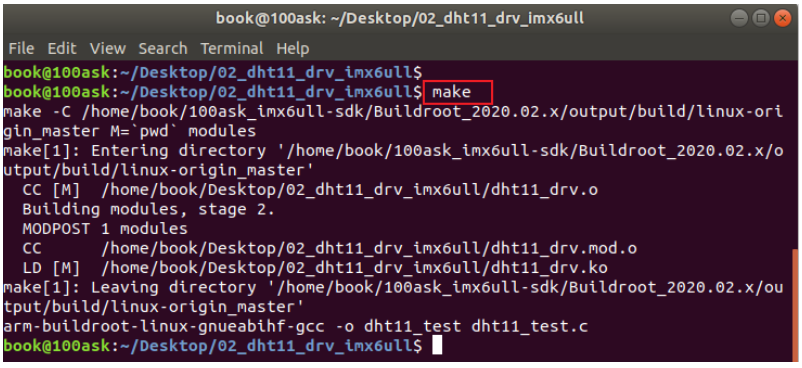


在驱动程序实验班里讲解驱动，本教程是“应用开发实验班（快速入门）”，所以这节课程只是使用驱动，不讲解驱动。

驱动程序：



在 Ubuntu 上，进入源码目录，执行 make 命令即可编译：



注意，如果使用 100ASK_IMX6ULL_Mini 开发板，需要修改 Makefile，指定内核目录，然后再执行 make 命令：

```
# 1. 使用不同的开发板内核时，一定要修改KERN_DIR
# 2. KERN_DIR中的内核要事先配置、编译，为了能编译内核，要先设置下列环境变量:
# 2.1 ARCH,          比如: export ARCH=arm64
# 2.2 CROSS_COMPILE, 比如: export CROSS_COMPILE=aarch64-linux-gnu-
# 2.3 PATH,          比如: export PATH=$PATH:/home/book/100ask_roc-rk3399-pc/ToolChain-6.3.1/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin
# 注意: 不同的开发板不同的编译器上述3个环境变量不一定相同,
#       请参考各开发板的高级用户使用手册

#KERN_DIR =  /home/book/100ask_imx6ull-sdk/Linux-4.9.88 # 板子所用内核源码的目录
#KERN_DIR =  /home/book/100ask_imx6ull_mini-sdk/Buildroot_2020.02.x/output/build/linux-origin_master
KERN_DIR =  /home/book/100ask_imx6ull-sdk/Buildroot_2020.02.x/output/build/linux-origin_master

all:
        make -C $(KERN_DIR) M=`pwd` modules
        $(CROSS_COMPILE)gcc -o dht11_test dht11_test.c
clean:
        make -C $(KERN_DIR) M=`pwd` modules clean
        rm -rf modules.order  dht11_test

# 参考内核源码drivers/char/ipmi/Makefile
# 要想把a.c, b.c编译成ab.ko, 可以这样指定:
# ab-y := a.o b.o
# obj-m += ab.o


obj-m += dht11_drv.o
```

如果使用100ASK_IMX6ULL_Mini
红框里第1行的"#"去掉，
红框里第2行前面加上"#"

把驱动程序 dht11_drv.ko、dht11_test 通过 ADB 上传到开发板，然后执行如下命令可以看到不断打印温湿度的值：

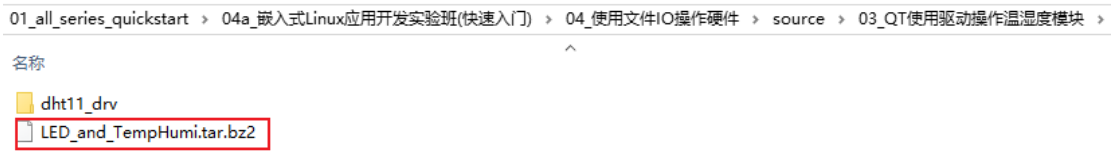
```
执行如下命令
insmod  /root/dht11_drv.ko
ls /dev/mydht11
/root/dht11_test  /dev/mydht11
```

如下图所示：

```
book@100ask: ~/Desktop/02_dht11_drv_imx6ull
File  Edit  View  Search  Terminal  Help
sh-5.0#
sh-5.0# insmod  /root/dht11_drv.ko
sh-5.0# ls /dev/mydht11
/dev/mydht11
sh-5.0# /root/dht11_test  /dev/mydht11
get Humidity: 76, Temperature : 31
get Humidity: 51, Temperature : 30
get Humidity: 51, Temperature : 30
get Humidity: 51, Temperature : 30
```

4.3.2 编写应用程序

本节源码为：



1. 创建 QT 线程

先在“dht11_thread.cpp、dht11_thread.h”里定义、实现DHT11Thread类。

dht11_thread.h代码如下：

```
#ifndef DHT11_THREAD_H
#define DHT11_THREAD_H
#include <QThread>
#include <QLabel>

// 1. 创建继承自QThread的子类
class DHT11Thread : public QThread {
public:
    void run() override ;
    void SetLables(QLabel *labelHumi, QLabel *labelTemp);
private:
    QLabel *labelHumi;
    QLabel *labelTemp;
};

#endif // DHT11_THREAD_H
```

dht11_thread.cpp代码如下：

```
#include <QThread>
#include <QDebug>
#include "dht11_thread.h"
#include "dht11.h"

// 2. 重写run()函数，这里是线程要执行的代码
void DHT11Thread::run()
{
    char humi;
    char temp;

    char buf[20];

    dht11_init();

    while(1) {
```

```
        if (0 == dht11_read(&humi, &temp))
        {
            /* display on qt */
            sprintf(buf, "%d%%", humi);
            labelHumi->setText(buf);
            sprintf(buf, "%d", temp);
            labelTemp->setText(buf);

            msleep(1000); // 线程休眠1秒
        }
    }
}

void DHT11Thread::SetLables(QLabel *labelHumi, QLabel *labelTemp)
{
    this->labelHumi = labelHumi;
    this->labelTemp = labelTemp;
}
```

然在main.cpp里创建线程，代码如下：

```
06 int main(int argc, char *argv[])
07 {
08     QLabel *labelHumi;
09     QLabel *labelTemp;
10
11     /* 1. init LED */
12     led_init();
13
14     QApplication a(argc, argv);
15     MainWindow w;
16     w.show();
17
18     labelHumi = w.GetHumiLabel();
19     labelTemp = w.GetTempLabel();
20
21
22     /* create dht11 thread */
23     // 3. 实例化线程并启动
24     DHT11Thread thread;
25     thread.SetLables(labelHumi, labelTemp);
26     thread.start();
27
28     return a.exec();
```

```
29 }
```

第24、26行，创建DHT11Thread实例化对象，并且启动线程。

2．读取温湿度值

代码为“dht11.cpp、dht11.h”。

dht11.cpp代码如下：

```
static int fd;

void dht11_init(void)
{
    fd = open("/dev/mydht11", O_RDWR | O_NONBLOCK);
    if (fd < 0)
    {
        qDebug()<<"open /dev/mydht11 failed";
    }
}

int dht11_read(char *humi, char *temp)
{
    char buf[2];

    if (2 == read(fd, buf, 2))
    {
        *humi = buf[0];
        *temp = buf[1];
        return 0;
    }
    else
    {
        return -1;
    }
}
```

3．修改图标文字

在mainwindow.h中添加了2个Qlabel成员、获得它们的函数，代码如下：

```
#include <QMainWindow>
#include <QLabel>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
    QLabel *GetHumiLabel();
    QLabel *GetTempLabel();


private slots:
    void on_pushButton_clicked();
private:
    Ui::MainWindow *ui;
    QLabel *labelHumi;
    QLabel *labelTemp;
};
#endif // MAINWINDOW_H
```

在mainwindow.cpp中找到label、实现对应的Get函数：

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
                        根据名字获得label
    labelHumi = this->findChild<QLabel*>("label");
    labelTemp = this->findChild<QLabel*>("label_2");
}
          别的代码可以通过这个函数获得湿度label

QLabel *MainWindow::GetHumiLabel()
{
    return labelHumi;
}
                别的代码可以通过这个函数获得温度label

QLabel *MainWindow::GetTempLabel()
{
    return labelTemp;
}
```

在main.cpp中从mainwindow获得label，并且设置给DHT11Thread：

```
int main(int argc, char *argv[])
{
    QLabel *labelHumi;
    QLabel *labelTemp;

    /* 1. init LED */
    led_init();

    QApplication a(argc, argv);
    MainWindow w;
    w.show();
                                从mainwindow里获得label
    labelHumi = w.GetHumiLabel();
    labelTemp = w.GetTempLabel();


    /* create dht11 thread */
    // 3. 实例化线程并启动
    DHT11Thread thread;         提供给DHT11Thread
    thread.SetLables(labelHumi, labelTemp);
    thread.start();

    return a.exec();
}
```

在dht11_thread.cpp里，读取DHT11的数值，设置label：

```
// 2. 重写run()函数，这里是线程要执行的代码
void DHT11Thread::run()
{
    char humi;
    char temp;

    char buf[20];

    dht11_init();

    while(1) {
        if (0 == dht11_read(&humi, &temp))   2.读取DHT11数值
        {
            /* display on qt */
            sprintf(buf, "%d%%", humi);      3.修改湿度label的文本
            labelHumi->setText(buf);
            sprintf(buf, "%d", temp);        4.修改温度label的文本
            labelTemp->setText(buf);

            msleep(1000); // 线程休眠1秒
        }
    }
}

void DHT11Thread::SetLables(QLabel *labelHumi, QLabel *labelTemp)
{
    this->labelHumi = labelHumi;     1.记录mainwindow的label
    this->labelTemp = labelTemp;
}
```

4.3.3 上机实验

确保驱动程序dht11_drv.ko、可执行程序LED_and_TempHumi放在开发板的/root目录下。

修改开发板“/etc/init.d/rcS”，加载驱动：

```
#!/bin/sh


# Start all init scripts in /etc/init.d
# executing them in numerical order.
#

insmod /root/led_drv.ko
insmod /root/dht11_drv.ko

psplash -n &
for i in /etc/init.d/S??* ;do
```

重启开发板，即可看到QT界面可以显示温湿度值。

第5章 使用 JsonRPC 实现前后台

5.1 把程序拆分为前后台

5.1.1 为何要拆分？

对于比较复杂的程序，前台界面显示、后台程序由不同的团队进行开发，双方定义好交互的接口即可。这样，前台、后台程序可以分别独立开发，降低相互之间的依赖。

比如：

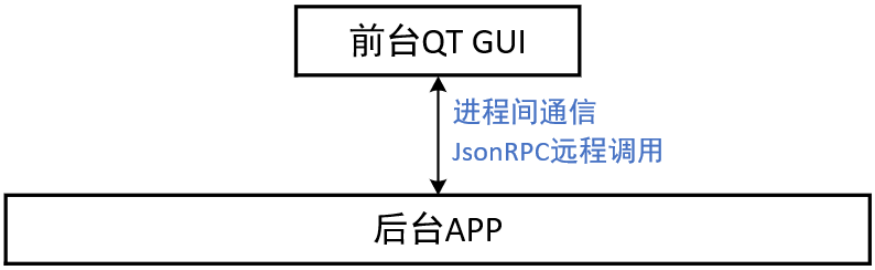
① 当更改硬件，比如更换LED引脚时，前台程序无需改变，只需要修改后台程序

② 想调整界面时，只需要修改前台程序，无需修改后台程序

5.1.2 如何拆分？

前台程序、后台程序分别属于不同的“进程”，它们之间的交互需要通过“进程间通信”来实现，比如：网络通信、管道、共享内存等等。

本课程使用基于网络通信的“JsonRPC远程调用”来实现前后台程序的交互：



RPC 是远程过程调用（Remote Procedure Call）的意思，而 json 是比较流行的传递信息的格式。

5.2 网络通信概述

5.2.1 IP 和端口

所有的数据传输，都有三个要素 ：源、目的、长度。
怎么表示源或者目的呢？请看图 5.1：



图 5.1 网络源和目的

所以，在网络传输中需要使用“IP 和端口”来表示源或目的。

5.2.2 网络传输中的 2 个对象：server 和 client

我们经常访问网站，这涉及 2 个对象：网站服务器，浏览器。网站服务器平时安静地呆着，浏览器主动发起数据请求。网站服务器、浏览器可以抽象成 2 个软件的概念：server 程序、client 程序。

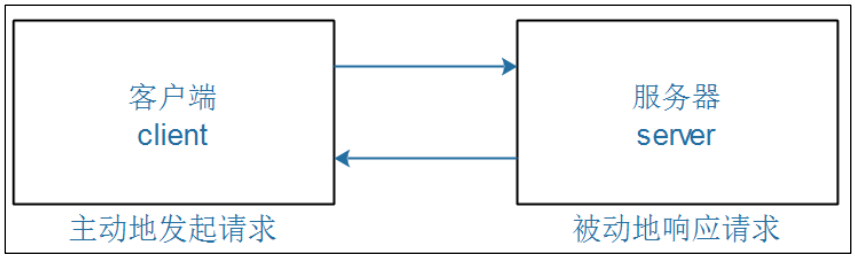


图 5.2 网络客户端和服务器

5.2.3 两种传输方式：TCP/UDP

在一般的网络书籍中，网络协议被分为 5 层，如图 5.3 所示：



图 5.3 网络协议层

应用层：它是体系结构中的最高层，直接为用户的应用进程（例如电子邮件、文件传输和终端仿真）提供服务。在因特网中的应用层协议很多，如支持万维网应用的 HTTP 协议，支持电子邮件的 SMTP 协议，支持文件传送的 FTP 协议，DNS，POP3，SNMP，Telnet 等等。

运输层：负责向两个主机中进程之间的通信提供服务。

运输层主要使用以下两种协议：

① 传输控制协议 TCP(Transmission Control Protocol)：面向连接的，数据传输的单位是报文段，能够提供可靠的交付。

② 用户数据包协议 UDP(User Datagram Protocol)：无连接的，数据传输的单位是用户数据报，不保证提供可靠的交付，只能提供“尽最大努力交付”。

- 网络层：负责将被称为数据包（datagram）的网络层分组从一台主机移动到另一台主机。
- 链路层：因特网的网络层通过源和目的地之间的一系列路由器路由数据报。
- 物理层：在物理层上所传数据的单位是比特。物理层的任务就是透明地传送比特流。

这些层对于初学者来说很难理解，我们只需要知道：我们需要使用“运输层”编写应用程序，我们的应用程序位于“应用层”。

使用“运输层”时，可以选择 TCP 协议，也可以选择 UDP 协议。

a) TCP 和 UDP 原理上的区别

TCP 向它的应用程序提供了面向连接的服务。这种服务有 2 个特点：可靠传输、流量控制（即发送方/接收方速率匹配）。它包括了应用层报文划分为短报文，并提供拥塞控制机制。

UDP 协议向它的应用程序提供无连接服务。它没有可靠性，没有流量控制，也没有拥塞控制。

b) 为何存在 UDP 协议

既然 TCP 提供了可靠数据传输服务，而 UDP 不能提供，那么 TCP 是否总是首选呢？

答案是否定的，因为有许多应用更适合用 UDP，举个例子：视频通话时，使用 UDP，偶尔的丢包、偶尔的花屏时可以忍受的；如果使用 TCP，每个数据包都要确保可靠传输，当它出错时就重传，这会导致后续的数据包被阻滞，视频效果反而不好。

使用 UDP 时，有如下特点：

① 关于何时发送什么数据控制的更为精细

采用 UDP 时只要应用进程将数据传递给 UDP，UDP 就会立即将其传递给网络层。而 TCP 有重传机制，而不管可靠交付需要多长时间。但是实时应用通常不希望过分的延迟报文段的传送，且能容忍一部分数据丢失。

② 无需建立连接，不会引入建立连接时的延迟。

③ 无连接状态，能支持更多的活跃客户。

④ 分组首部开销较小。

c) TCP/UDP 网络通信大概交互图

下面我们分别画出运用 TCP 协议和运用 UDP 协议的客户端和服务器大概交互图。

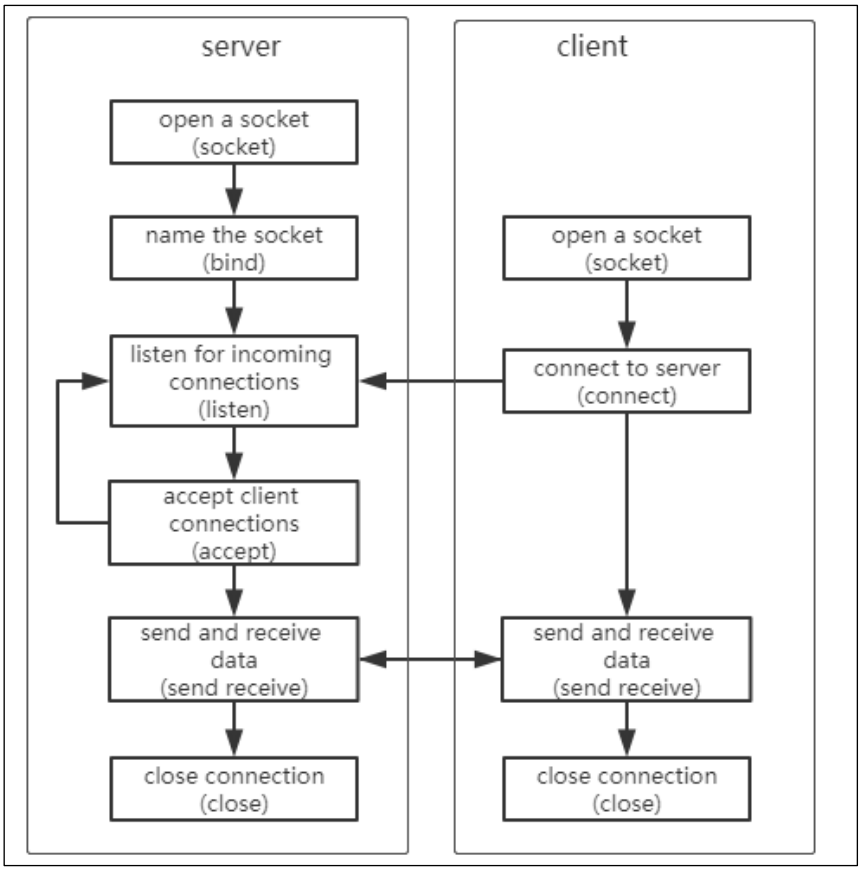


图 5.4 面向连接的 TCP 流模式

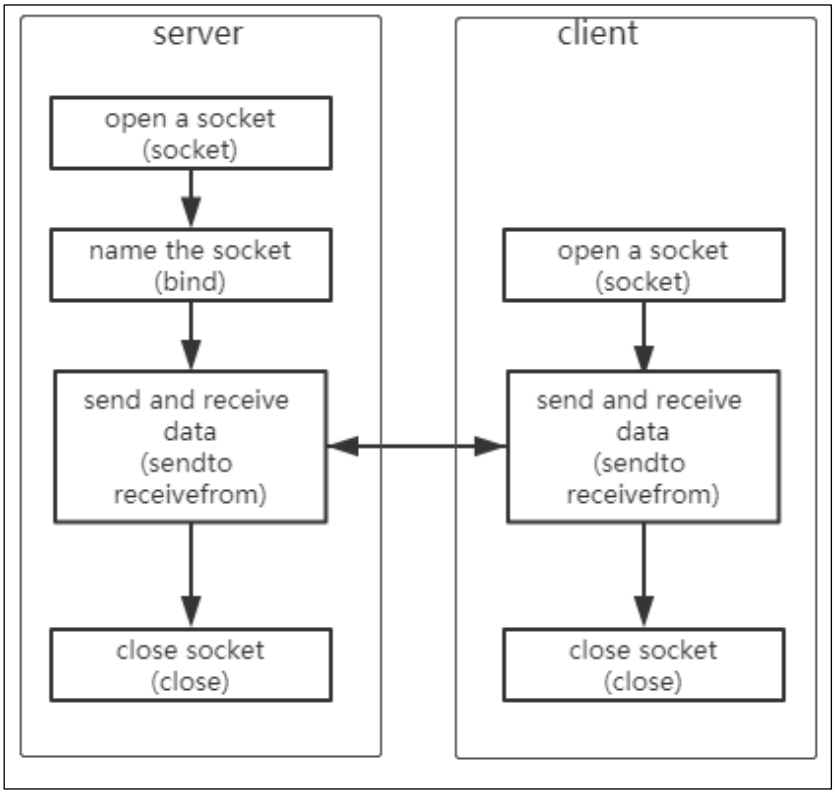


图 5.5UDP 用户数据包模式

5.3 网络编程主要函数介绍

5.3.1 socket 函数

```
int socket(int domain, int type,int protocol);
```

此函数用于创建一个套接字。

- **domain** 是网络程序所在的主机采用的通讯协族(AF_UNIX 和 AF_INET 等)。
  - AF_UNIX 只能够用于单一的 Unix 系统进程间通信，而 AF_INET 是针对 Internet 的，因而可以允许远程通信使用。
- **type** 是网络程序所采用的通讯协议(SOCK_STREAM,SOCK_DGRAM 等)。
  - SOCK_STREAM 表明用的是 TCP 协议，这样会提供按顺序的，可靠，双向，面向连接的比特流。
  - SOCK_DGRAM 表明用的是 UDP 协议，这样只会提不可靠，无连接的通信。
- 关于 **protocol**，由于指定了 type，所以这个地方一般只要用 0 来代替就可以了。

此函数执行成功时返回文件描述符，失败时返回-1,看 errno 可知道出错的详细情况。

5.3.2 bind 函数

```
int bind(int sockfd, struct sockaddr *my_addr, int addrlen);
```

从函数用于将地址绑定到一个套接字。

- **sockfd** 是由 socket 函数调用返回的文件描述符。
- **my_addr** 是一个指向 sockaddr 的指针。
- **addrlen** 是 sockaddr 结构的长度。

sockaddr 的定义：

```
struct sockaddr{
unisgned short  as_family;
char sa_data[14];
};
```

不过由于系统的兼容性,我们一般使用另外一个结构(struct sockaddr_in) 来代替。

sockaddr_in 的定义：

```
struct sockaddr_in{
unsigned short   sin_family;
unsigned short     sin_port;
struct in_addr   sin_addr;
unsigned char   sin_zero[8];
}
```

如果使用 Internet 所以 sin_family 一般为 AF_INET。

- sin_addr 设置为 INADDR_ANY 表示可以和任何的主机通信。
- sin_port 是要监听的端口号。
- bind 将本地的端口同 socket 返回的文件描述符捆绑在一起. 成功是返回 0,失败的情况和 socket 一样。

5.3.3 listen 函数

```
int listen(int sockfd,int backlog);
```

此函数宣告服务器可以接受连接请求。

- **sockfd** 是 bind 后的文件描述符。
- **backlog** 设置请求排队的最大长度。当有多个客户端程序和服务端相连时，使用这个表示可以介绍的排队长度。
- listen 函数将 bind 的文件描述符变为监听套接字，返回的情况和 bind 一样。

5.3.4 accept 函数

```
int accept(int sockfd, struct sockaddr *addr,int *addrlen);
```

服务器使用此函数获得连接请求，并且建立连接。

- **sockfd** 是 listen 后的文件描述符。
- **addr，addrlen** 是用来给客户端的程序填写的,服务器端只要传递指针就可以了， bind,listen 和 accept 是服务器端用的函数。
- accept 调用时，服务器端的程序会一直阻塞到有一个客户程序发出了连接。 accept 成功时返回最后的服务器端的文件描述符，这个时候服务器端可以向该描述符写信息了，失败时返回-1 。

5.3.5 connect 函数

```
int connect(int sockfd, struct sockaddr * serv_addr,int addrlen);
```

可以用connect建立一个连接，在connect中所指定的地址是想与之通信的服务器的地址。

- **sockfd** 是 socket 函数返回的文件描述符。
- **serv_addr** 储存了服务器端的连接信息，其中 sin_add 是服务端的地址。
- **addrlen** 是 serv_addr 的长度
- connect 函数是客户端用来同服务端连接的.成功时返回 0，sockfd 是同服务端通讯的文件描述符，失败时返回-1。

5.3.6 send 函数

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

- **sockfd** 指定发送端套接字描述符；
- **buf** 指明一个存放应用程序要发送数据的缓冲区；
- **len** 指明实际要发送的数据的字节数；
- **flags** 一般置 0。
- 客户或者服务器应用程序都用 send 函数来向 TCP 连接的另一端发送数据

5.3.7 recv 函数

```
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

- **sockfd** 指定接收端套接字描述符；

- **buf** 指明一个缓冲区，该缓冲区用来存放 recv 函数接收到的数据；
- **len** 指明 buf 的长度；
- **flags** 一般置 0。
- 客户或者服务器应用程序都用 recv 函数从 TCP 连接的另一端接收数据。

5.3.8 recvfrom 函数

```
    ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                             struct sockaddr *src_addr, socklen_t *addrl
en);
```

- recvfrom 通常用于无连接套接字，因为此函数可以获得发送者的地址。
- **src_addr** 是一个 struct sockaddr 类型的变量，该变量保存源机的 IP 地址及端口号。
- **addrlen** 常置为 sizeof （struct sockaddr）。

5.3.9 sendto 函数

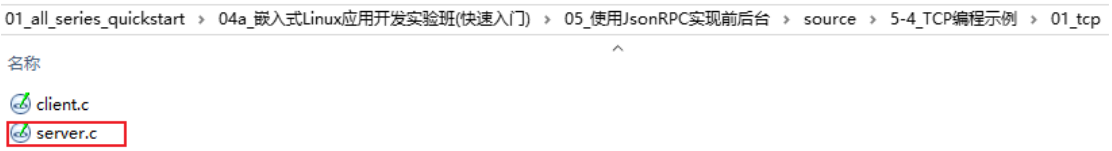
```
    ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
                           const struct sockaddr *dest_addr, socklen_t a
ddrlen);
```

- sendto 和 send 相似，区别在于 sendto 允许在无连接的套接字上指定一个目标地址。
- **dest_addr** 表示目地机的 IP 地址和端口号信息，
- **addrlen** 常常被赋值为 sizeof （struct sockaddr）。
- **sendto** 函数也返回实际发送的数据字节长度或在出现发送错误时返回-1。

5.4 TCP 编程示例

5.4.1 server 程序

源码位于：



代码如下：

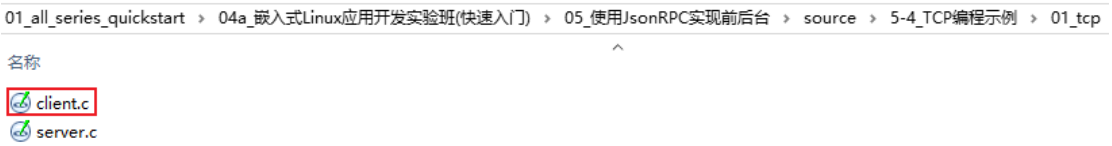
```
01 #include <sys/types.h>          /* See NOTES */
02 #include <sys/socket.h>
03 #include <string.h>
04 #include <sys/socket.h>
05 #include <netinet/in.h>
06 #include <arpa/inet.h>
07 #include <unistd.h>
08 #include <stdio.h>
```

```
09 #include <signal.h>
10
11
12 /* socket
13  * bind
14  * listen
15  * accept
16  * send/recv
17  */
18
19 #define SERVER_PORT 8888
20 #define BACKLOG     10
21
22 int main(int argc, char **argv)
23 {
24     int iSocketServer;
25     int iSocketClient;
26     struct sockaddr_in tSocketServerAddr;
27     struct sockaddr_in tSocketClientAddr;
28     int iRet;
29     int iAddrLen;
30
31     int iRecvLen;
32     unsigned char ucRecvBuf[1000];
33
34     int iClientNum = -1;
35
36     signal(SIGCHLD,SIG_IGN);
37
38     iSocketServer = socket(AF_INET, SOCK_STREAM, 0);
39     if (-1 == iSocketServer)
40     {
41             printf("socket error!\n");
42             return -1;
43     }
44
45     tSocketServerAddr.sin_family      = AF_INET;
46     tSocketServerAddr.sin_port        = htons(SERVER_PORT);  /* host
 to net, short */
47     tSocketServerAddr.sin_addr.s_addr = INADDR_ANY;
48     memset(tSocketServerAddr.sin_zero, 0, 8);
49
50     iRet = bind(iSocketServer, (const struct sockaddr *)&tSocketServ
erAddr, sizeof(struct sockaddr));
51     if (-1 == iRet)
52     {
53             printf("bind error!\n");
54             return -1;
55     }
56
57     iRet = listen(iSocketServer, BACKLOG);
58     if (-1 == iRet)
59     {
60             printf("listen error!\n");
61             return -1;
62     }
```

```
63
64      while (1)
65      {
66              iAddrLen = sizeof(struct sockaddr);
67              iSocketClient = accept(iSocketServer, (struct sockaddr *)
&tSocketClientAddr, &iAddrLen);
68              if (-1 != iSocketClient)
69              {
70                      iClientNum++;
71                      printf("Get connect from client %d : %s\n",  iClie
ntNum, inet_ntoa(tSocketClientAddr.sin_addr));
72                      if (!fork())
73                      {
74                              /* 子进程的源码 */
75                              while (1)
76                              {
77                                      /* 接收客户端发来的数据并显示出来 */
78                                      iRecvLen = recv(iSocketClient, ucRe
cvBuf, 999, 0);
79                                      if (iRecvLen <= 0)
80                                      {
81                                              close(iSocketClient);
82                                              return -1;
83                                      }
84                                      else
85                                      {
86                                              ucRecvBuf[iRecvLen] = '\0';
87                                              printf("Get Msg From Client
 %d: %s\n", iClientNum, ucRecvBuf);
88                                      }
89                              }
90                      }
91              }
92      }
93
94      close(iSocketServer);
95      return 0;
96 }
```

5.4.2 client 程序

源码位于：



代码如下：

```
01 #include <sys/types.h>          /* See NOTES */
02 #include <sys/socket.h>
03 #include <string.h>
04 #include <sys/socket.h>
05 #include <netinet/in.h>
06 #include <arpa/inet.h>
07 #include <unistd.h>
08 #include <stdio.h>
```

```
09
10 /* socket
11  * connect
12  * send/recv
13  */
14
15 #define SERVER_PORT 8888
16
17 int main(int argc, char **argv)
18 {
19     int iSocketClient;
20     struct sockaddr_in tSocketServerAddr;
21
22     int iRet;
23     unsigned char ucSendBuf[1000];
24     int iSendLen;
25
26     if (argc != 2)
27     {
28             printf("Usage:\n");
29             printf("%s <server_ip>\n", argv[0]);
30             return -1;
31     }
32
33     iSocketClient = socket(AF_INET, SOCK_STREAM, 0);
34
35     tSocketServerAddr.sin_family      = AF_INET;
36     tSocketServerAddr.sin_port        = htons(SERVER_PORT);  /* host
 to net, short */
37     //tSocketServerAddr.sin_addr.s_addr = INADDR_ANY;
38     if (0 == inet_aton(argv[1], &tSocketServerAddr.sin_addr))
39     {
40             printf("invalid server_ip\n");
41             return -1;
42     }
43     memset(tSocketServerAddr.sin_zero, 0, 8);
44
45
46     iRet = connect(iSocketClient, (const struct sockaddr *)&tSocketS
erverAddr, sizeof(struct sockaddr));
47     if (-1 == iRet)
48     {
49             printf("connect error!\n");
50             return -1;
51     }
52
53     while (1)
54     {
55             if (fgets(ucSendBuf, 999, stdin))
56             {
57                     iSendLen = send(iSocketClient, ucSendBuf, strlen(u
cSendBuf), 0);
58                     if (iSendLen <= 0)
59                     {
60                             close(iSocketClient);
61                             return -1;
```

```
62                     }
63             }
64     }
65
66     return 0;
67 }
68
```

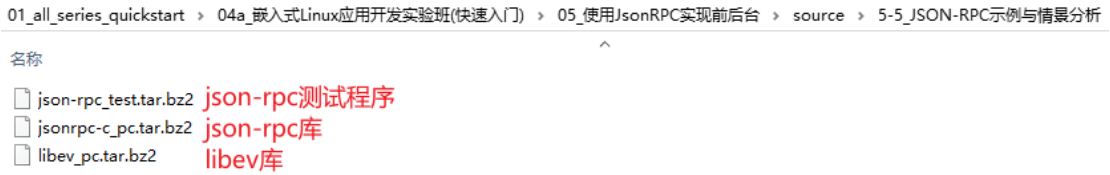
5.4.3 上机实验

5.5 JSON-RPC 示例与情景分析

理解了 JSON-RPC 的使用，再去改造 QT 程序会容易很多。

使用 C 语言实现的 JSON-RPC 代码位于 GIT 仓库：https://github.com/hmng/jsonrpc-c。它依赖于 libev 库。

本节程序在如下目录：



5.5.1 JSON 是什么

JSON（JavaScript Object Notation，JavaScript 对象表示法）是基于 ECMAScript 的一个子集设计的，是一种开放标准的文件格式和数据交换格式，它易于人阅读和编写，同时也易于机器解析和生成。JSON 独立于语言设计，很多编程语言都支持 JSON 格式的数据交换。JSON 是一种常用的数据格式，在电子数据交换中有多种用途，包括与服务器之间的 Web 应用程序的数据交换。其简洁和清晰的层次结构有效地提升了网络传输效率，使其成为理想的数据交换语言。其文件通常使用扩展名.json。

它的关键成员是“name:value”，value 有多种形式。

```
{ "name": "John", "age": 30, "isStudent": false , "ptr": null }
```

上述例子中，有 3 种取值："John"是字符串，30 是整数，false 是 bool 类型，null 是空值。

值的类型也可以是数组，比如：

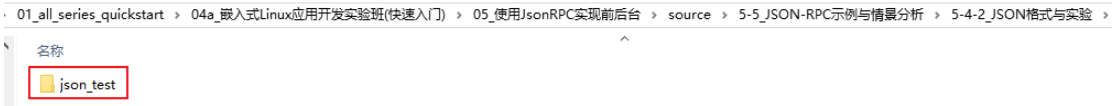
```
{ "fruits": ["apple","banana", "cherry"] }
```

值的类型，还可以是一个 JSON 对象，比如：

```
{
  "title":"JSON Example",
  "author": {
      "name":"John Doe",
      "age": 35,
      "isVerified":true
    },
    "tags":["json", "syntax", "example"],
    "rating": 4.5,
    "isPublished":false,
    "comments": null
}
```

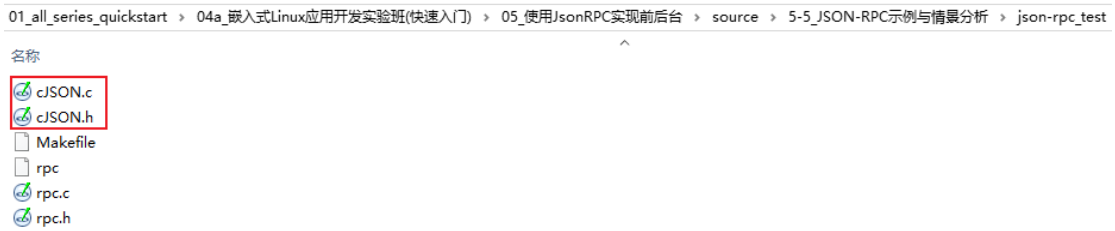
5.5.2 常用的 JSON 函数

示例代码：



进入“json_test”目录后，执行 make 命令，即可运行“test”程序。

JSON 函数在如下文件中：



1. 创建 JSON

使用字符串创建一个 cJSON 结构体：

```
CJSON_PUBLIC(cJSON *) cJSON_Parse(const char *value);
```

创建空的 cJSON 结构体：

```
CJSON_PUBLIC(cJSON *) cJSON_CreateObject(void);
```

往 cJSON 结构体里添加内容：

```
CJSON_PUBLIC(cJSON*) cJSON_AddNullToObject(cJSON * const object, const
char * const name);
CJSON_PUBLIC(cJSON*) cJSON_AddTrueToObject(cJSON * const object, const
char * const name);
CJSON_PUBLIC(cJSON*) cJSON_AddFalseToObject(cJSON * const object, const
 char * const name);
CJSON_PUBLIC(cJSON*) cJSON_AddBoolToObject(cJSON * const object, const
char * const name, const cJSON_bool boolean);
CJSON_PUBLIC(cJSON*) cJSON_AddNumberToObject(cJSON * const object, cons
t char * const name, const double number);
CJSON_PUBLIC(cJSON*) cJSON_AddStringToObject(cJSON * const object, cons
t char * const name, const char * const string);
CJSON_PUBLIC(cJSON*) cJSON_AddRawToObject(cJSON * const object, const c
har * const name, const char * const raw);
CJSON_PUBLIC(cJSON*) cJSON_AddObjectToObject(cJSON * const object, cons
t char * const name);
CJSON_PUBLIC(cJSON*) cJSON_AddArrayToObject(cJSON * const object, const
 char * const name);
```

2. 获得 JSON

根据名字获得 cJSON 结构体：

```
CJSON_PUBLIC(cJSON *) cJSON_GetObjectItem(const cJSON * const object, c
onst char * const string);
```

根据数组下标获得 cJSON 结构体：

```
CJSON_PUBLIC(cJSON *) cJSON_GetArrayItem(const cJSON *array, int inde
x);
```

3. 获得 JSON 的值

cJSON 结构体里存有值：valuestring、valueint、valuedouble，直接使用即可：

```
101:
102: /* The cJSON structure: */
103: typedef struct cJSON
104: {
105:     /* next/prev allow you to walk array/object chains. Alternatively, use GetArraySize/GetArrayItem/GetObjectItem */
106:     struct cJSON *next;
107:     struct cJSON *prev;
108:     /* An array or object item will have a child pointer pointing to a chain of the items in the array/object. */
109:     struct cJSON *child;
110:
111:     /* The type of the item, as above. */
112:     int type;
113:
114:     /* The item's string, if type==cJSON_String  and type == cJSON_Raw */
115:     char *valuestring;
116:     /* writing to valueint is DEPRECATED, use cJSON_SetNumberValue instead */
117:     int valueint;
118:     /* The item's number, if type==cJSON_Number */
119:     double valuedouble;
120:
121:     /* The item's name string, if this item is the child of, or is in the list of subitems of an object. */
122:     char *string;
123: } « end cJSON »  cJSON;
```

示例：

```
cJSON *port = cJSON_GetObjectItem(params, "port");
if (port)
{
    tMQTTInfo.port = port->valueint;
}

cJSON *client_id = cJSON_GetObjectItem(params, "client_id");
if (client_id)
{
    strcpy(tMQTTInfo.client_id, client_id->valuestring);
}
```

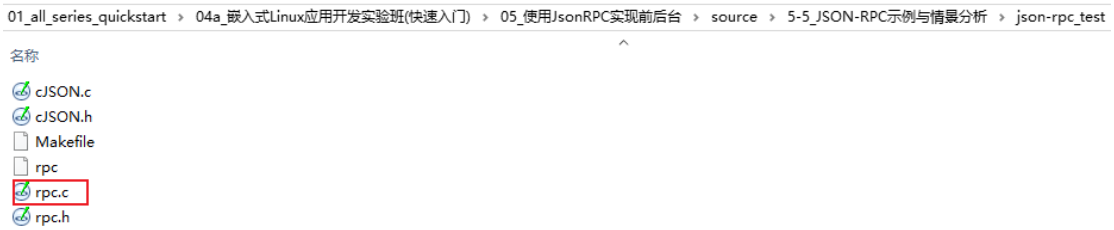
4. 删除 JSON

函数如下：

```
CJSON_PUBLIC(void) cJSON_Delete(cJSON *item);
```

5.5.3 服务器程序启动

服务器程序、客户端程序的源码都在如下文件里：



参考《5.5.5 上机实验》，启动服务器程序后，它的核心代码如下：

```
152 int RPC_Server_Init(void)
153 {
154     int err;
155
156     err = jrpc_server_init(&my_server, PORT);
157     if (err)
158     {
159         printf("jrpc_server_init err : %d\n", err);
160     }
161
```

```
162     jrpc_register_procedure(&my_server, say_hello, "sayHello", NULL
                );
163     jrpc_register_procedure(&my_server, add, "add", NULL );
164
165     jrpc_server_run(&my_server);
166     jrpc_server_destroy(&my_server);
167
168     return 0;
169 }
```

第 156 行，初始化服务器，使用端口“PORT”，它被定义为 1234（可以修改）。

第 162~163 行，注册 2 个函数：前台程序使用 RPC 传来参数“sayHello”时会导致 say_hello 程序被调用；前台程序使用 RPC 传来参数“add”时会导致 add 程序被调用。

第 165 行：循环等待前台程序发来请求，收到请求后处理请求。

5.5.4 客户端程序发出请求

假设执行了如下命令：

```
$ ./rpc add 3 4
```

它执行的代码如下：

```
193         int sum;
194         int fd = RPC_Client_Init();
195
196         if (fd < 0)
197         {
198             printf("RPC_Client_Init err : %d\n", fd);
199             return -1;
200         }
201
202         if (argc == 4 && !strcmp(argv[1], "add"))
203         {
204             int a = (int)strtoul(argv[2], NULL, 0);
205             int b = (int)strtoul(argv[3], NULL, 0);
206             int err = rpc_add(fd, a, b, &sum);
```

第 194 行，初始化客户端，就是连接服务器。

第 206 行：调用 rpc_add 函数。

rpc_add 函数代码如下：

```
16 int rpc_add(int iSocketClient, int a, int b, int *sum)
17 {
18     char buf[100];
19     int iLen;
20     sprintf(buf, "{\"method\": \"add\", \"params\": [%d,%d], \"id\":
                \"2\" }", a, b);
21     iLen = send(iSocketClient, buf, strlen(buf), 0);
22     if (iLen ==  strlen(buf))
23     {
24         while (1)
25         {
26             iLen = read(iSocketClient, buf, sizeof(buf));
27             buf[iLen] = 0;
28             if (iLen == 1 && (buf[0] == '\r' || buf[0] == '\n'))
29                 continue;
30             else
31                 break;
```

```
32          }
33
34          if (iLen > 0)
35          {
36              cJSON *root = cJSON_Parse(buf);
37              cJSON *result = cJSON_GetObjectItem(root, "result");
38              *sum = result->valueint;
39              cJSON_Delete(root);
40              return 0;
41          }
42          else
43          {
44              printf("read rpc reply err : %d\n", iLen);
45              return -1;
46          }
47      }
48      else
49      {
50          printf("send rpc request err : %d, %s\n", iLen, strerror(err
                    no));
51          return -1;
52      }
53 }
```

第 20 行：构造一个 JSON 字符串，里面有：

① “\"method\": \"add\"”：这表示要调用服务器的 add 函数

② “\"params\": [%d,%d]”：这表示传递给 add 函数的参数

③ “\"id\": \"2\"”：ID，这不重要。

5.5.5 服务器处理请求

服务器接收到网络数据后，根据里面的“method”值调用 add 函数。add 函数代码如下：

```
117 cJSON * add(jrpc_context * ctx, cJSON * params, cJSON *id) {
118     char buf[100];
119     cJSON * a = cJSON_GetArrayItem(params,0);
120     cJSON * b = cJSON_GetArrayItem(params,1);
121     return cJSON_CreateNumber(a->valueint + b->valueint);
122 }
```

第 119~120 行：从 JSON 格式的参数里提取出 2 个整数。

第 121 行：求和，然后返回一个 JSON 格式的数据。

5.5.6 客户端程序解析数据

在《5.5.2 客户端发出请求》的第 36~38 行代码中，就是解析服务器程序返回的结构体。

5.5.7 上机实验

本节在 Ubuntu 中做实验。先编译 libev 库，再编译 jsonrpc 库，最后编译、执行测试程序 json-rpc_test。

假设这 3 个文件放在同一个目录下：

```
$ ls
jsonrpc-c_pc.tar.bz2  json-rpc_test.tar.bz2  libev_pc.tar.bz2
```

1．编译 libev 库

操作命令如下：

```
$ tar xjf libev_pc.tar.bz2
$ cd libev_pc/
$ CC=gcc ./configure --prefix=$PWD/tmp
$ make -j 16
$ make install
$ ls tmp/
include  lib  share
$ sudo cp -rf tmp/include/*   /usr/include/
$ sudo cp -rfd tmp/lib/*      /usr/lib
```

2．编译 jsonrpc 库

操作命令如下：

```
$ sudo apt install libtool
$ tar xjf jsonrpc-c_pc.tar.bz2
$ cd jsonrpc-c_pc/
$ autoreconf -i
$ CC=gcc ./configure --prefix=$PWD/tmp
$ make -j 16
$ make install
$ ls tmp/
include  lib
$ sudo cp -rf tmp/include/*   /usr/include/
$ sudo cp -rfd tmp/lib/*      /usr/lib
```

3．编译测试程序

操作命令如下：

```
$ tar xjf json-rpc_test.tar.bz2
$ cd json-rpc_test/
$ make
```

4．执行程序

操作命令如下：

```
$ ./rpc server &
[1] 15028
$ ./rpc add 2 6
sum = 8
$ ./rpc hello 100ask
Hello, 100ask
```

也可以执行这样的命令发出 RPC 请求：

```
echo "{\"method\": \"add\", \"params\": [2,4], \"id\": \"2\" }" | nc lo
                  calhost 1234
```

```
echo {\"method\": \"sayHello\"," "\"params\":" "{\"name\": \"100ask\"},
                    \"id\": \"2\" } | nc localhost 1234
```

注意：如果要重新运行 rpc 服务器程序，需要先关闭它，可以执行如下命令：

```
$ killall rpc
```

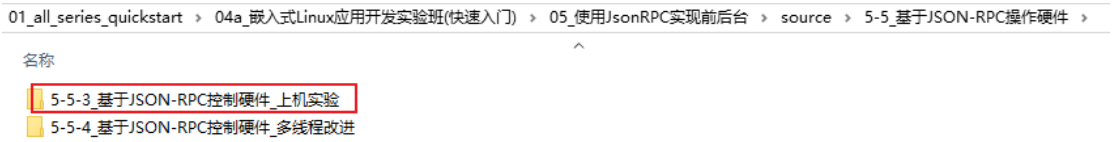
5.6 基于 JSON-RPC 操作硬件

5.6.1 功能

程序A：调用rpc_led_control发起远程调用，让程序B去控制LED。
程序A：调用rpc_dht11_read发起远程调用，让程序B去读取温湿度的值。

5.6.2 编写程序

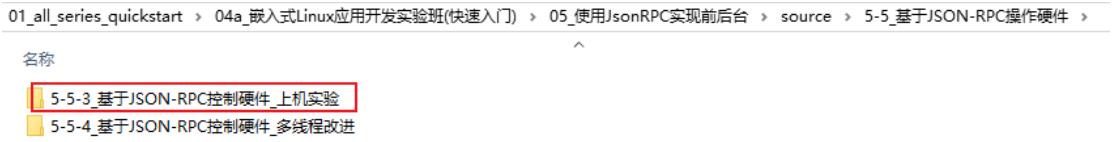
写好的程序在这里：



5.6.3 交叉编译程序与上机实验

本节在 Ubuntu 中交叉编译程序。先编译 libev 库，再编译 jsonrpc 库，最后编译、执行测试程序测试程序。

假设这 4 个文件或文件夹放在同一个目录下：



1. 编译 libev 库

操作命令如下：

```
$ tar xjf libev.tar.bz2
$ cd libev/
$ ./configure  --host=arm-buildroot-linux-gnueabihf  --prefix=$PWD/tmp
$ make -j 16
$ make install
$ ls tmp/
include  lib  share
```

2. 编译 jsonrpc 库

操作命令如下：

```
$ sudo apt install libtool
$ tar xjf jsonrpc-c.tar.bz2
$ cd jsonrpc-c/
$ autoreconf -i
$ ./configure  --host=arm-buildroot-linux-gnueabihf  --prefix=$PWD/tmp
                    CFLAGS="-I$PWD/../libev/tmp/include"  LDFLAGS="-L$P
                  WD/../libev/tmp/lib"

$ make -j 16
$ make install
$ ls tmp/
include  lib
```

3. 编译测试程序

操作命令如下：

```
$ cd  rpc_server
$ make
$ adb  push  rpc_server  /root
$ cd ../rpc_client
$ make
$ adb  push  rpc_client  /root
```

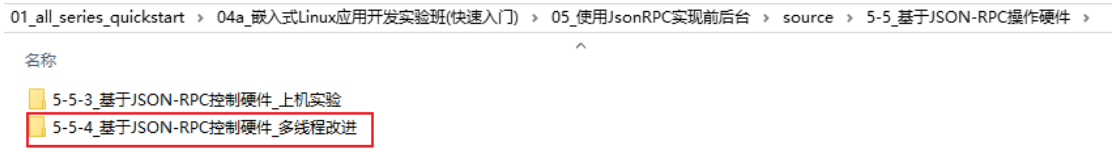
4. 执行程序

在开发板上，操作命令如下：

```
$ ./rpc_server &
$ ./rpc_client led 0
$ ./rpc_client led 1
$ ./rpc_client dht11
```

5.6.4 使用多线程改进程序

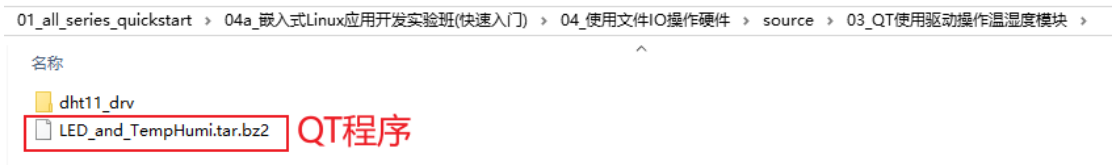
改好的程序在这里：



5.7 基于 JSON-RPC 改造 QT 程序

5.7.1 合并程序

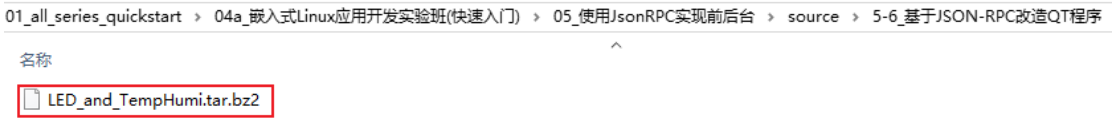
修改一下 QT 程序：



把里面的硬件操作函数，改为 rpc 远程调用，仿照以下的代码：



最后的代码为：



5.7.2 上机实验

可以使用我们实现制作好的 APP 和脚本：



在 Ubuntu 上通过 adb 命令把上述文件放到开发板上：

```
$ adb  push  LED_and_TempHumi   /root
$ adb  push  rpc_server         /root
$ adb  push  rcS       /etc/init.d
$ adb  push  S99myqt   /etc/init.d
```

然后在开发板上执行：

```
#  chmod  +x  /root/LED_and_TempHumi
#  chmod  +x  /root/rpc_server
#  chmod  +x  /etc/init.d/rcS
#  chmod  +x  /etc/init.d/S99myqt
```

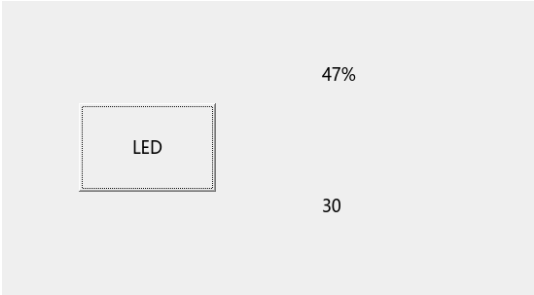
最后重启系统，就可以在 QT 界面进行操作了。

第6章 智能家居产品设计

6.1 项目展示与技术点

6.1.1 功能

① 本地控制：在开发板上运行 QT 程序，在界面控制 LED、显示温湿度值



② 微信小程序控制：在界面控制 LED、显示温湿度值



6.1.2 技术

① MQTT 网络技术
② 配置文件处理
③ RPC 远程调用

6.2 MQTT 开发基础

6.2.1 MQTT 概述

MQTT 协议全称是 Message Queuing Telemetry Transport，即“消息队列遥测传输协议”，它是物联网常用的应用层协议，运行在 TCP/IP 中的应用层中，依赖 TCP 协议，因此它具有非常高的可靠性。同时它是基于 TCP 协议的“客户端-服务器”模型发布/订阅主题消息的轻量级协议。

MQTT 协议提供一对多的消息发布，可以降低应用程序的耦合性，用户只需要编写极少量的应用代码就能完成一对多的消息发布与订阅，该协议是基于“客户端-服务器”模型，在协议中主要有三种身份：发布者（Publisher）、服务器（Broker）以及订阅者（Subscriber）。其中，MQTT 消息的发布者和订阅者都是客户端，服务器只是作为一个中转的存在，将发布者发布的消息进行转发给所有订阅该主题的订阅者；发布者可以发布在其权限之内的所有主题，并且消息发布者可以同时是订阅者，实现了生产者与消费者的脱耦，发布的消息可以同时被多个订阅者订阅。

MQTT 通信模型示意图如下：



MQTT 客户端的功能：

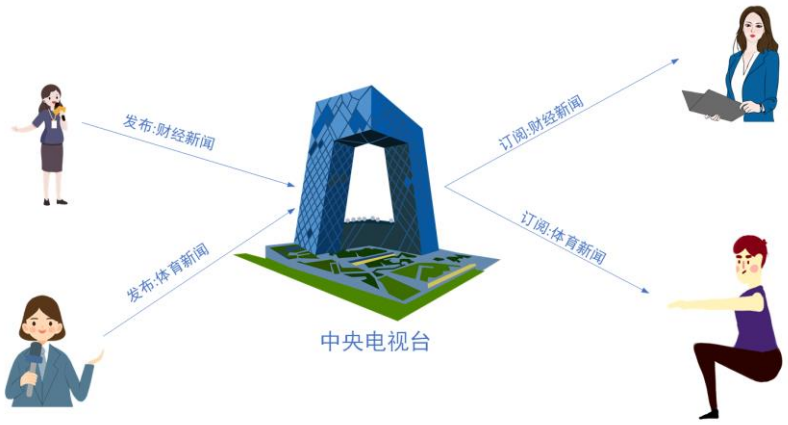
① 发布消息给其它相关的客户端。
② 订阅主题请求接收相关的应用消息。
③ 取消订阅主题请求移除接收应用消息。
④ 从服务端终止连接。

MQTT 服务器常被称为 Broker（消息代理），以是一个应用程序或一台设备，它一般为云服务器，比如 BTA 三巨头的一些物联网平台就是常使用 MQTT 协议，它是位于消息发布者和订阅者之间，以便用于接收消息并发送到订阅者之中，它的功能有：

① 接受来自客户端的网络连接请求。
② 接受客户端发布的应用消息。
③ 处理客户端的订阅和取消订阅请求。
④ 转发应用消息给符合条件的已订阅客户端（包括发布者自身）。

6.2.2 MQTT 协议形象理解

使用电视台、记者、观众三个角色来类比更容易理解：



电视台：在 MQTT 里被称为服务器(broker)，有如下作用

① 接受来自客户的网络连接； // 记者/观众连接进电视台
② 接受客户发布的应用信息； // 接受记者发布的消息
③ 处理来自客户端的订阅和退订请求； // 处理观众的订阅、退订请求
④ 向订阅的客户转发应用程序消息 // 给观众转发记者报道的消息

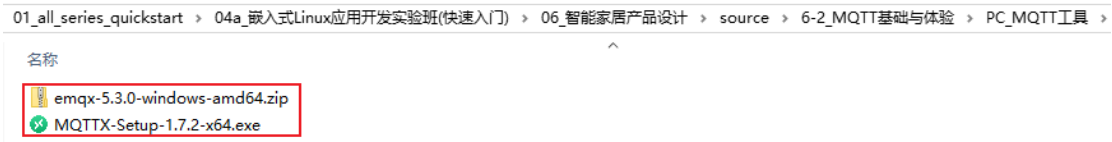
记者和观众都是客户端，记者也可以当观众，观众也可以当记者，有如下作用：

① 发布信息； // publish，记者
② 订阅消息；// Subscribe ，观众
③ 退订或删除消息；
④ 断开与服务器连接

6.2.3 PC MQTT 工具使用

1. 安装软件

软件在 GIT 仓库如下目录：



在 PC 上，需要运行 MQTT 服务器软件 emqx、MQTT 客户端软件 MQTTX。

对于 emqx，把“emqx-5.3.0-windows-amd64.zip”解压到一个无中文的目录即可。假设解压到了“d:\abc”下。

对于 MQTTX，双击安装“MQTTX-Setup-1.7.2-x64.exe”即可。

2. 启动与停止 MQTT 服务

假设 emqx 安装目录为“D:\abc\emqx-5.3.0-windows-amd64”，在命令行执行以下命令启动 MQTT 服务：

```
D:\abc\emqx-5.3.0-windows-amd64\bin\emqx    start
```

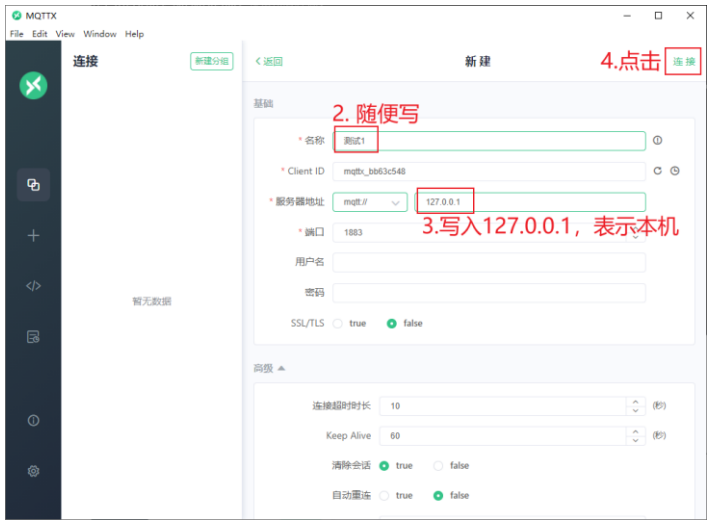
然后可通过浏览器访问“http://localhost:18083”以访问 EMQX Dashboard 管理控制台，进行设备连接与相关指标监控管理。默认用户名为“admin”，密码为“public”

如果要停止 MQTT 服务，在命令行下执行如下命令：

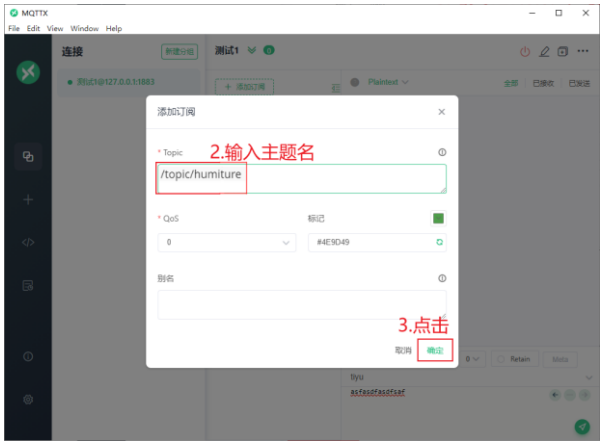
```
D:\abc\emqx-5.3.0-windows-amd64\bin\emqx    stop
```

3. 使用 MQTT 客户端

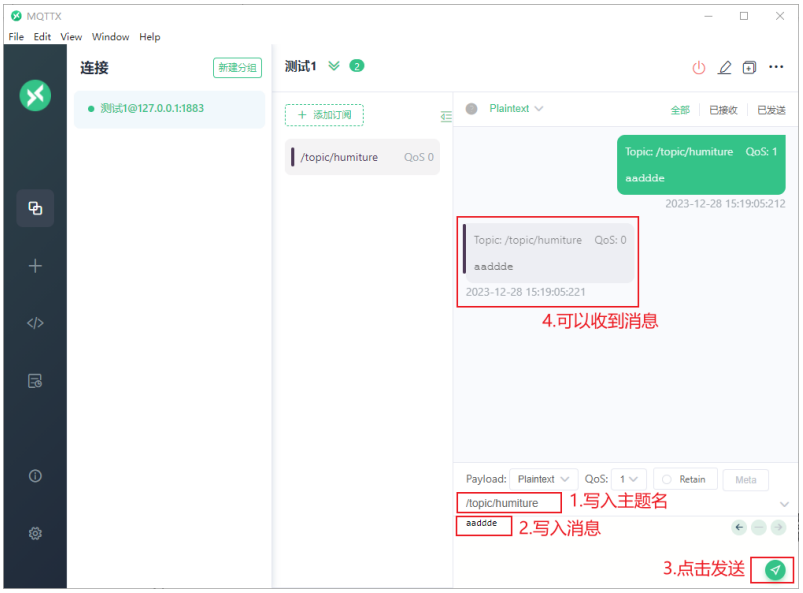
运行 MQTTX 后，如下图操作建立连接：



建立连接后，如下图操作订阅主题（比如“/topic/humiture”）：



然后，可以自己给自己发布消息，如下图操作：



6.3 微信小程序开发

6.3.1 任务描述

在腾讯云平台上，创建项目、新建产品、新建设备，以及配置云平台相关界面，通过 MQTT 客户端软件连接云平台，以 MQTT 订阅和发布方式，实现下发指令和上传数据功能。

6.3.2 微信云平台任务实施

1.登录腾讯物联网开发平台

地址：https://cloud.tencent.com/，如下图点击“登录”。



然后使用微信扫码登录：



点击控制台：

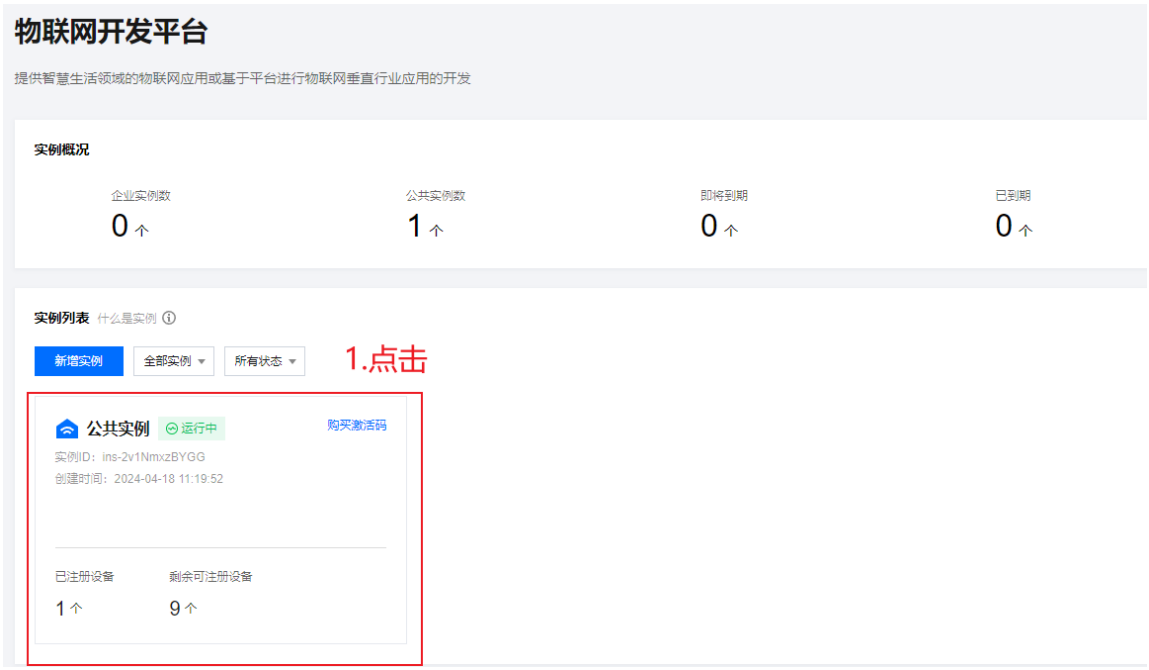


然后如下操作进入“物联网开发平台”：



2．创建腾讯云平台项目

（1）点击公共实例



（2）点击产品开发



3．在项目内新建产品

（1）点击，新建产品。



（2）选择产品品类、设置产品名称等信息，最后点击“新建产品”，如下图所示。



最后，可以看到如下产品：



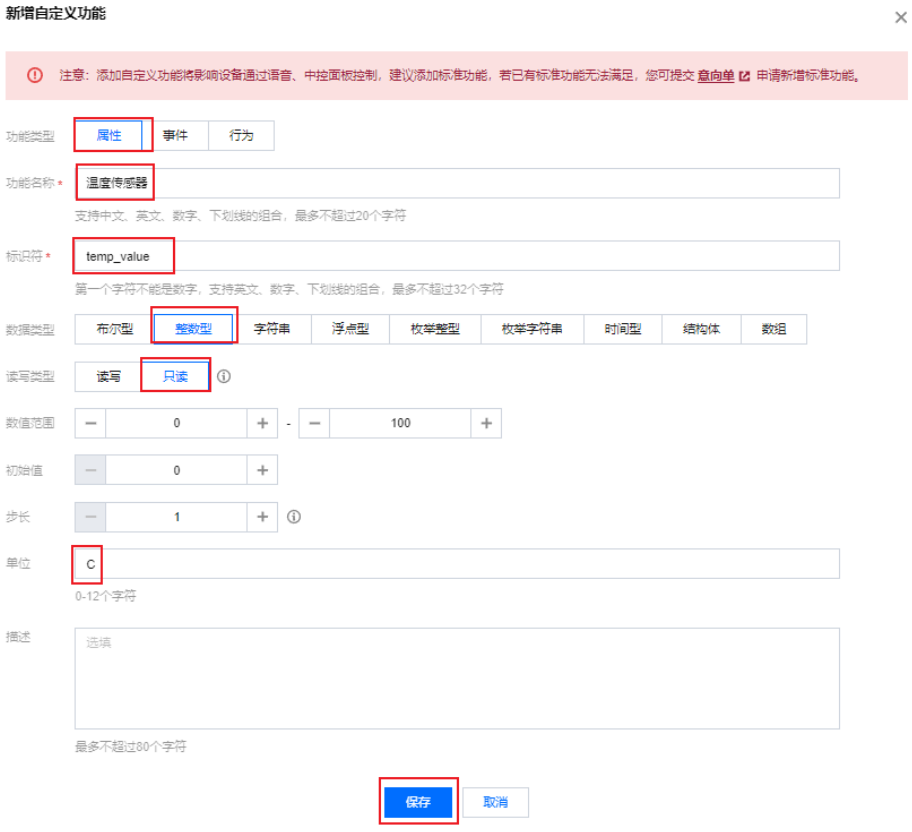
4．给产品添加温湿度功能

（1）在产品里添加自定义功能

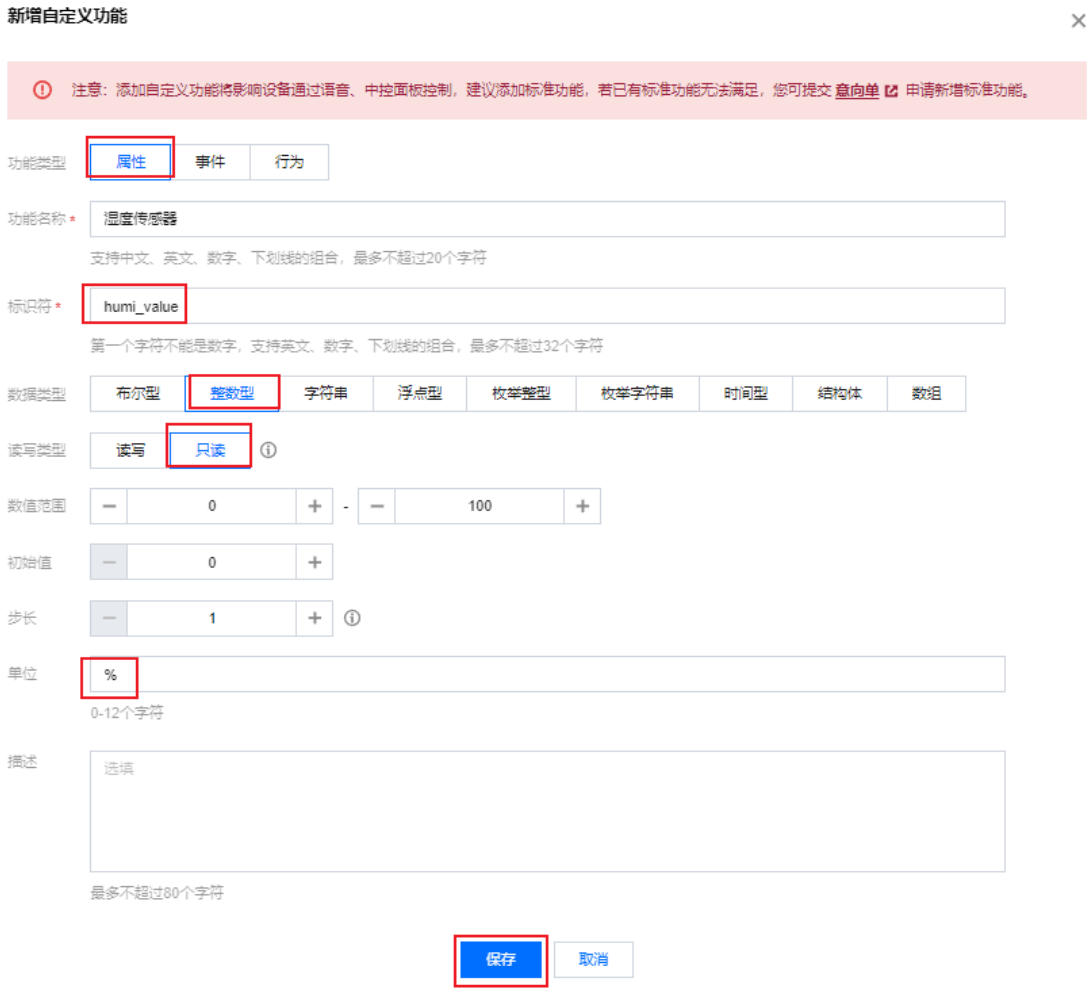




（2）设置温度功能的属性



（3）重复步骤（1）、（2），如下创建湿度功能：



效果如下：



5．给产品添加 LED 控制功能

（1）在产品里添加自定义功能



（2）添加“LED1”：



最后结果如下图所示：

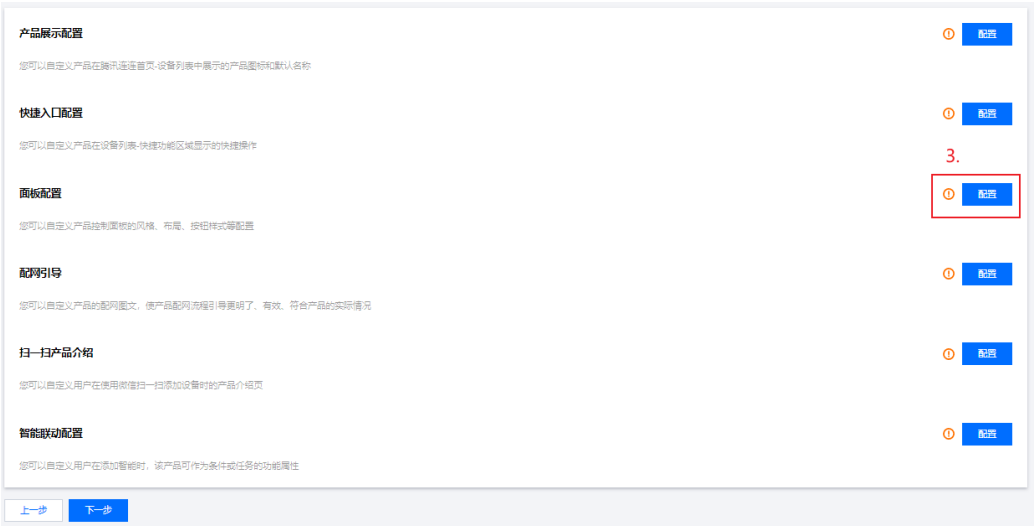


6．开发交互界面

点击“下一步”进入交互开发，配置小程序界面，如下操作：



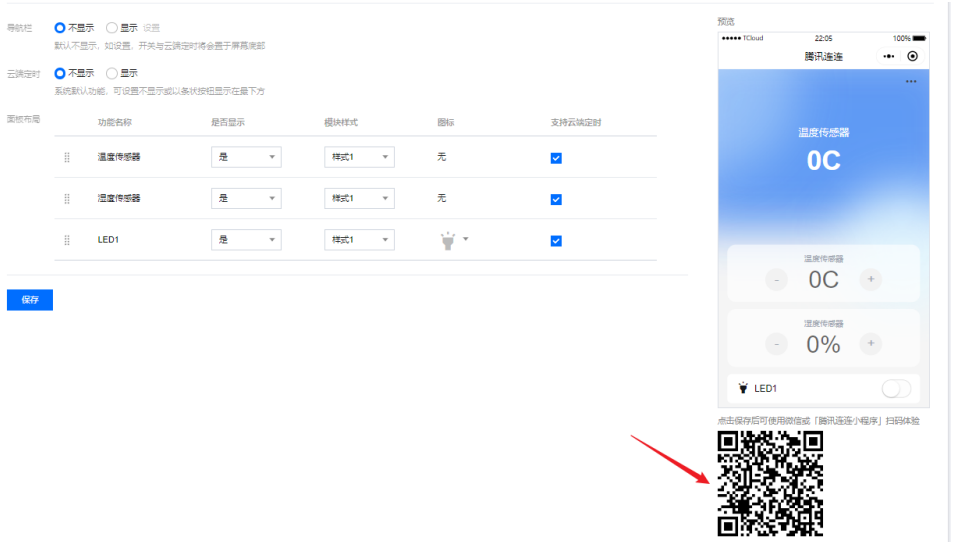




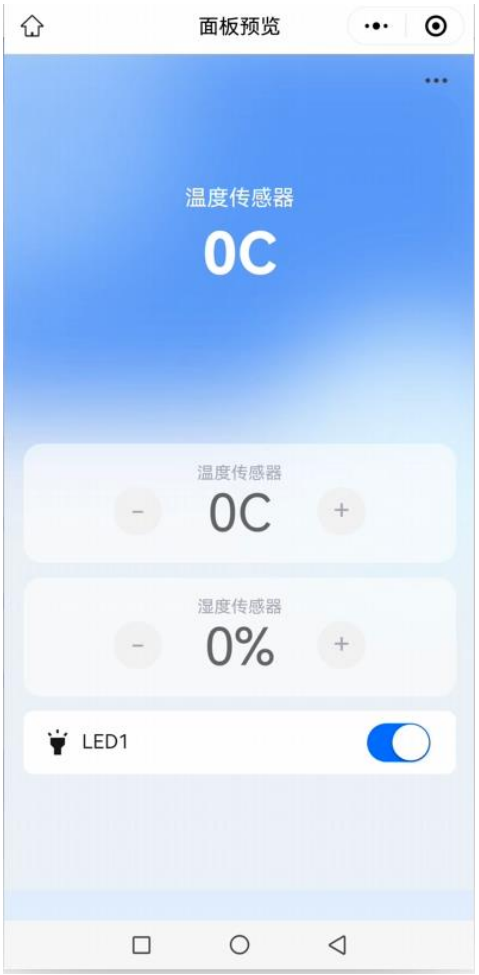

面板类型
标准面板
H5自定义面板
导航栏
不显示
显示
设置
默认不显示，如设置，开关与云端定时将会置于屏幕底部
云端定时
不显示
显示
系统默认功能，可设置不显示或以条状按钮显示在最下方
面板布局
功能名称
是否显示
模块样式
图标
支持云端定时
温度传感器
是
样式1
无
湿度传感器
是
样式1
无
LED1
是
样式1
4.
保存
5.

7．验证界面

使用微信扫码类似下图界面中的二维码：



在手机上可以看到类型下图的界面：

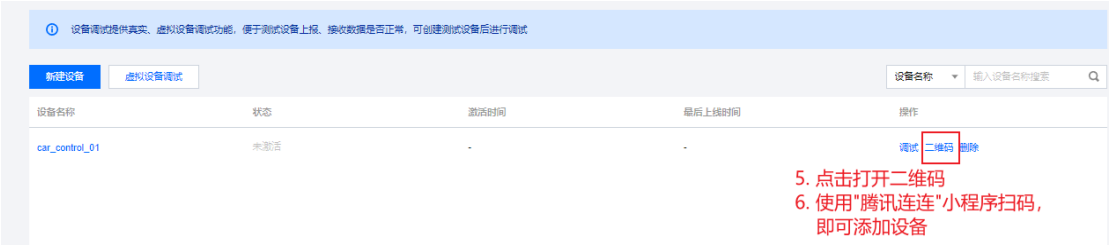


8．新建设备

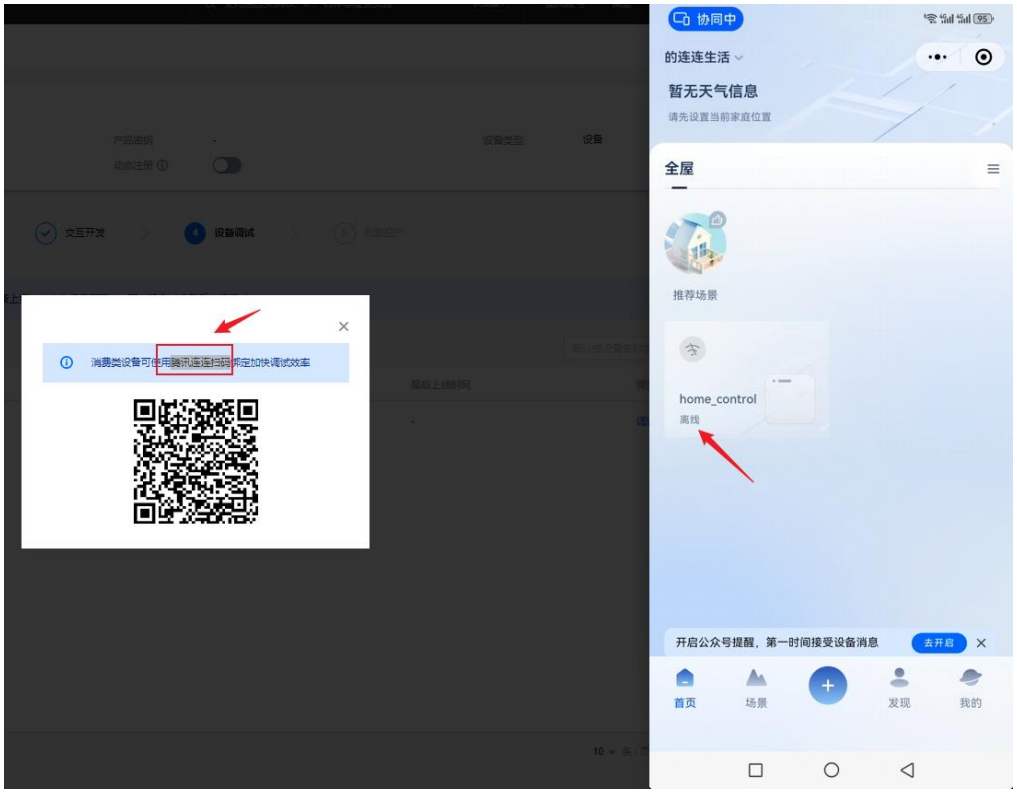
如下图操作：







在手机上，效果如下图所示：

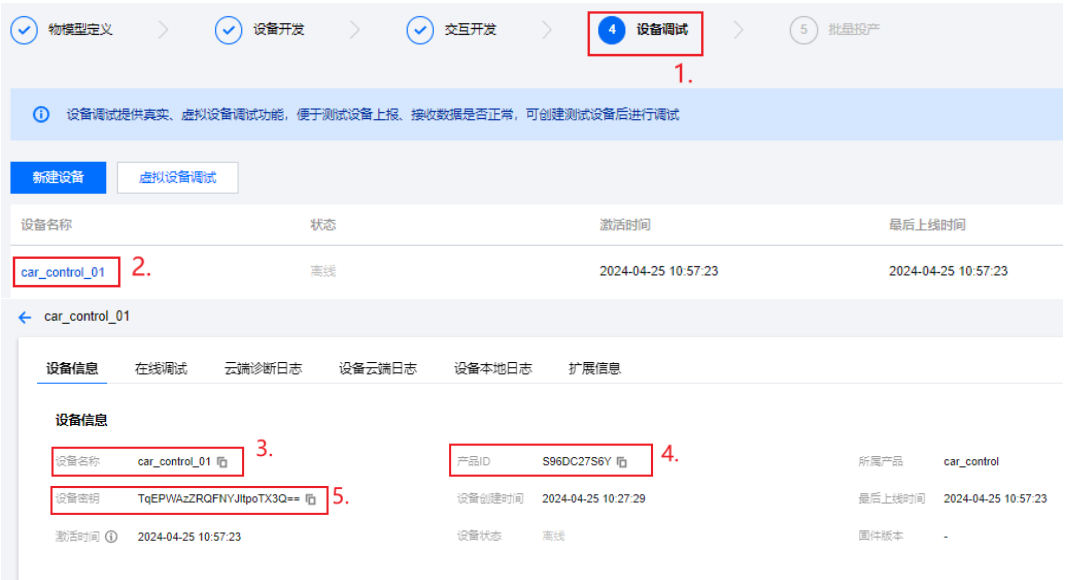


9. PC MQTT 连接

使用 PC MQTT 客户端软件 MQTTX 模拟一个设备，连接到小程序后，可以看到设备状态变为“在线”。使用 MQTTX 时，需要指定这几个参数：

① broker 地址
② client ID
③ 用户名
④ 密码

怎么生成上述 4 个信息？先记录设备信息，如下图操作：



记录上图 3、4、5 的信息，作为例子，我们记录如下：

① 设备名称：home_control_01

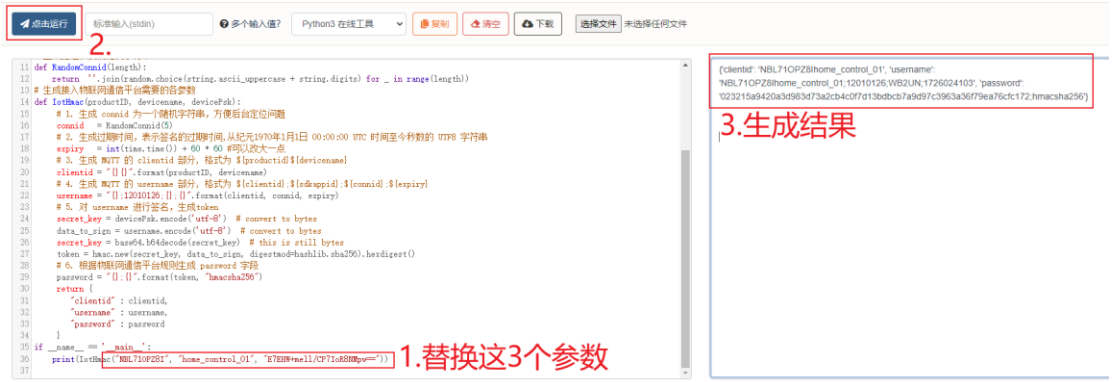
② 产品 ID：NBL710PZ8I
③ 设备秘钥：E7EHW+mell/CP7IoR8NMpw==

根据文档 https://cloud.tencent.com/document/product/1081/102803，可以知道：
① broker 地址为：${productId}.iotcloud.tencentdevices.com
所以，本设备的 boroker 地址为“NBL710PZ8I.iotcloud.tencentdevices.com”
② 生成 client ID、用户名、密码：
打开 Python 在线编辑器 https://www.jyshare.com/compile/9/， 输入如下代码，并把代码最后一行的“sys.argv[1] ”替换为产品 ID“NBL710PZ8I”，“sys.argv[2] ”替换为设备名称“ home_control_01 ”，“ sys.argv[3] ”替换为设备秘钥“E7EHW+mell/CP7IoR8NMpw==”。
注意：从 PDF 里复制以下代码容易出错，可以用记事本打开“01_all_series_quickstart\04a_嵌入式 Linux 应用开发实验班(快速入门)\06_智能家居产品设计\source\6-3_微信小程序开发\wechat.py”，再复制。

```
#!/usr/bin/python
# -*- coding: UTF-8 -*-
import base64
import hashlib
import hmac
import random
import string
import time
import sys
# 生成指定长度的随机字符串
def RandomConnid(length):
    return ''.join(random.choice(string.ascii_uppercase + string.digits) for _ in
range(length))
# 生成接入物联网通信平台需要的各参数
def IotHmac(productID, devicename, devicePsk):
    # 1. 生成 connid 为一个随机字符串，方便后台定位问题
    connid   = RandomConnid(5)
    # 2. 生成过期时间，表示签名的过期时间,从纪元1970年1月1日 00:00:00 UTC 时间至今秒数的
UTF8 字符串
    expiry   = int(time.time()) + 60 * 60 #可以改大一点
    # 3. 生成 MQTT 的 clientid 部分，格式为 ${productid}${devicename}
    clientid = "{}{}".format(productID, devicename)
    # 4. 生成 MQTT 的 username 部分，格式为 ${clientid};${sdkappid};${connid};${expiry}
    username = "{};12010126;{};{}".format(clientid, connid, expiry)
    # 5. 对 username 进行签名，生成token
    secret_key = devicePsk.encode('utf-8')  # convert to bytes
    data_to_sign = username.encode('utf-8')  # convert to bytes
    secret_key = base64.b64decode(secret_key)  # this is still bytes
    token = hmac.new(secret_key, data_to_sign, digestmod=hashlib.sha256).hexdigest()
    # 6. 根据物联网通信平台规则生成 password 字段
    password = "{};{}".format(token, "hmacsha256")
    return {
```

```
        "clientid" : clientid,
        "username" : username,
        "password" : password
    }
if __name__ == '__main__':
    print(IotHmac(sys.argv[1], sys.argv[2], sys.argv[3]))
```

操作如下图所示：



生成了如下信息：

① clientid：

```
NBL71OPZ8Ihome_control_01
```

② 用户名：

```
NBL71OPZ8Ihome_control_01;12010126;WB2UN;1726024103
```

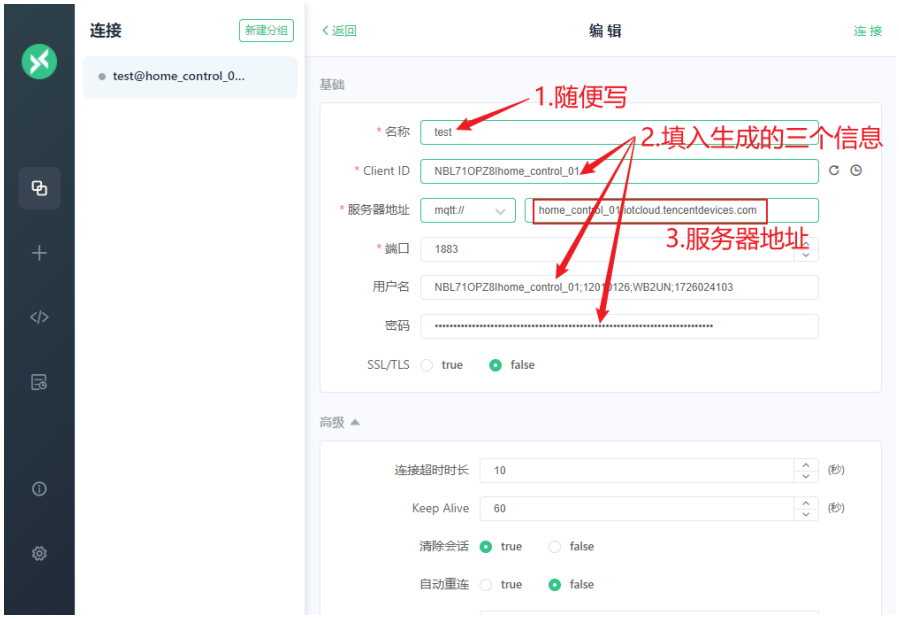
③ 密码：

```
023215a9420a3d983d73a2cb4c0f7d13bdbcb7a9d97c3963a36f79ea76cfc172;hmacsha256
```

④ 服务器地址：${productId}替换为产品 ID

```
${productId}.iotcloud.tencentdevices.com
```

现在，可以打开 MQTT 客户端软件，填写 broker、clientid、用户名、密码，然后连接：



最后，在手机上的“腾讯连连”小程序上，点击右上角“…”，然后再点“重新进入小程序”。发现设备“离线”字样消失，变为“在线”。





10. MQTT 消息订阅和发布

接下来就是在 PC MQTT 客户端订阅主题、发布主题。格式可以参考：https://cloud.tencent.com/document/product/1081/34916，总结如下：

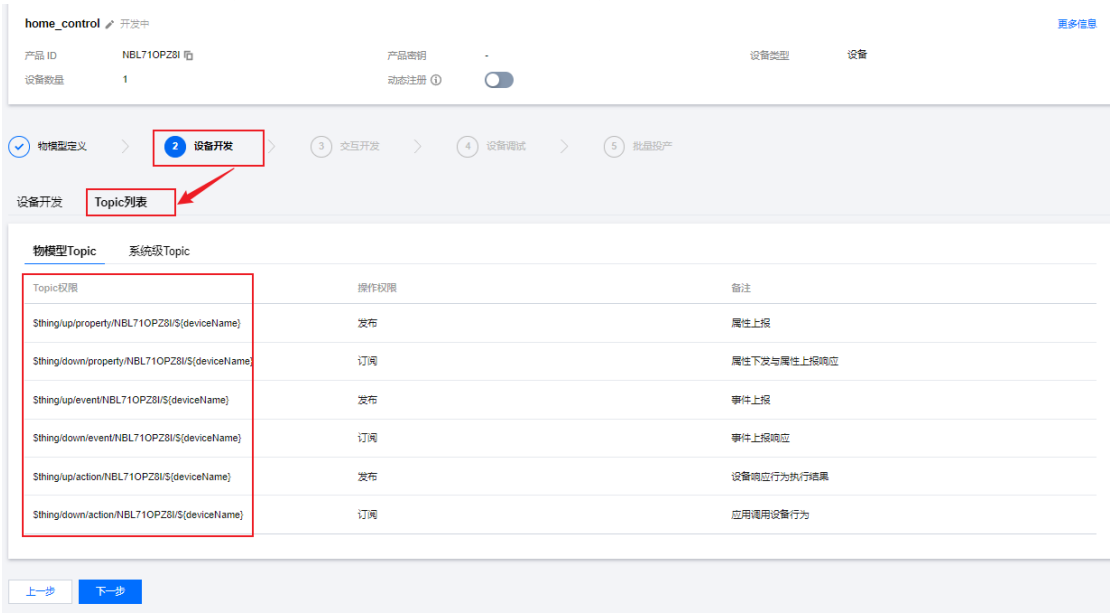
① 本地接收（订阅）云端的信息：

```
$thing/down/property/{ProductID}/{DeviceName}
```

② 本地发送（发布）信息给云端：

```
$thing/up/property/{ProductID}/{DeviceName}
```

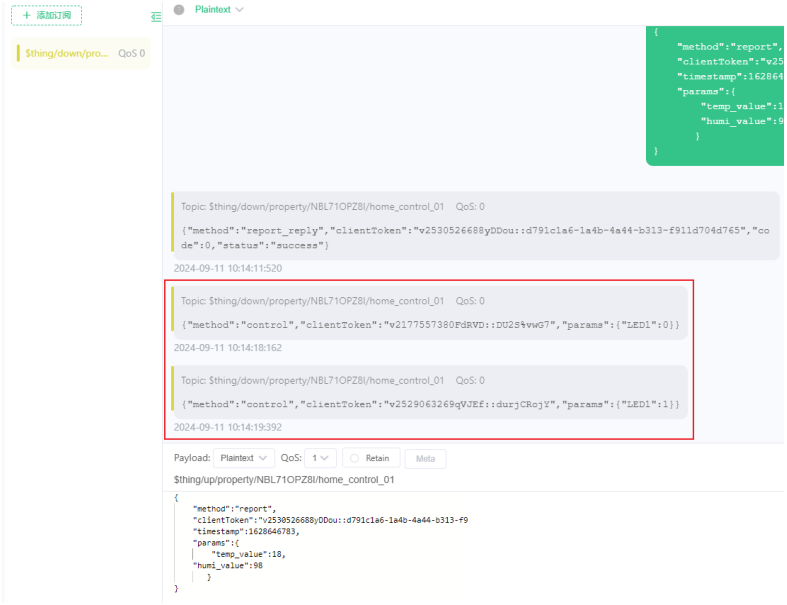
可以在云平台看到如下界面，对应 PC 客户端能发布的主题、能订阅的主题：



在 PC MQTT 客户端软件 MQTTX 中订阅主题：

```
$thing/down/property/NBL71OPZ8I/home_control_01
```

然后在小程序上点击 LED 按钮，可以看到如下图所示的信息：



信息格式为：

```
{"method":"control","clientToken":"v2177557380FdRVD::DU2S%vwG7","params":{"LED1":0}}
```

在 PC MQTT 客户端软件 MQTTX 中发布消息：

① 主题为：

```
$thing/up/property/NBL710PZ8I/home_control_01
```

② 消息为（上报温度为 28 度、湿度 98%）：

```
{
    "method":"report",
    "clientToken":"v2530526688yDDou::d791c1a6-1a4b-4a44-b313-f911d704d765",
    "timestamp":1628646783,
    "params":{
        "temp_value":28,
         "humi_value":98
       }
}
```

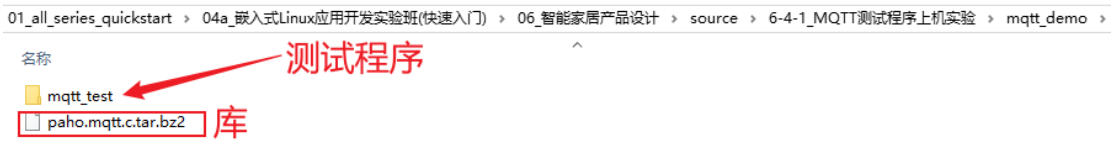
可以在微信小程序看到如下效果：


协同中
home_control
温度传感器
18C
温度传感器
18C
湿度传感器
98%
LED1

6.4 设备端程序开发

6.4.1 MQTT 测试程序上机实验

本节源码为：



把 mqtt_test、paho.mqtt.c.tar.bz2 放到 Ubuntu 上同一个目录下。

1. 编译 paho 库

在 Ubuntu 下执行如下命令：

```
$ tar xjf paho.mqtt.c.tar.bz2
$ cd  paho.mqtt.c
```

修改 CMakeLists.txt，在最前面增加如下内容：

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)
SET(CMAKE_C_COMPILER  /home/book/100ask_imx6ull-sdk/ToolChain/arm-build
                  root-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-
                  linux-gnueabihf-gcc)
SET(CMAKE_CXX_COMPILER  /home/book/100ask_imx6ull-sdk/ToolChain/arm-bui
                  ldroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroo
                  t-linux-gnueabihf-g++)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
```

注意：从 PDF 复制一定会出错，请人眼检查。

然后执行如下命令编译：

```
$ mkdir build
$ cd build
$ cmake ../ -DCMAKE_INSTALL_PREFIX=output
$ make
$ make install
```

如果一切顺利，会在 build 目录下生成一个 output 目录（有些错误信息提示权限问题，无关紧要），里面有很多库文件。内容如下：

```
$ ls output/
bin  include  lib  share
```

把编译得到的库文件，放到开发板上：

```
$ cd output/lib
$ adb push libpaho-mqtt3* /lib
```

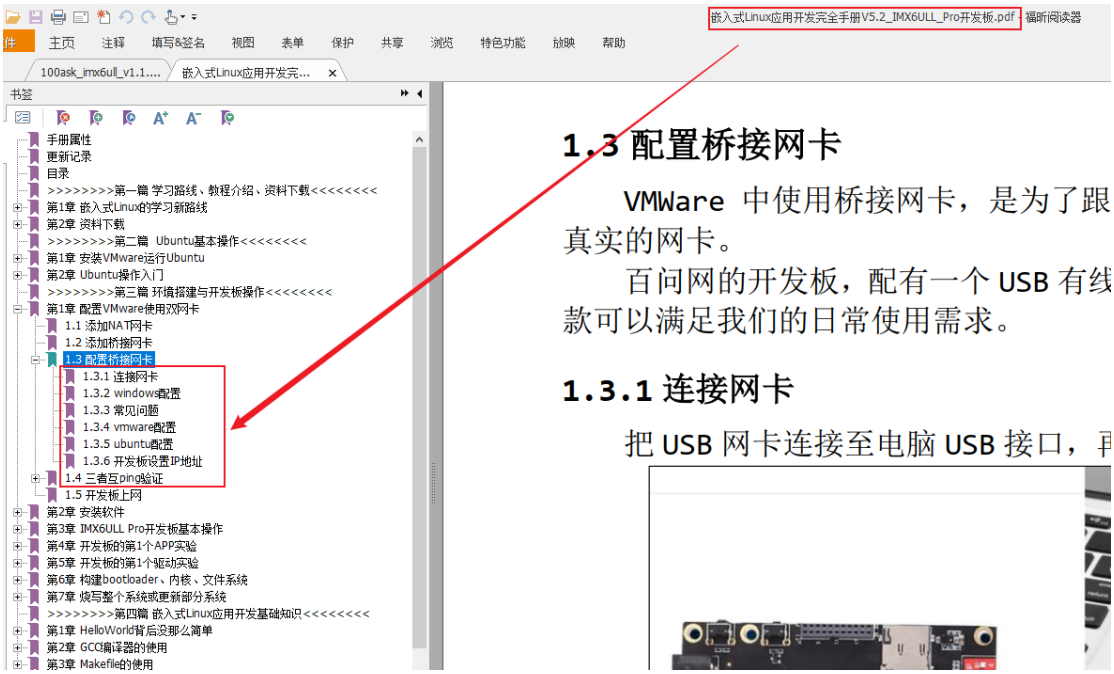
2. 编译测试程序

在 Ubuntu 上执行如下命令：

```
$ cd mqtt_test
$ make
$ adb  push  mqtt_test  /root
```

3．确保开发板和 PC 网络互通

参考手册：

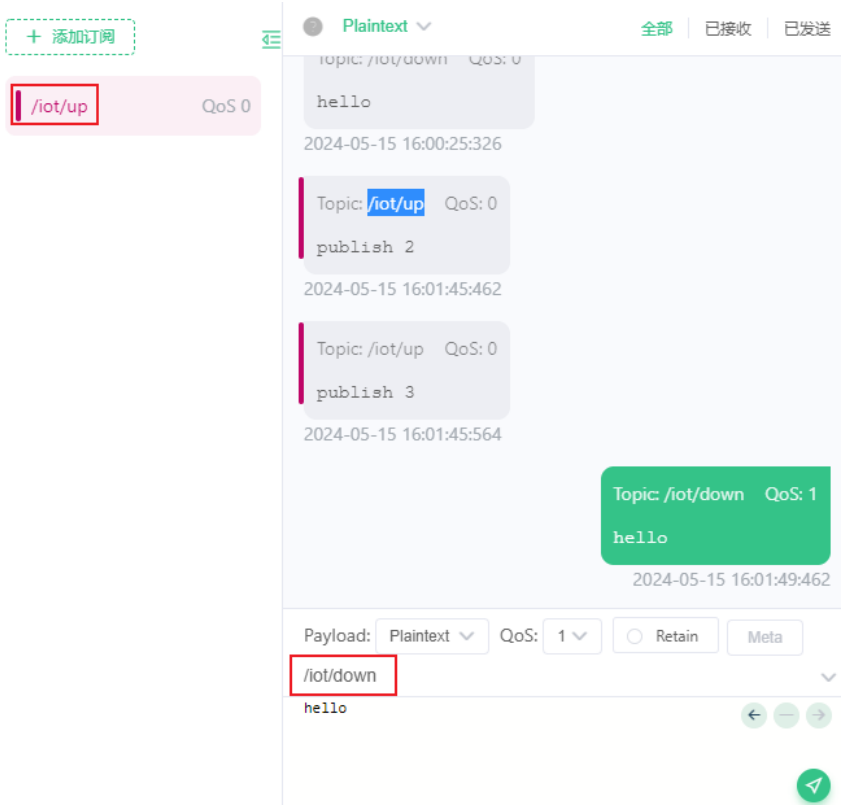


4．测试

按照《6.2.3 PC MQTT 工具使用》，先启动 MQTT 服务软件，然后再启动 MQTTX，如下配置、连接：



在 MQTTX 里订阅主题为“/iot/up”的消息，发送主题为“/iot/down”的消息，如下图所示：



最后，在开发板上执行 mqtt_test 程序，如下：

```
[root@100ask:~]#
[root@100ask:~]# ./mqtt_test 192.168.5.10
Subscribing to topic /iot/down
for client ExampleClientSub using QoS1

Press Q<Enter> to quit

1
Message with token value 2 delivery confirmed
Message with delivery token 2 delivered
Message with token value 3 delivery confirmed
Message with delivery token 3 delivered
Message arrived
     topic: /iot/down
   message: www.100ask.net
```

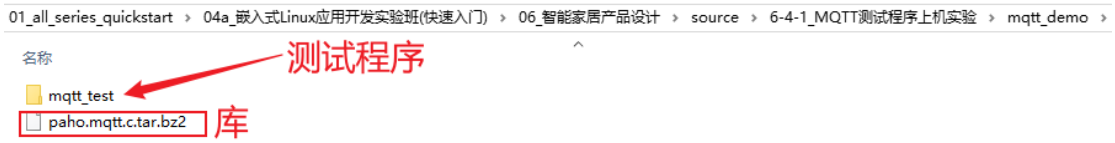
windows IP

1.输入任意字母并回车，会发送消息

2.在PC上发布消息，这里会收到消息

6.4.2 Paho mqtt 源码分析

本节源码为：



paho.mqtt.c.tar.bz2 是库、示例代码。
mqtt_test 是本节对应的源码。
测试程序“mqtt_test.c”核心源码如下：

1. 定义一些参数

```
22 #define ADDRESS     "tcp://192.168.5.10:1883"
23 #define CLIENTID    "ExampleClientSub"
24 #define TOPIC_SUBSCRIBE     "/iot/down"
25 #define TOPIC_PUBLISH       "/iot/up"
26 #define QOS         1
27 #define TIMEOUT     10000L
28
29 #define USER_NAME "100ask"
30 #define PASSWORD  "100ask"
```

2. 连接 broker

代码如下：

```
57 int main(int argc, char* argv[])
58 {
59     MQTTClient client;
60     MQTTClient_connectOptions conn_opts = MQTTClient_connectOptions_
                initializer;
61     int rc;
62
63     if ((rc = MQTTClient_create(&client, ADDRESS, CLIENTID,
64         MQTTCLIENT_PERSISTENCE_NONE, NULL)) != MQTTCLIENT_SUCCESS)
65     {
66         printf("Failed to create client, return code %d\n", rc);
67         rc = EXIT_FAILURE;
68         goto exit;
69     }
70
71     if ((rc = MQTTClient_setCallbacks(client, NULL, connlost, msgarr
                vd, delivered)) != MQTTCLIENT_SUCCESS)
72     {
73         printf("Failed to set callbacks, return code %d\n", rc);
74         rc = EXIT_FAILURE;
75         goto destroy_exit;
76     }
77
78     conn_opts.keepAliveInterval = 20;
79     conn_opts.cleansession = 1;
80     conn_opts.username = USER_NAME;
81     conn_opts.password = PASSWORD;
82     if ((rc = MQTTClient_connect(client, &conn_opts)) != MQTTCLIENT_
                SUCCESS)
83     {
```

```
84          printf("Failed to connect, return code %d\n", rc);
85          rc = EXIT_FAILURE;
86          goto destroy_exit;
87      }
```

第 63 行，创建一个客户端。

第 71 行，设置回调函数：连接断开时的回调函数、收到消息的回调函数、消息发布成功后的回调函数。

第 78~81 行，设置用户名、密码。

第 82 行，连接 broker。

3．订阅消息

代码如下：

```
91      if ((rc = MQTTClient_subscribe(client, TOPIC_SUBSCRIBE, QOS)) !=
                    MQTTCLIENT_SUCCESS)
92      {
93       printf("Failed to subscribe, return code %d\n", rc);
94       rc = EXIT_FAILURE;
95      }
```

当接收到订阅的消息时，前面第 71 行设置的“收到消息的回调函数”被调用，代码如下：

```
40 int msgarrvd(void *context, char *topicName, int topicLen, MQTTClien
                   t_message *message)
41 {
42      printf("Message arrived\n");
43      printf("     topic: %s\n", topicName);
44      printf("   message: %.*s\n", message->payloadlen, (char*)message
                   ->payload);
45      MQTTClient_freeMessage(&message);
46      MQTTClient_free(topicName);
47      return 1;
48 }
```

这个函数仅仅是把消息打印出来。

4．发布消息

代码如下：

```
98              int ch;
99              int cnt = 0;
100             MQTTClient_message pubmsg = MQTTClient_message_initializ
                    er;
101             char buf[100];
102             MQTTClient_deliveryToken token;
103
104              while (1)
105              {
106              ch = getchar();
107
108              if (ch =='Q' || ch == 'q')
109              {
110                  break;
111              }
112
```

```
113             sprintf(buf, "publish %d", cnt++);
114             pubmsg.payload = buf;
115             pubmsg.payloadlen = (int)strlen(buf);
116             pubmsg.qos = QOS;
117             pubmsg.retained = 0;
118
119             if ((rc = MQTTClient_publishMessage(client, TOPIC_PUBLI
                  SH, &pubmsg, &token)) != MQTTCLIENT_SUCCESS)
120             {
121                 printf("Failed to publish message, return code %d\
                  n", rc);
122                 continue;
123             }
124
125             rc = MQTTClient_waitForCompletion(client, token, TIMEOU
                  T);
126             printf("Message with delivery token %d delivered\n", to
                  ken);
127         }
```

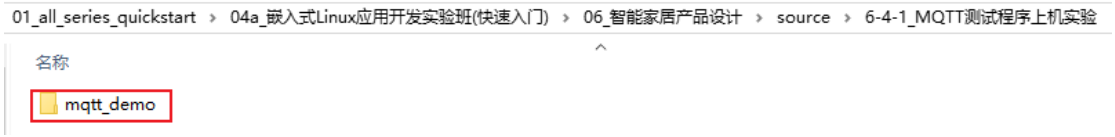
第 100、113~117 行：设置消息。消息保存在 buf 里。

第 119 行：发布消息。

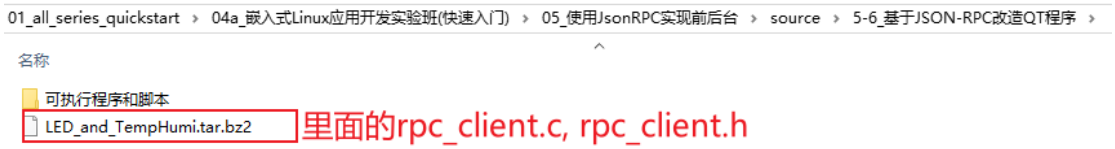
第 125 行：等待消息发送完毕。

6.4.3 MQTT 设备程序编写

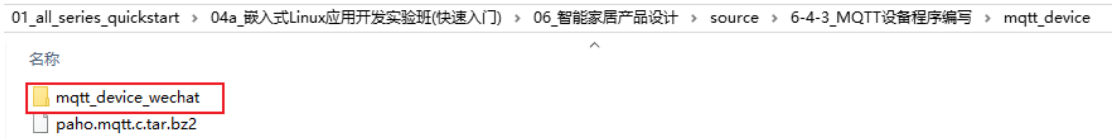
修改 MQTT 测试程序，连接微信小程序、提供对应的密码、用户名：



在里面添加硬件操作代码，参考：

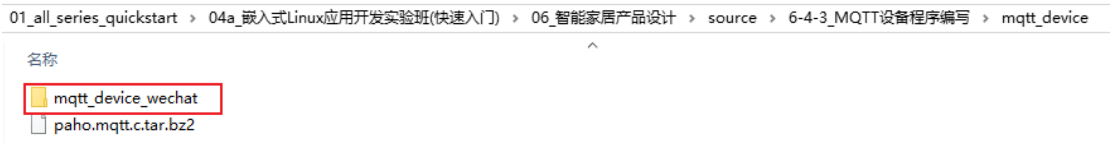


最终代码为：



6.4.4 编译程序

代码为：



1．编译 paho 库

在 Ubuntu 下执行如下命令：

```
$ tar xjf paho.mqtt.c.tar.bz2
$ cd  paho.mqtt.c
```

修改 CMakeLists.txt，在最前面增加如下内容：

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR arm)
SET(CMAKE_C_COMPILER  /home/book/100ask_imx6ull-sdk/ToolChain/arm-build
                  root-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-
                  linux-gnueabihf-gcc)
SET(CMAKE_CXX_COMPILER  /home/book/100ask_imx6ull-sdk/ToolChain/arm-bui
                  ldroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroo
                  t-linux-gnueabihf-g++)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
```

注意：从 PDF 复制一定会出错，请人眼检查。

然后执行如下命令编译：

```
$ mkdir build
$ cd build
$ cmake ../ -DCMAKE_INSTALL_PREFIX=output
$ make
$ make install
```

如果一切顺利，会在 build 目录下生成一个 output 目录（有些错误信息提示权限问题，无关紧要），里面有很多库文件。内容如下：

```
$ ls output/
bin  include  lib  share
```

把编译得到的库文件，放到开发板上：

```
$ cd output/lib
$ adb push libpaho-mqtt3* /lib
```

2．编译测试程序

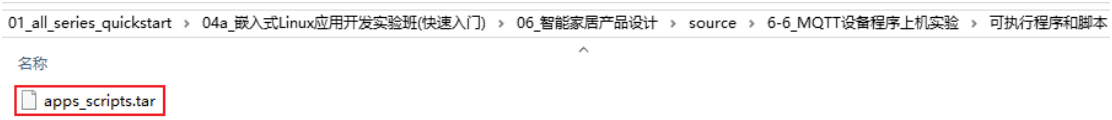
在 Ubuntu 上执行如下命令：

```
$ cd mqtt_device_wechat
$ make
$ adb  push  mqtt_device_wechat  /root
```

6.4.5 上机实验

1．给开发板添加文件

事先编译好的程序放在如下目录，解压后可以直接使用：



你可以根据教程自己编译程序、放到板子上，也可以直接使用上述文件。把上述文件放到开发板去后，在板子上执行如下命令：

```
# 移除原来的 GUI(提示 lvgl 不存在也不用理会)
mv /etc/init.d/*lvgl* /root
mv /etc/init.d/*hmi* /root

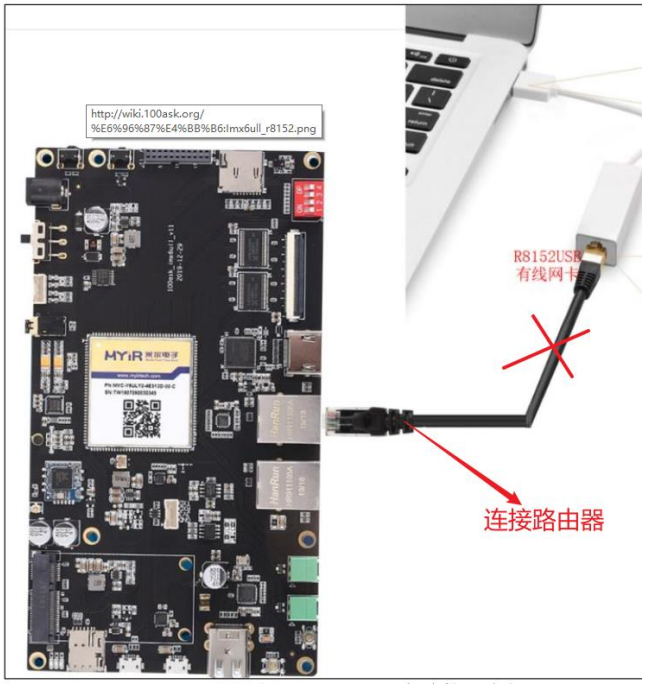
# 移除原来的驱动(提示文件不存在也不用理会)
mv /etc/init.d/S09modload /root

# 解压、复则文件到开发板
tar xf apps_scripts.tar
cd apps_scripts
cp * -rfd /

# 添加可执行权限
chmod +x /root/rpc_server
chmod +x /root/LED_and_TempHumi
chmod +x /root/mqtt_device_wechat
chmod +x /etc/init.d/rcS
chmod +x /etc/init.d/S99myqt
```

2．连接外网

我们要访问外网，一定一定一定不再使用 USB 网卡，开发板的网口要用网线连接到路由器：



然后配置开发板使用 DHCP，修改“/etc/network/interfaces”，内容如下：

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

重启开发板，确定能 ping 通外网：

```
ping www.100ask.net
```

3．修改配置文件

根据《6.3.2 微信云平台任务实施》之《9. PC MQTT 连接》重新生成用户名、密码登信息（之前生成的可能已经过期无法使用），写入开发板的“/etc/wechat.cfg”文件，内容示例如下：

```
{ "URI":"NBL71OPZ8I.iotcloud.tencentdevices.com",
 "clientid" : "NBL71OPZ8Ihome_control_01",
 "username" : "NBL71OPZ8Ihome_control_01;12010126;WB2UN;1726024103",
 "password" : "023215a9420a3d983d73a2cb4c0f7d13bdbcb7a9d97c3963a36f79ea
                  76cfc172;hmacsha256"
 "ProductID": "NBL71OPZ8I",
 "DeviceName": "home_control_01"
}
```

4．运行程序

在开发板上运行程序：

```
/root/mqtt_device_wechat
```

即可通过微信小程序操作开发板了。